

DTCP Firmware Updates

Dmytro Stavskyi

01/21/2026

Objective

- Be able to communicate with the AFE chip using SPI (receive any valid data from the chip, e.g., register values)

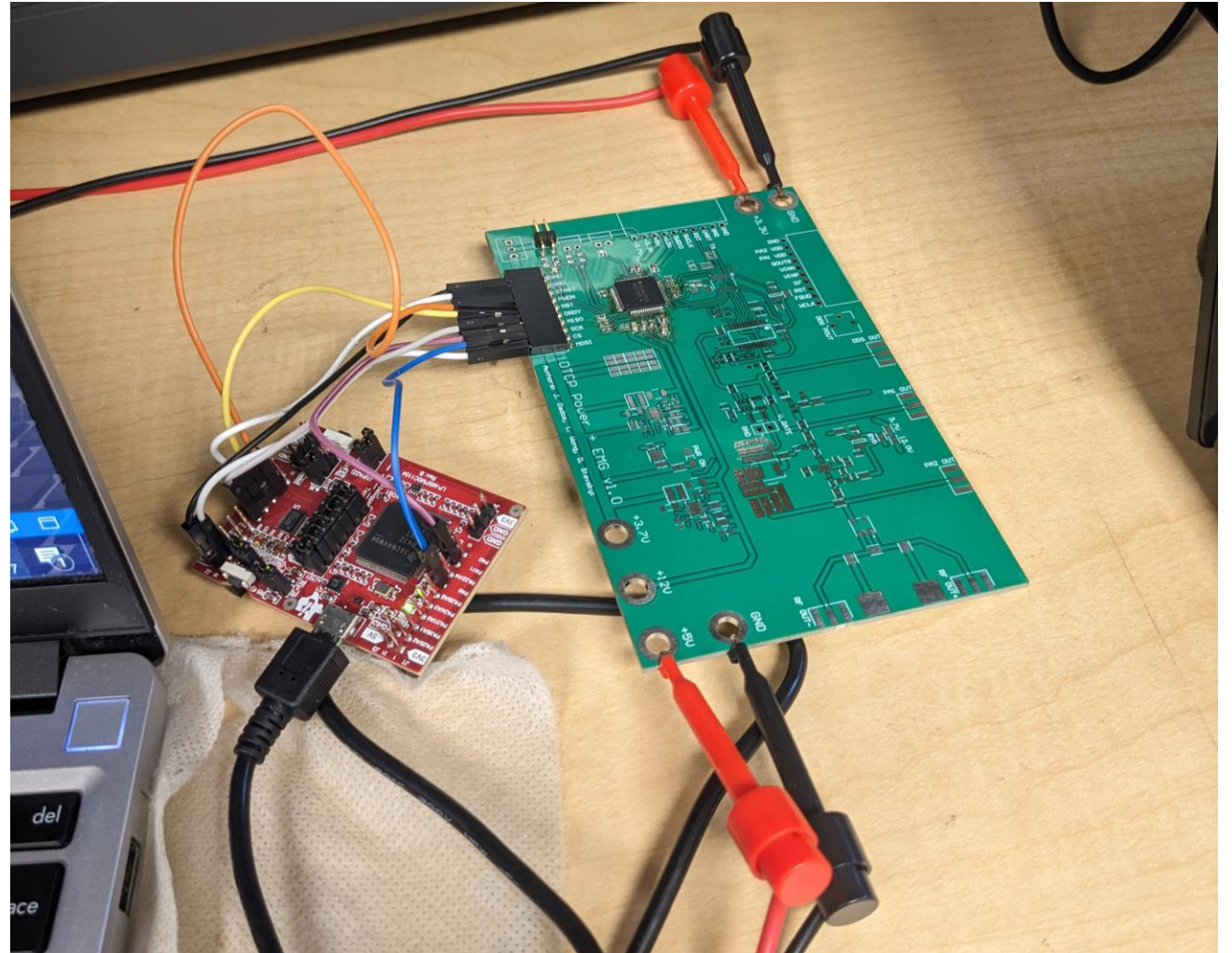
Methods

Assemble a separate board with only AFE in it for component isolation

Use the debug probs and the LaunchPad to program the chip

Measure the SCLK, CS and MOSI / MISO using oscilloscope

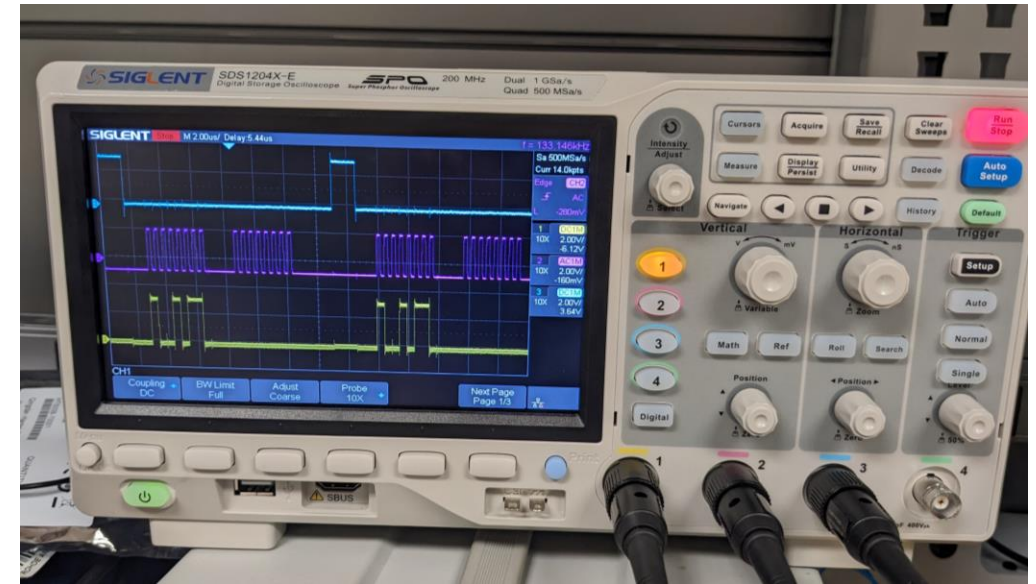
Measure the current consumption on the power supply



Results

At this point, we can write to registers and read from them and receive proper values

Due to incorrect configuration for one pin, the current consumption was 35 mA, but after the fix, it went back to 1~2 mA



Observation / Conclusion

- In RDATA (Read Data Continuous) mode, you cannot read or write to registers
 - After issuing that mode using SPI, trying to read a register yields nothing, but DRDY (Data Ready) is never asserted
- Possible sources of errors: incorrect values for configuration registers

Next Steps

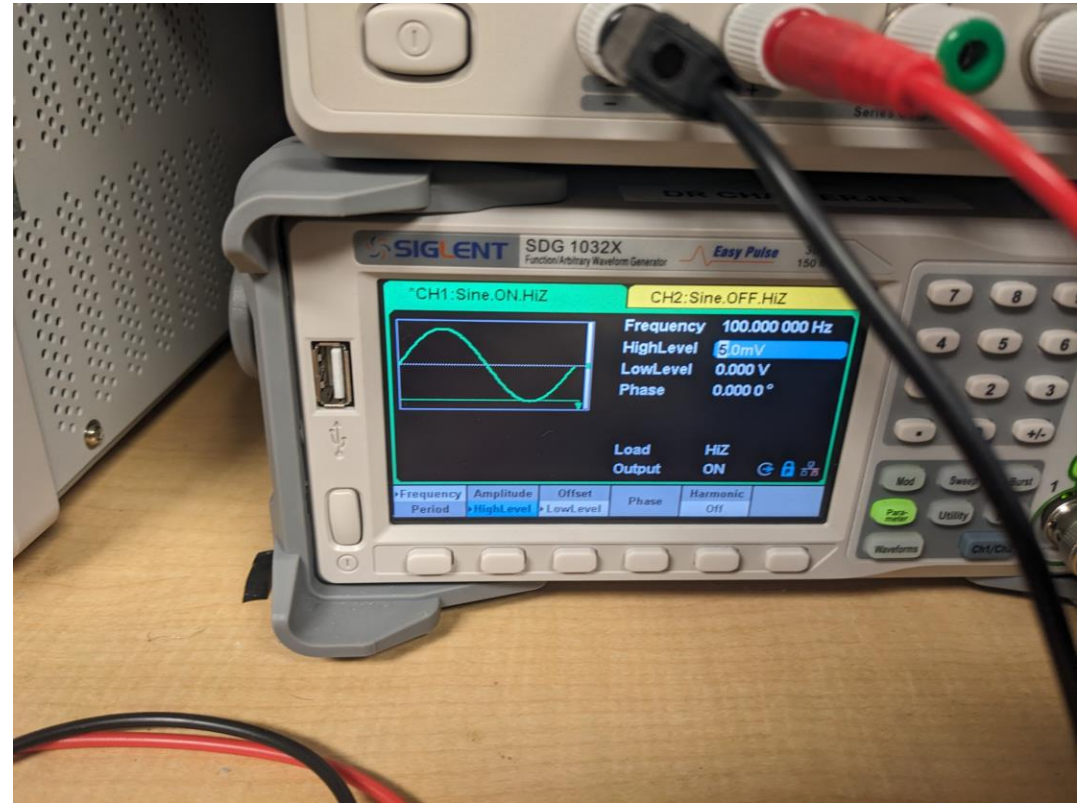
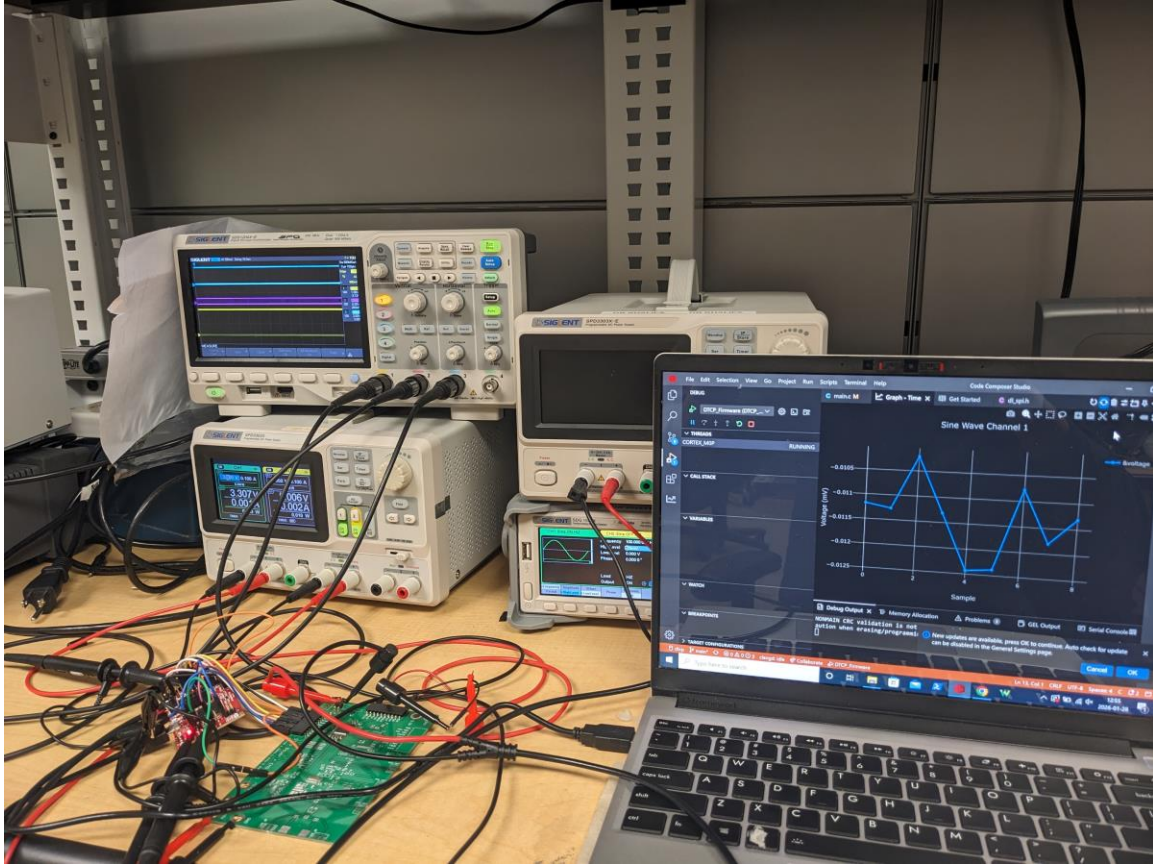
- Get conversion data from the analog channels

01/28/2026

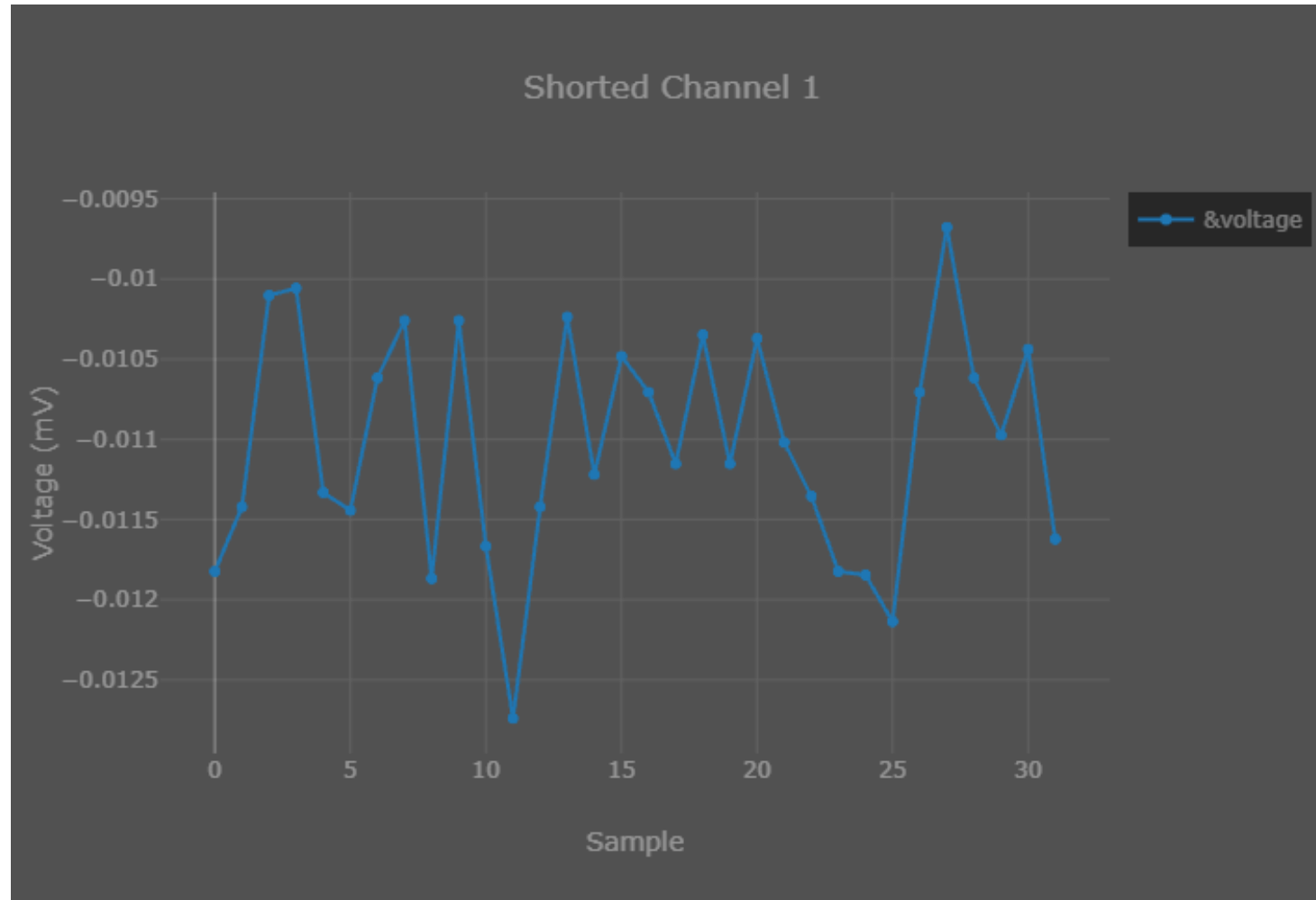
Objective

- Get conversion data from the analog channels

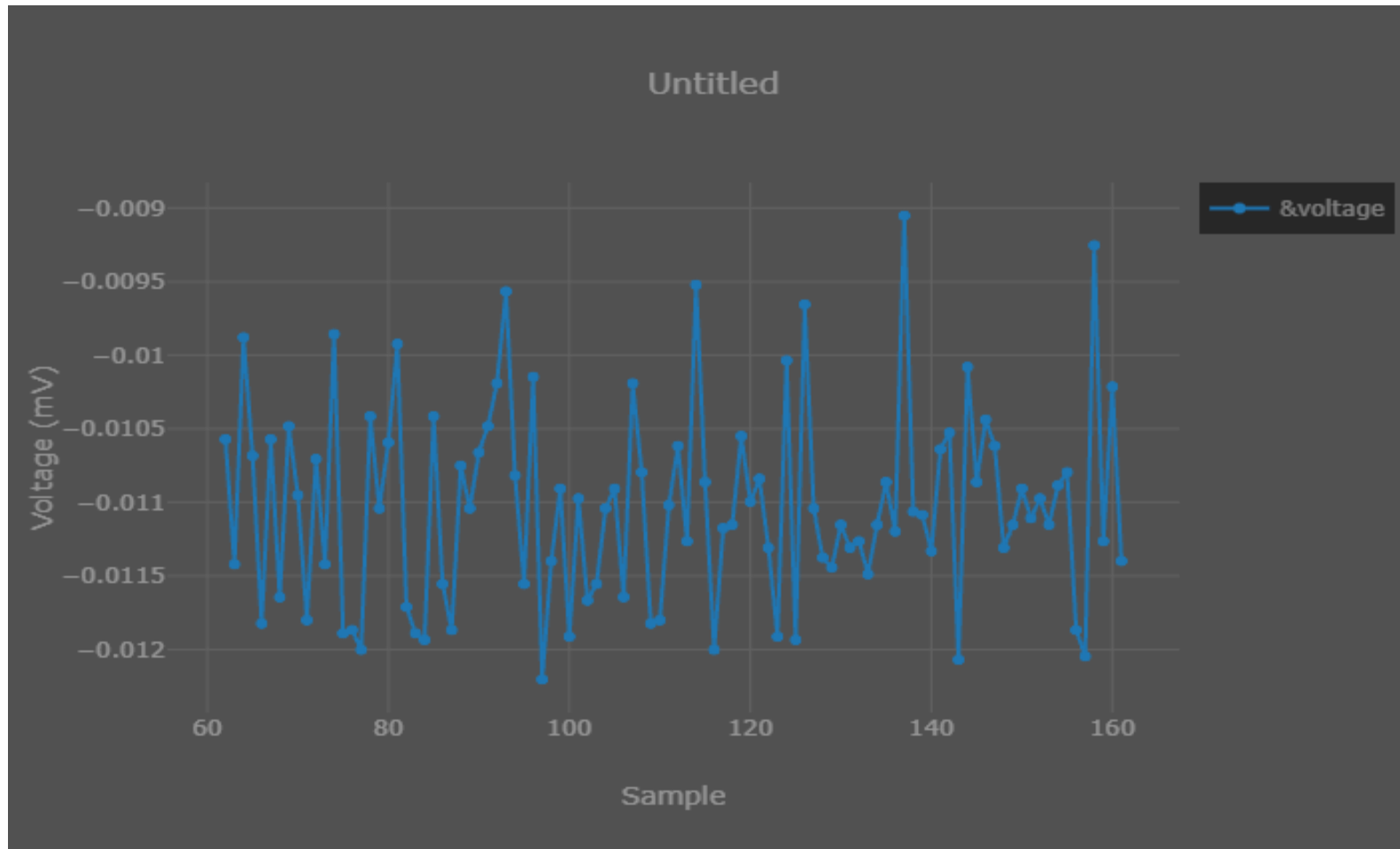
Methods



Results (shorted inputs; nominal sampling rate of 1000)



Results (100 Hz 5 mV sine wave;
nominal sampling rate of 1000)



Observation / Conclusion

- There were timing issues with the SPI, but adding some delays fixed the issues
- The first 24 bits are status bits, and they are what they should be, so we can assume that we are reading the data correctly

Next steps

- Improve firmware (interrupts, etc.)
- Test the conversion with different configuration values

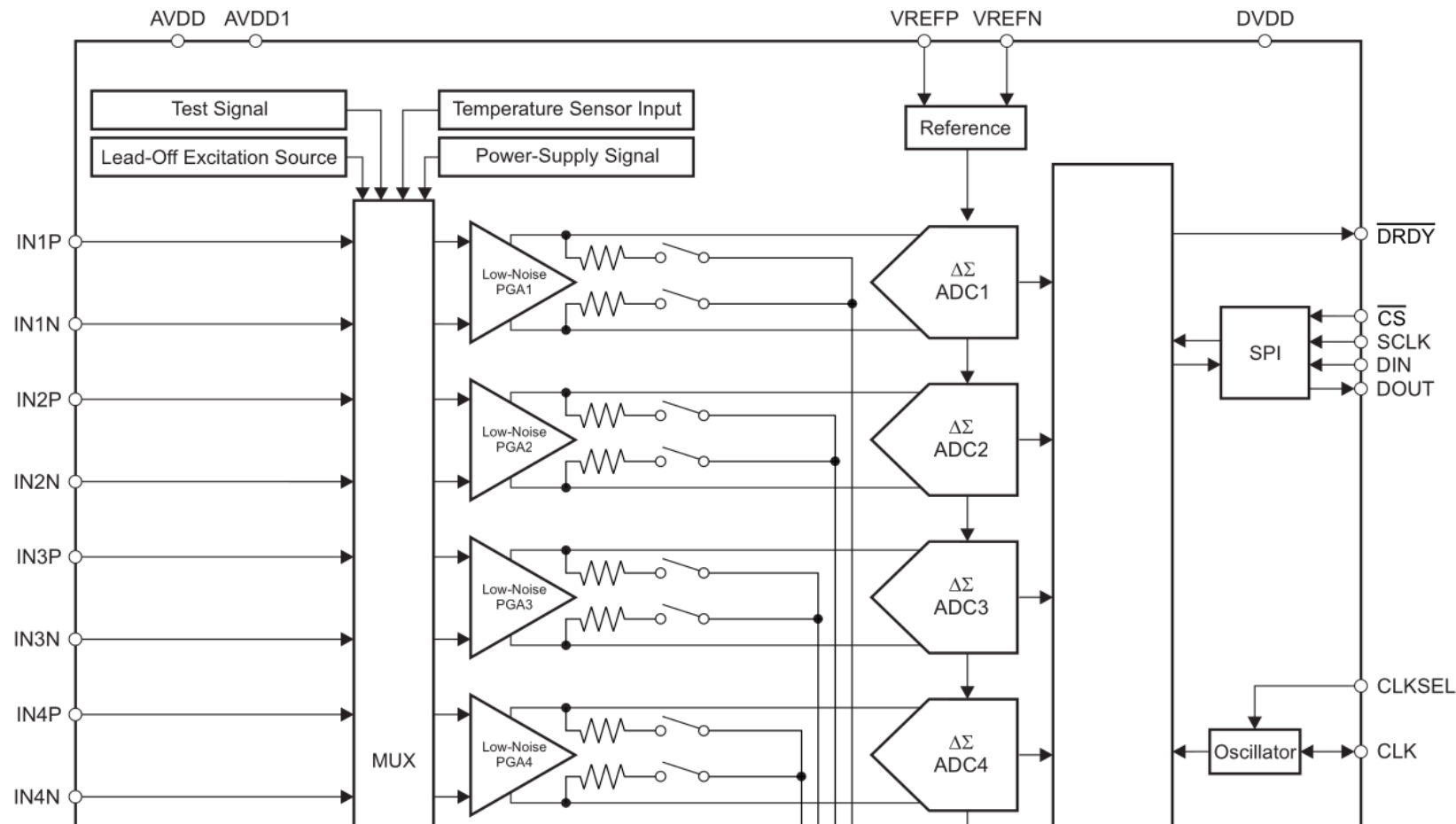
02/04/2026

Objective

- Determine if the chip can properly process any data
- Measure a wave provided by the function generator (e.g., 10 mV 100 Hz sine)

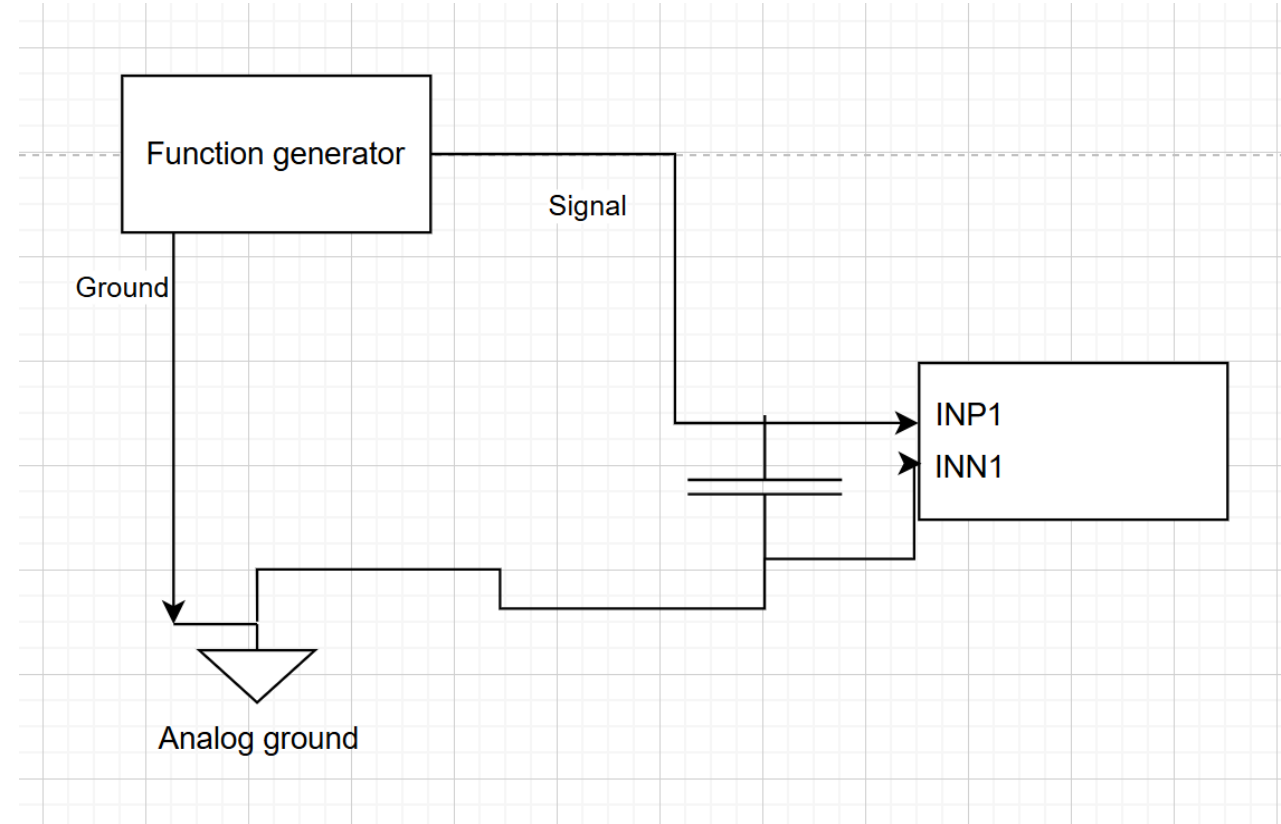
Methods

9.2 Functional Block Diagram

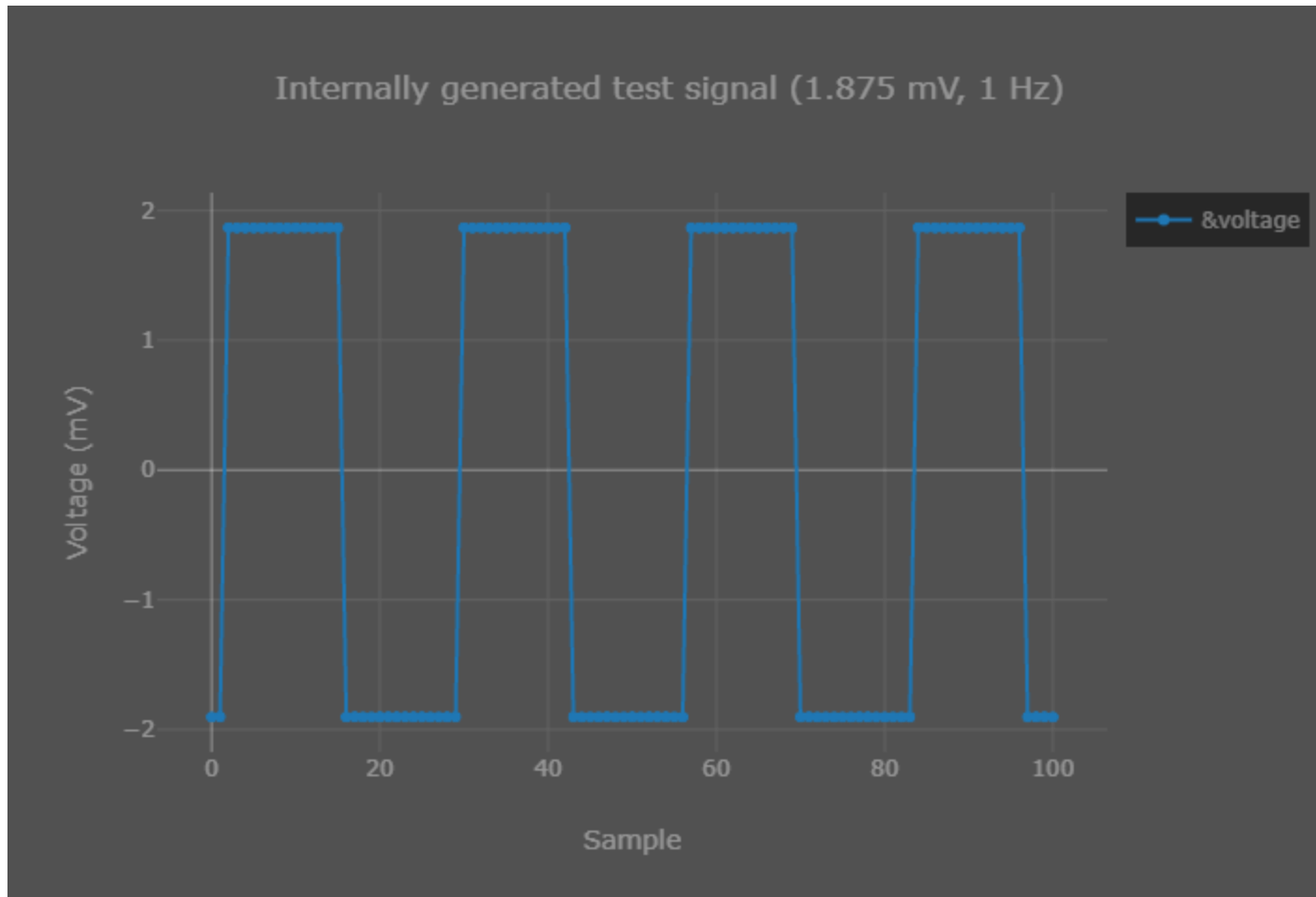


You can route the input to the channel from the normal external terminals to some other source like a test signal or a power-supply signal using a MUX

Methods



Results (Test signal measurement)

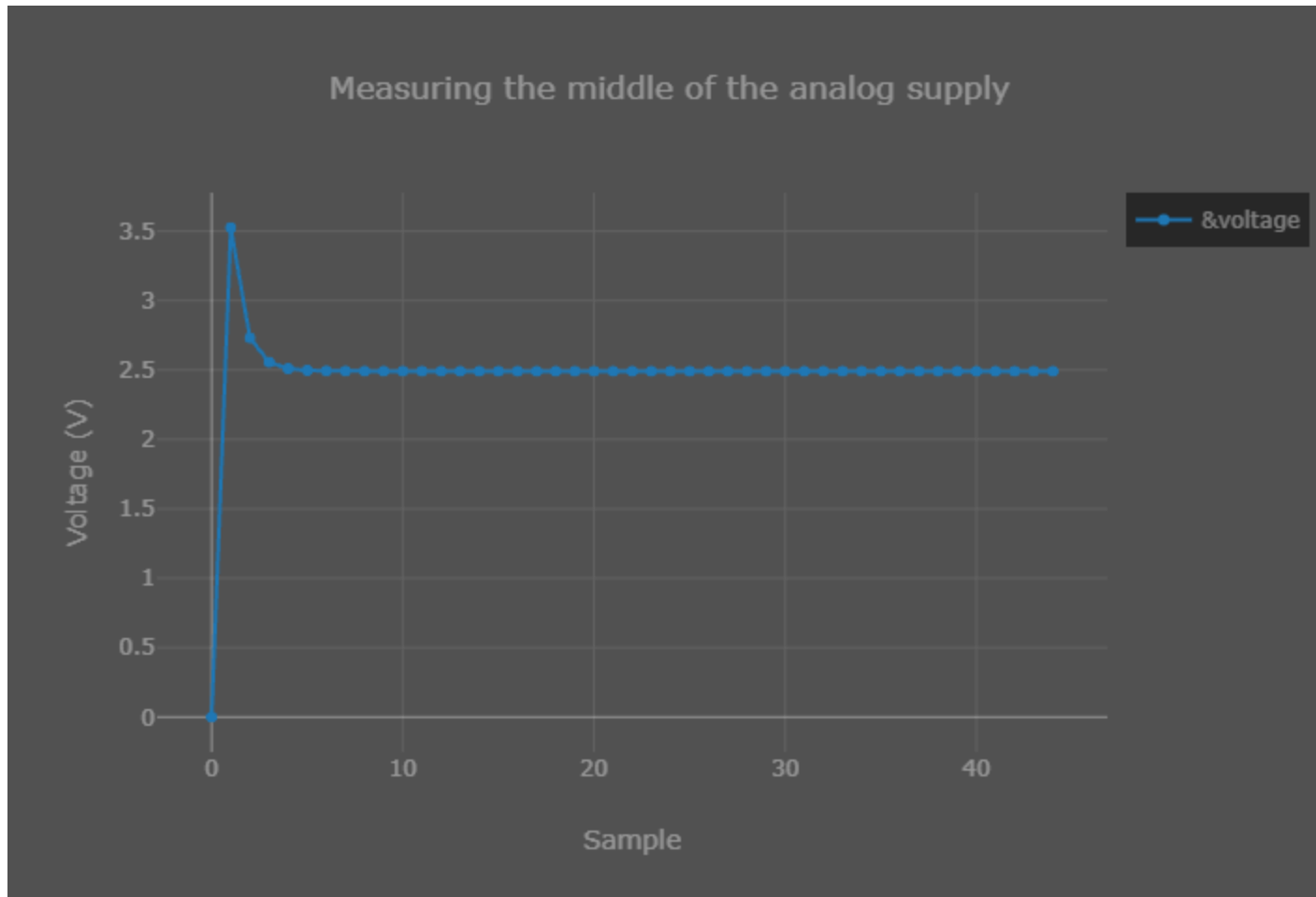


Top: 1.8487 mV

Bottom: -1.897537 mV

Measured Amplitude:
1.873 mV

Results (Supply measurement)



The input (based on the datasheet) is

$$(AVDD + AVSS) / 2$$
$$= (5 \text{ V} + 0) / 2$$
$$= 2.5 \text{ V}$$

Results (Sampling rate)



Frequency of the data ready signal (986.45 Hz) which is equivalent to the sampling rate (1 kHz)

Observation / Conclusion

- When using the normal input to the channel, the measurements are still essentially noise (even when I did a 1 Hz square wave)
- Since the test signal measurements are fine, the problem is probably to the left of the MUX, so the input terminals and/or function generator and/or grounding

Observation / Conclusion

9.3.1.2 Analog Input

The analog inputs to the device connect directly to an integrated low-noise, low-drift, high input impedance, programmable gain amplifier. The amplifier is located following the individual channel multiplexer.

The ADS1299-x analog inputs are fully differential. The differential input voltage ($V_{INXP} - V_{INXN}$) can span from $-V_{REF} / \text{gain}$ to V_{REF} / gain . See the [Data Format](#) section for an explanation of the correlation between the analog input and digital codes. There are two general methods of driving the ADS1299-x analog inputs: pseudo-differential or fully-differential, as shown in [Figure 20](#), [Figure 21](#), and [Figure 22](#).

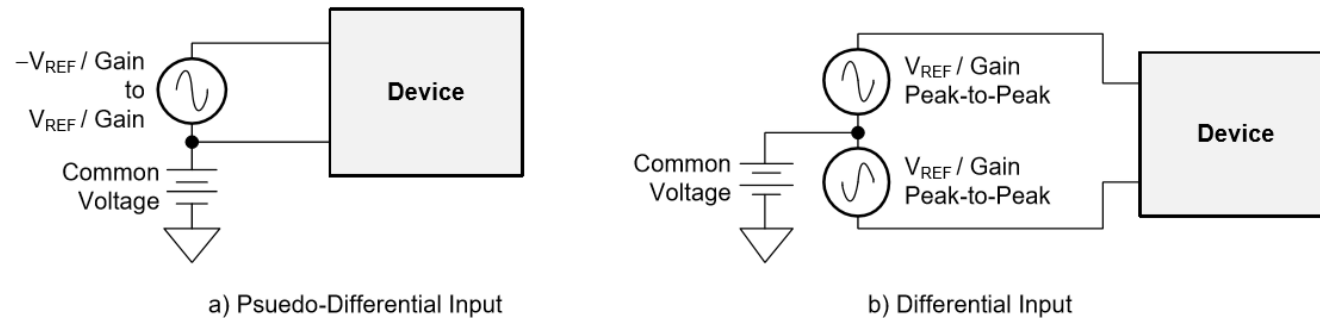


Figure 20. Methods of Driving the ADS1299-x: Pseudo-Differential or Fully Differential

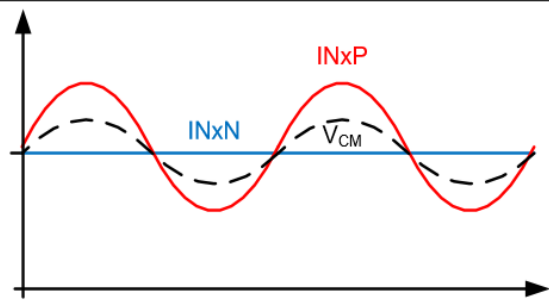


Figure 21. Pseudo-Differential Input Mode

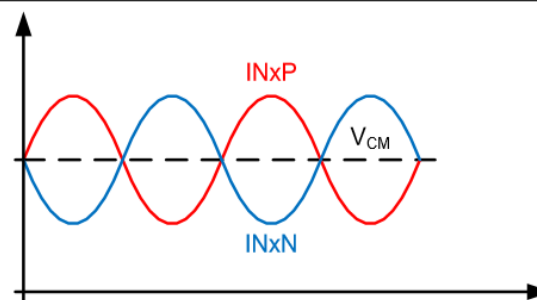


Figure 22. Fully-Differential Input Mode

The datasheet recommends the common voltage to be mid-supply (2.5 V in our case)

$$\begin{aligned} V_{\text{ref}} / \text{gain} &= 4.5 \\ / 24 &= 0.1875 \text{ V} \\ &= 187.5 \text{ mV} \end{aligned}$$

Next steps

- Research the bias amplifier
- Test with a different channel

02/11/2026

Dmytro Stavskyi and Luis Wong

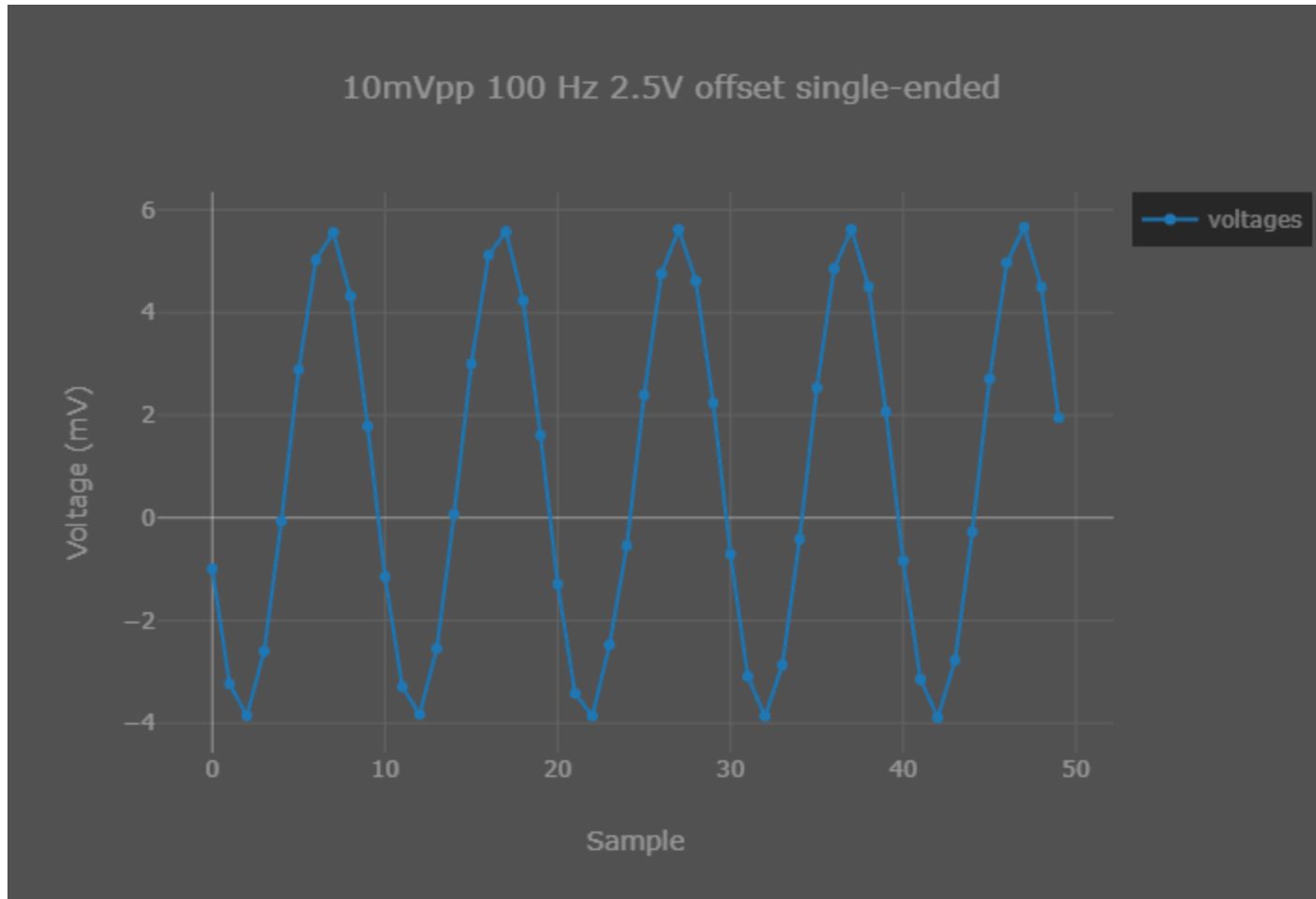
Objective

- Capture normal input data provided by the function generator

Methods

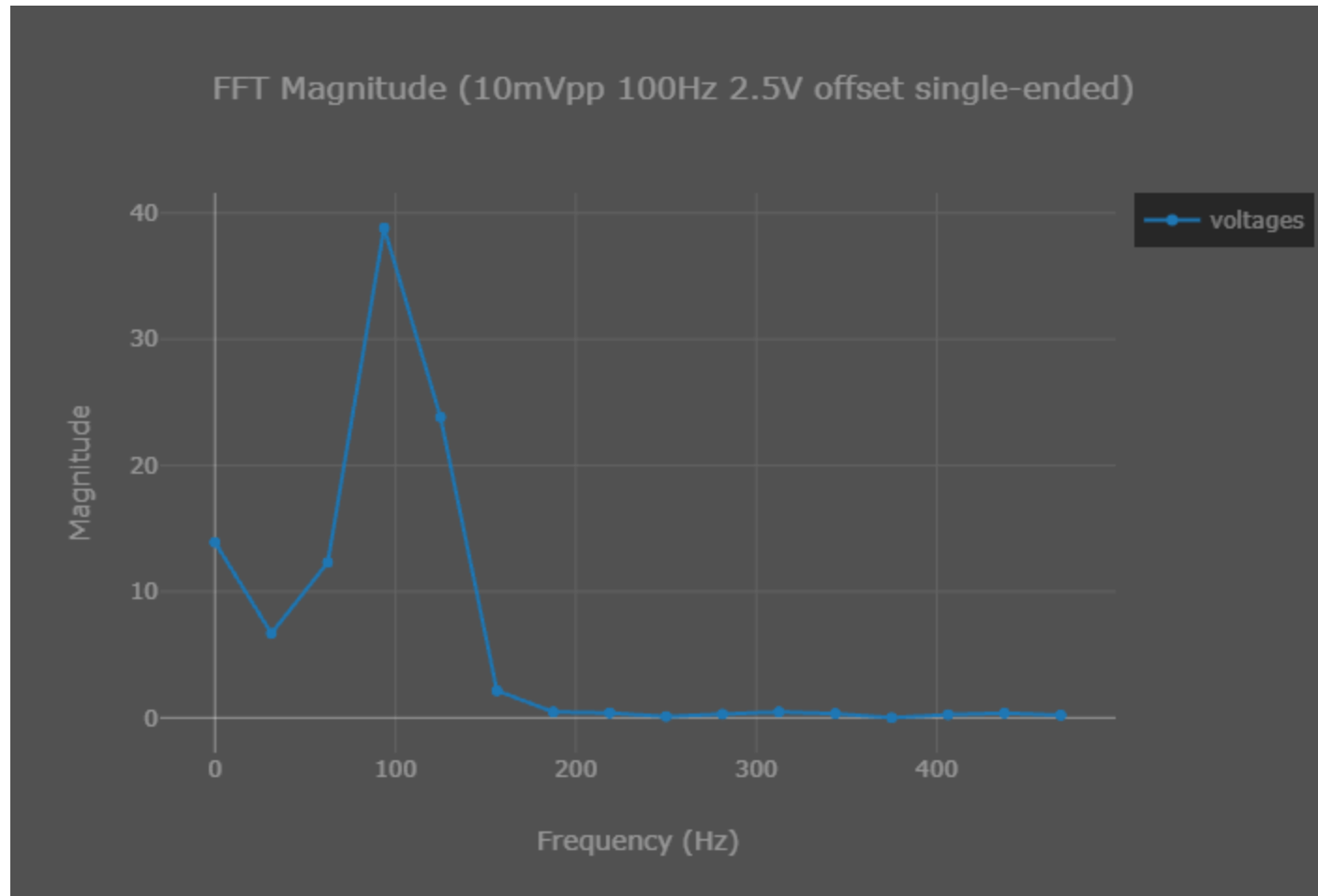
- Instead of continuously updating and plotting a variable, we save a number of points (e.g., 50) in an array and then plot those points
- Use FFT to calculate the frequencies of the captured signal
- For single-ended measurements, we applied the sine wave signal to the positive terminal (offset provided by the function generator) and either analog ground or some DC offset to the negative terminal

Results (5mV 100Hz 2.5 V offset single-ended)



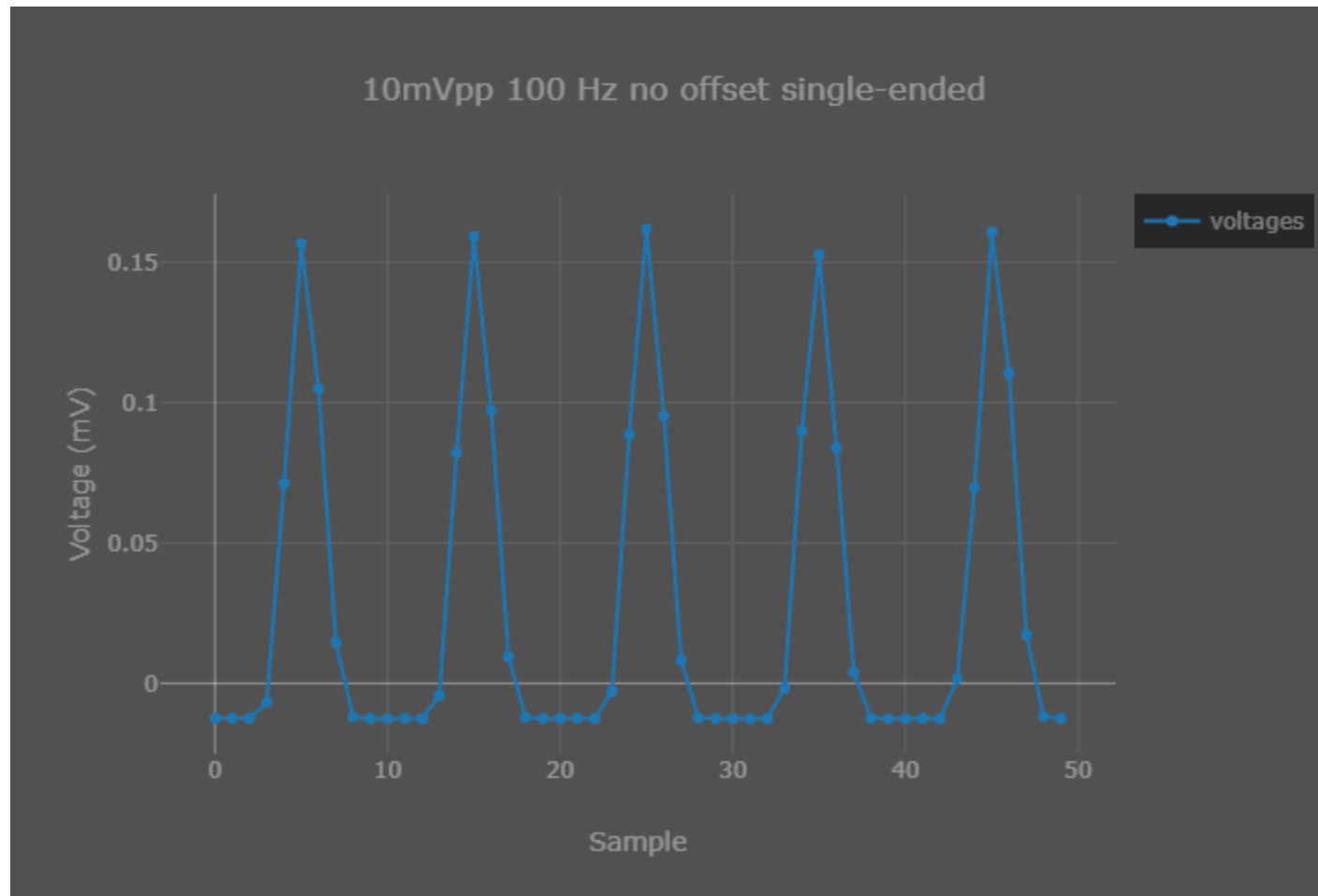
- Result was achieved by storing the data points in an array rather than a single variable.
- The graph is generated every time a breakpoint is reached, by default this is located at the location of the variable we are watching.
- According to the CCS documentation the graph runs through the debug probe and can only update every 0.1s under extremely ideal conditions.
- The [Sampling Rate] setting in CCS is arbitrary, and the true sampling rate is determined by the peripheral.

Results (5mV 100Hz 2.5 V offset single-ended FFT)



- Used an FFT to confirm that the frequency of the signal captured is 100Hz just like we generated.

Results (5mV 100 Hz no offset single-ended)



- No offset causes the device to essentially only capture noise.

The usable input common-mode range of the front-end depends on various parameters, including the maximum differential input signal, supply voltage, PGA gain, and the 200 mV for the amplifier headroom. This range is described in [Equation 4](#):

$$AVDD - 0.2 \text{ V} - \left(\frac{\text{Gain} \times V_{\text{MAX_DIFF}}}{2} \right) > CM > AVSS + 0.2 \text{ V} + \left(\frac{\text{Gain} \times V_{\text{MAX_DIFF}}}{2} \right)$$

where:

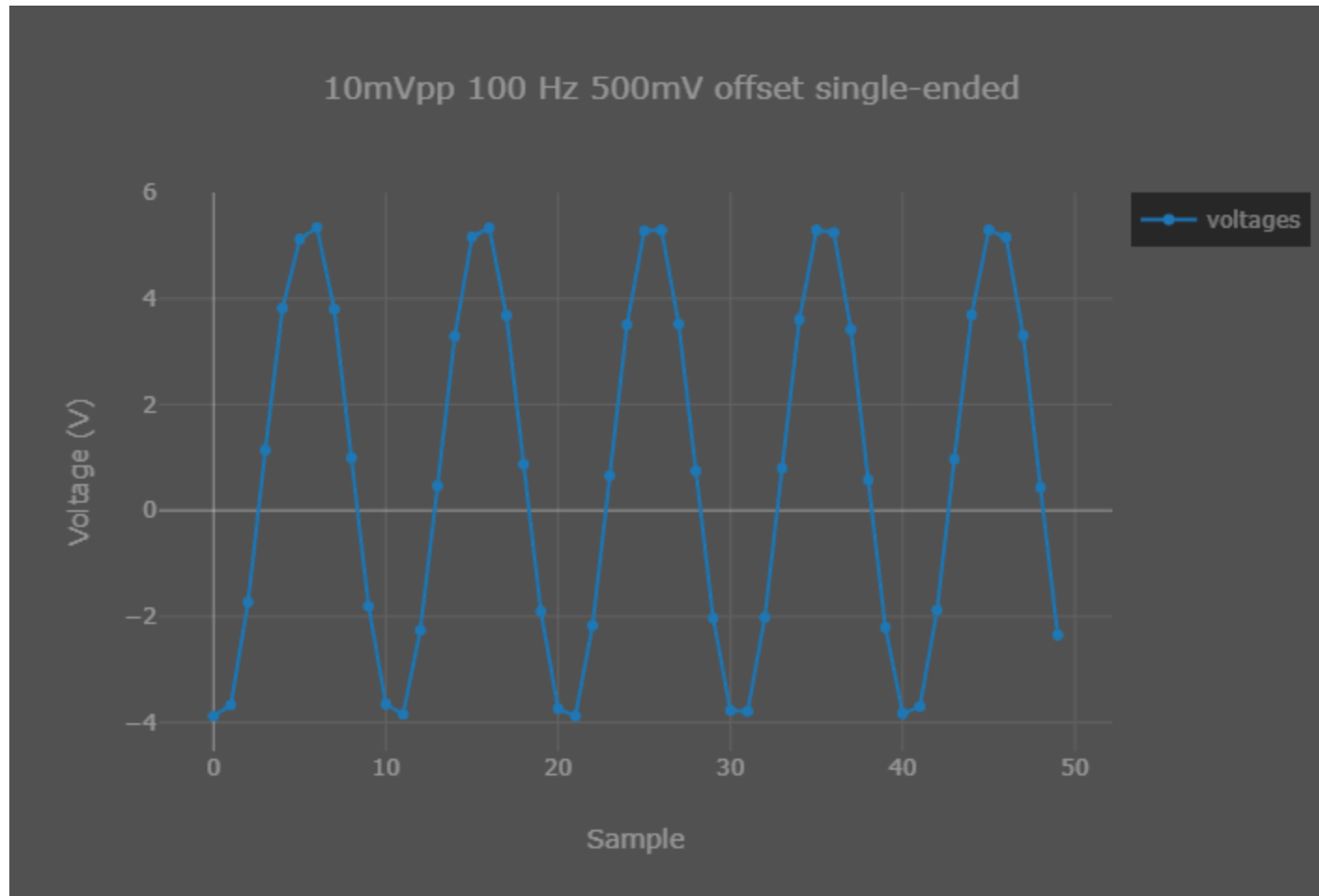
$V_{\text{MAX_DIFF}}$ = maximum differential signal at the PGA input

CM = common-mode range

(4)

- $AVDD = 5\text{V}$
- $\text{Gain} = 24$
- $AVSS = 0\text{V}$
- $V_{\text{max_diff}}$ = maximum differential signal at the PGA input = 10mV
- $0.32\text{V} < CM < 4.68\text{V}$

Results (5mV 100 Hz 500mV offset single-ended)



Observation / Conclusion

- We need to offset the signal internally which can be achieved through the bias circuit.
- We did not wire the bias circuit when creating this PCB, so it may not work properly

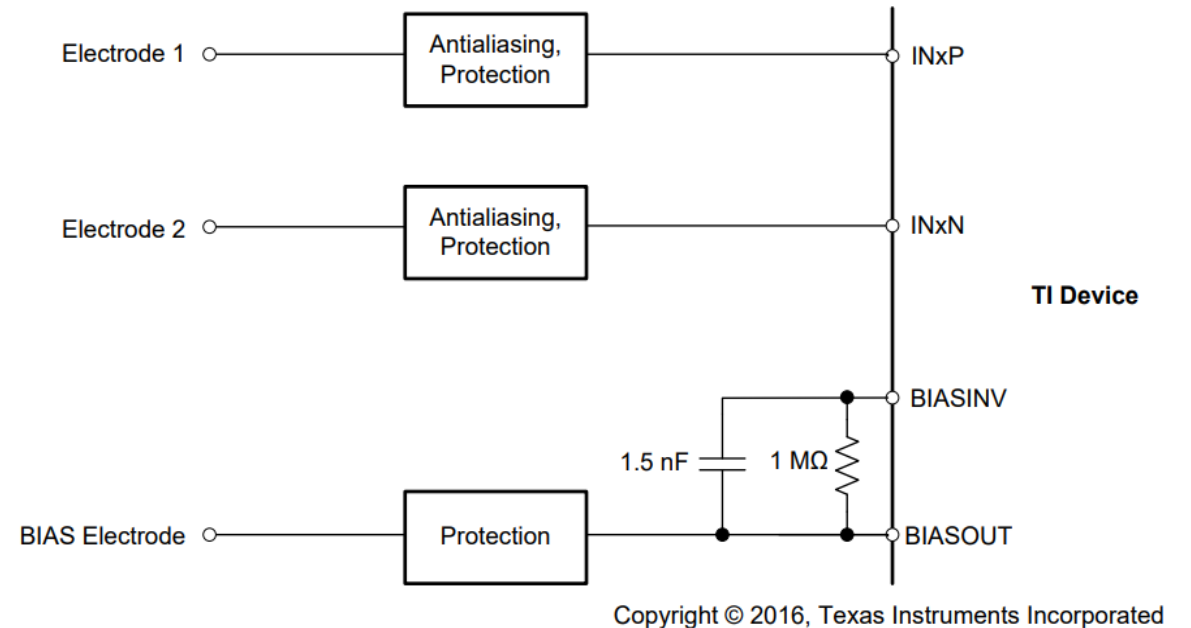
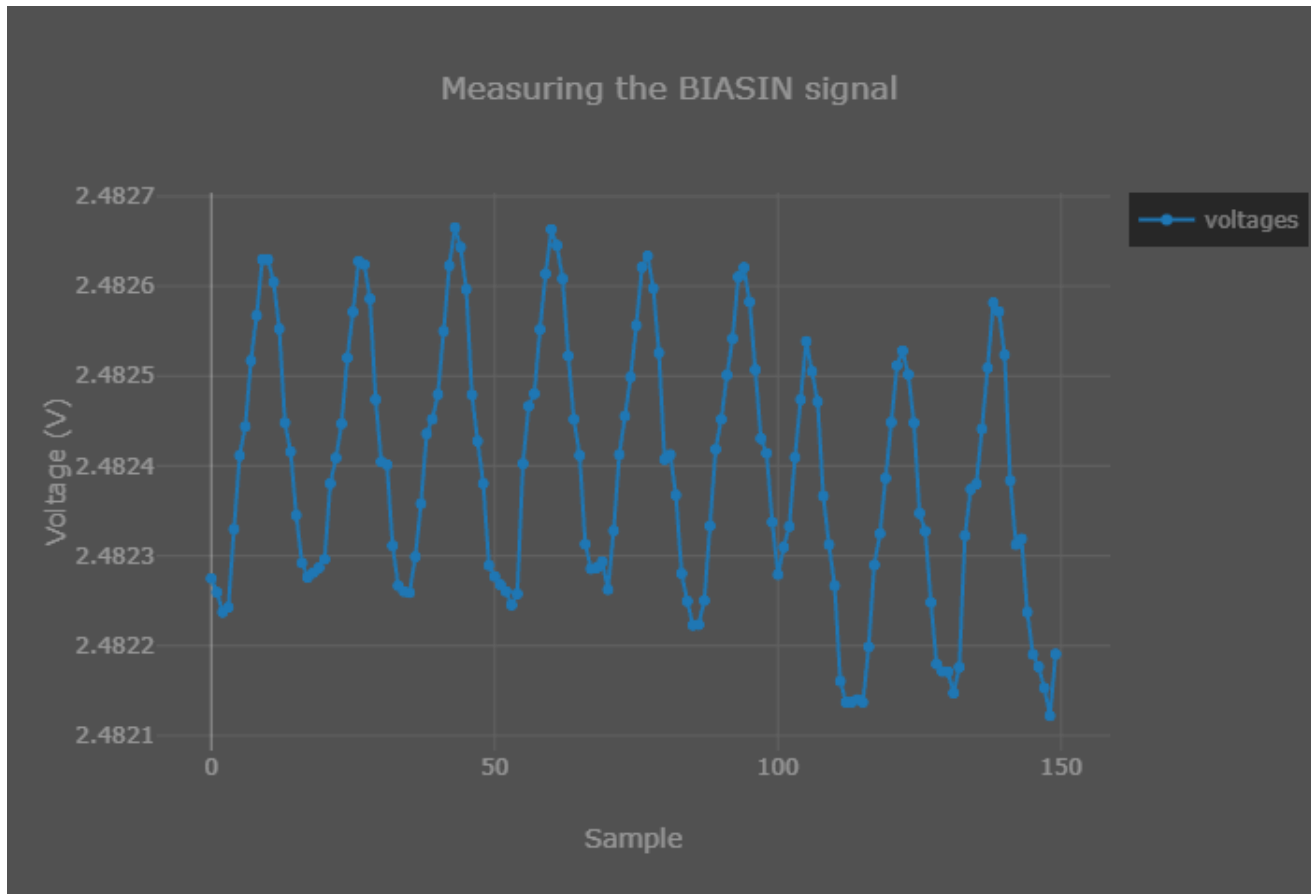


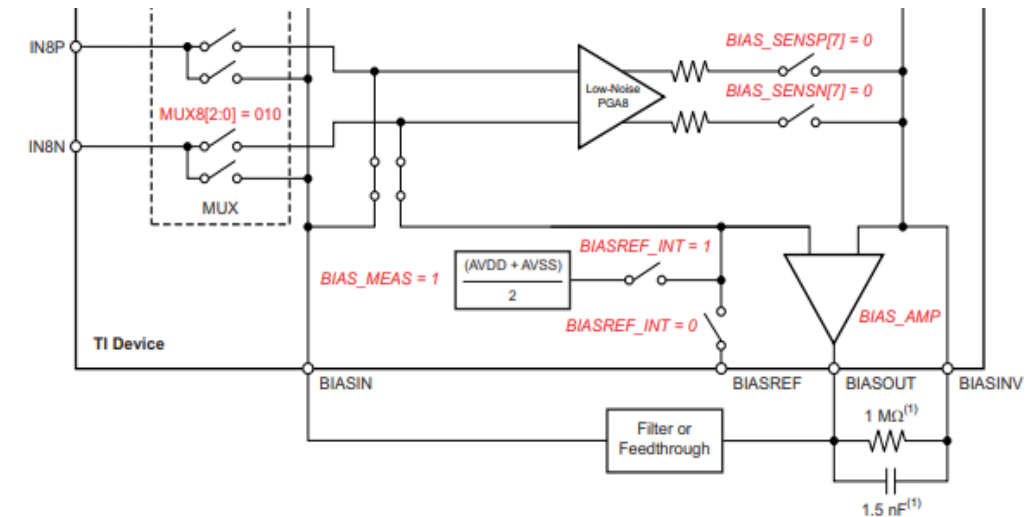
Figure 68. Setting Common-Mode Using BIAS Electrode

Observation / Conclusion



Right now, BIASIN and BIASOUT are shorted.

There is an option to route the BIASIN signal to a channel using MUX for measurement.



Copyright © 2016, Texas Instruments Incorporated

(1) Typical values for example only.

Figure 34. BIASOUT Signal Configured to be Read Back by Channel 8

Next steps

- Test the bias amplifier more
- Begin revising the schematic of the AFE circuit for the new design

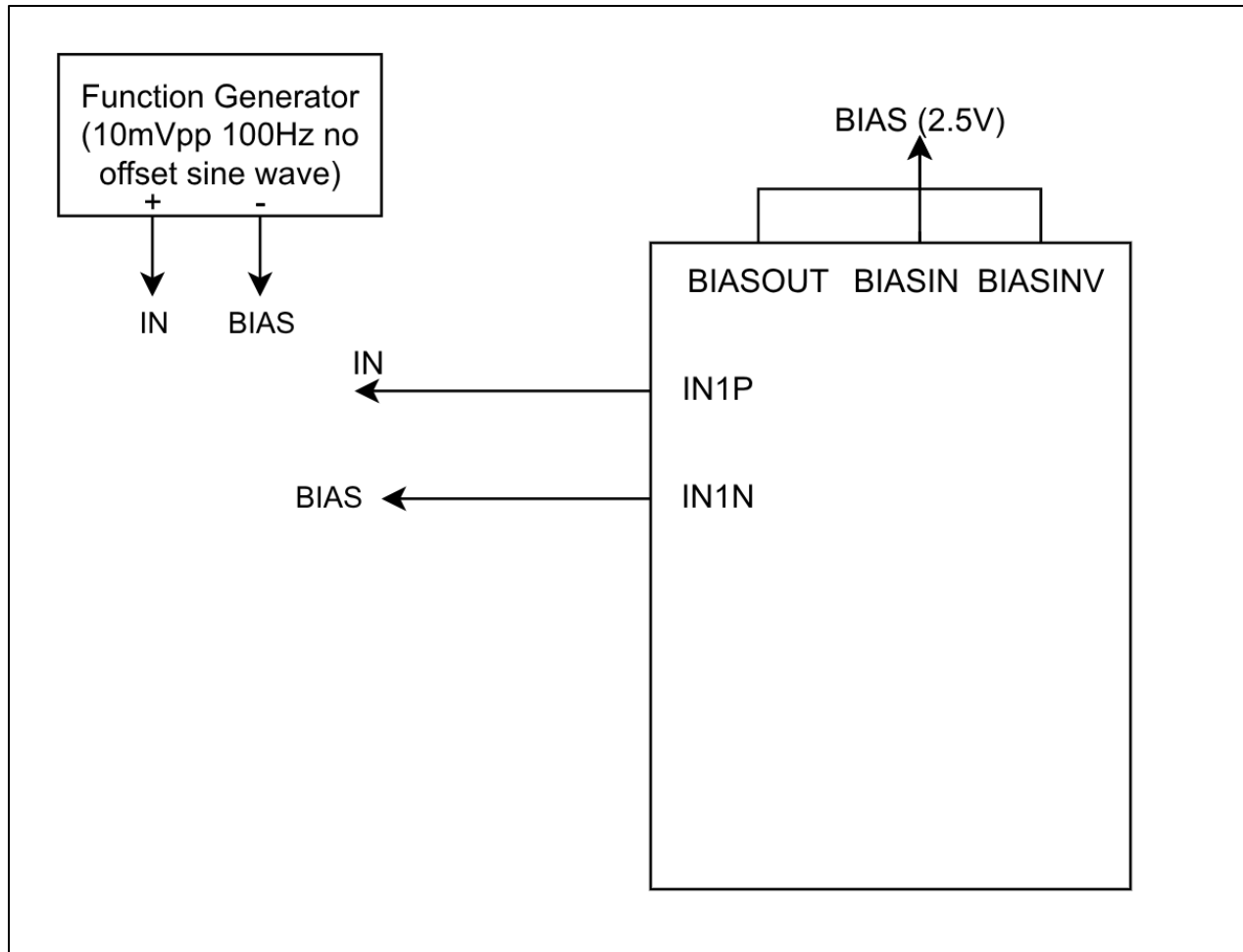
02/18/2026

Dmytro Stavskyi and Luis Wong

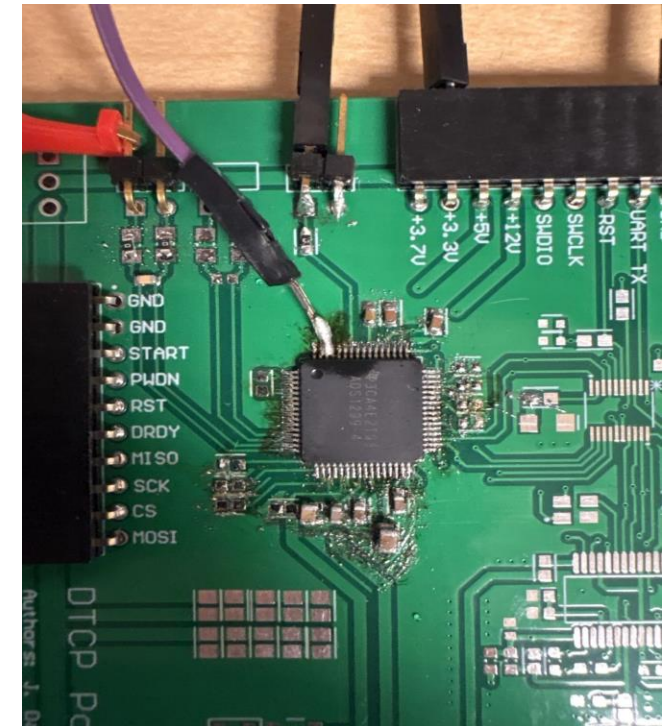
Objective

- Capture a normal input without providing offset in the function generator
 - Offset the external signal internally

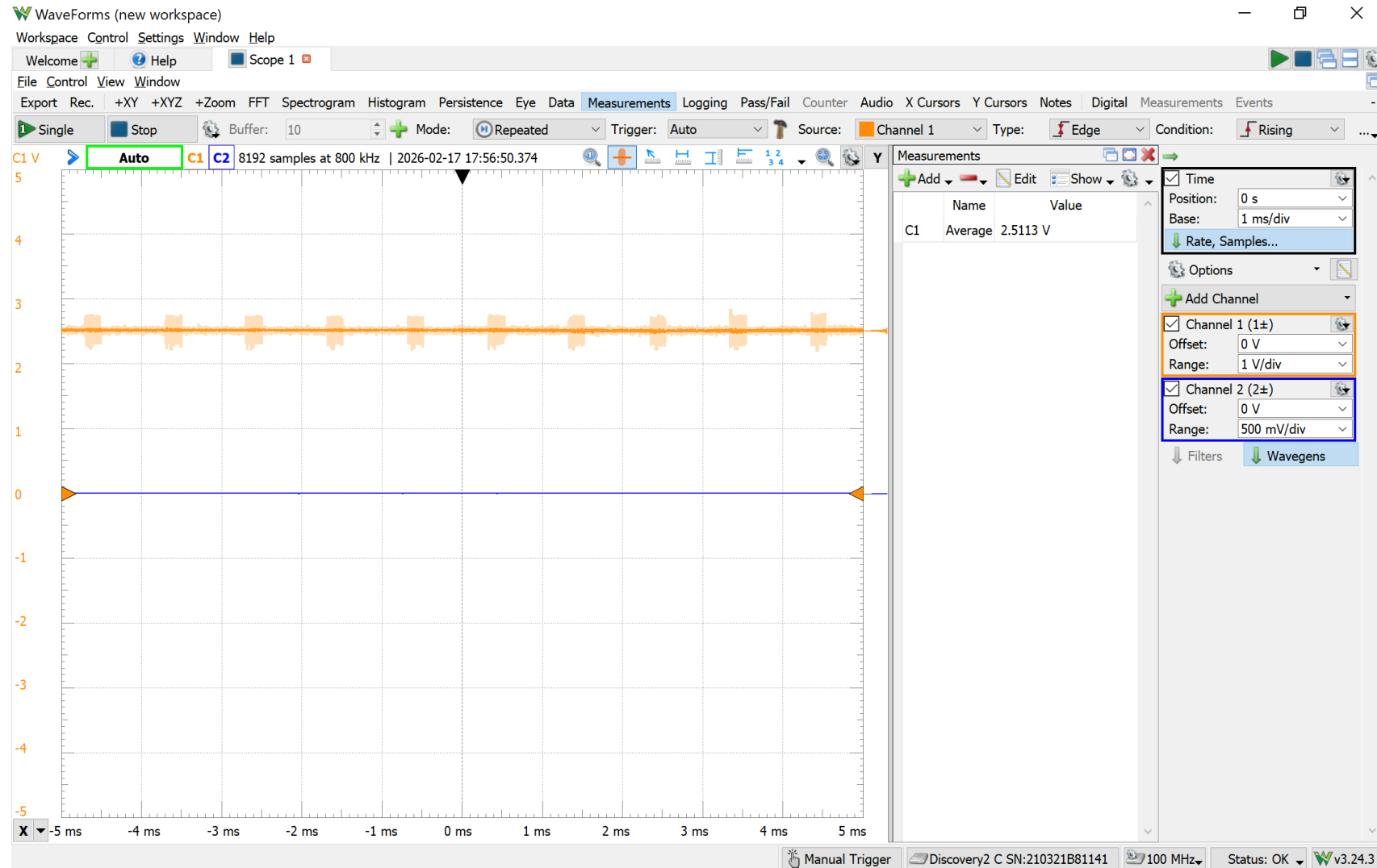
Methods



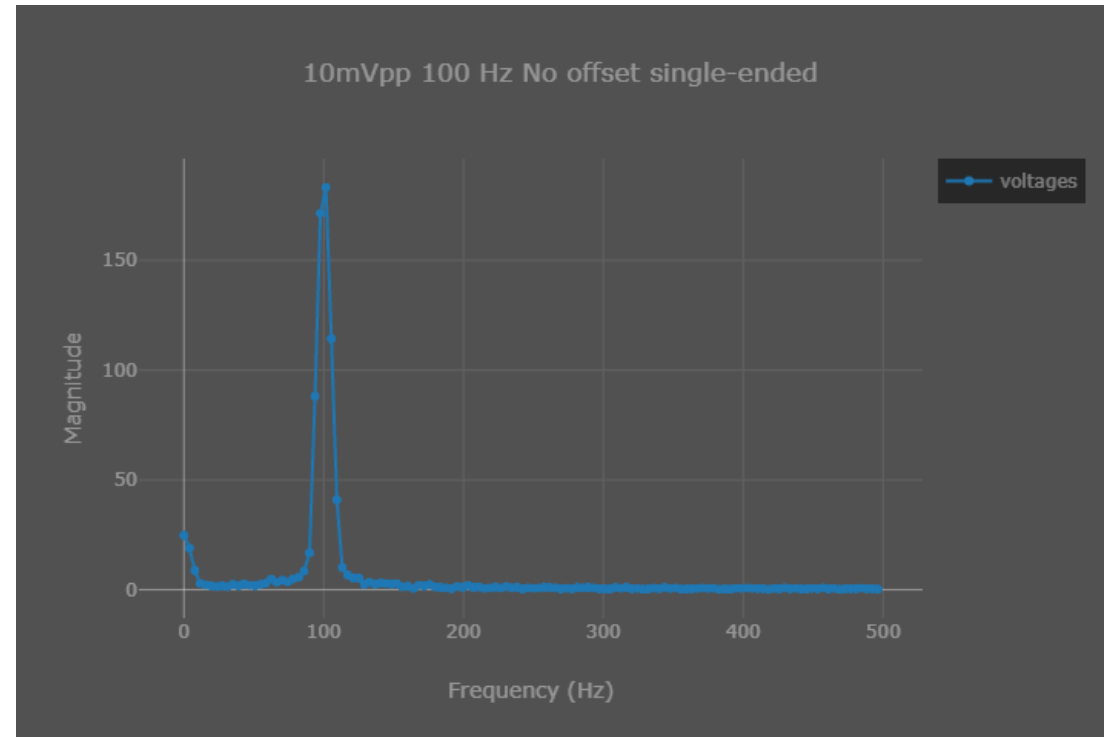
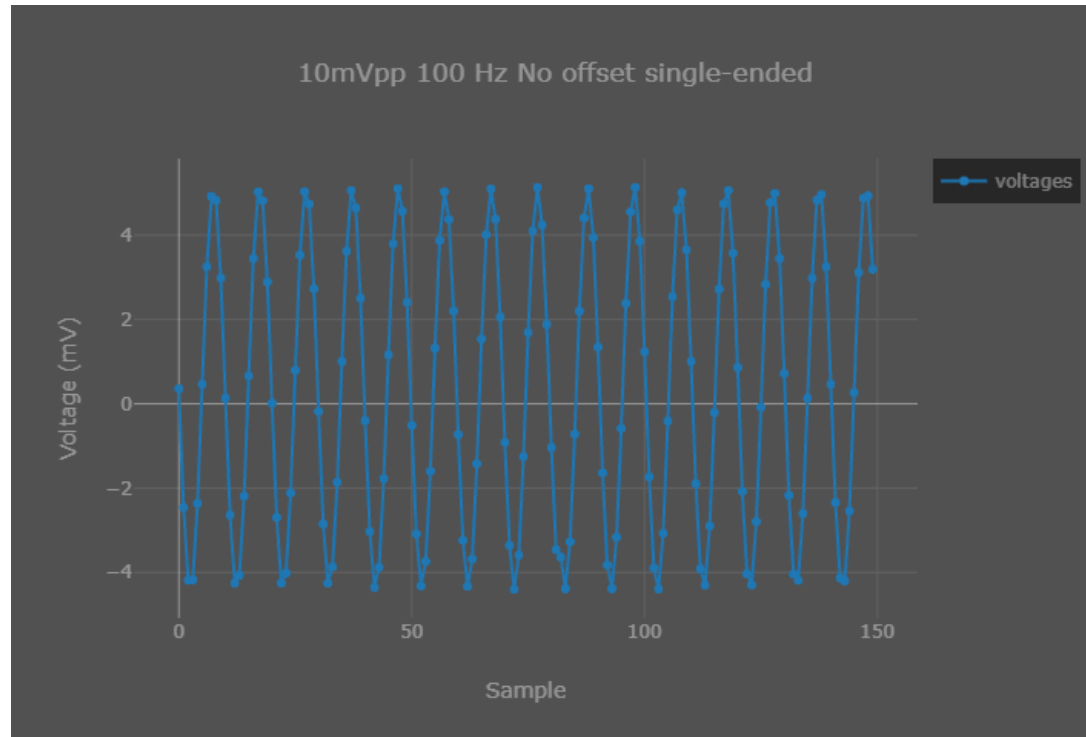
- BIASOUT, BIASIN, and BIASINV are shorted
- The bias is connected to the negative terminal of the channel and to the reference terminal of the input signal



Results (Measurement of the bias)



Results (no offset in the function generator)



Observation / Conclusion

- SRB1 pin connected to bias

9.6.1.15 MISC1: Miscellaneous 1 Register (address = 15h) (reset = 00h)

This register provides the control to route the SRB1 pin to all inverting inputs of the four, six, or eight channels (ADS1299-4, ADS1299-6, or ADS1299).

Figure 64. MISC1: Miscellaneous 1 Register

7	6	5	4	3	2	1	0
0	0	SRB1	0	0	0	0	0
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

Table 26. Miscellaneous 1 Register Field Descriptions

Bit	Field	Type	Reset	Description
7:6	Reserved	R/W	0h	Reserved Always write 0h
5	SRB1	R/W	0h	Stimulus, reference, and bias 1 This bit connects the SRB1 to all 4, 6, or 8 channels inverting inputs 0 : Switches open 1 : Switches closed
4:0	Reserved	R/W	0h	Reserved Always write 0h

Next steps

- Improve the code
- Research the biasing more
- Continue to revise the AFE schematic for the next design

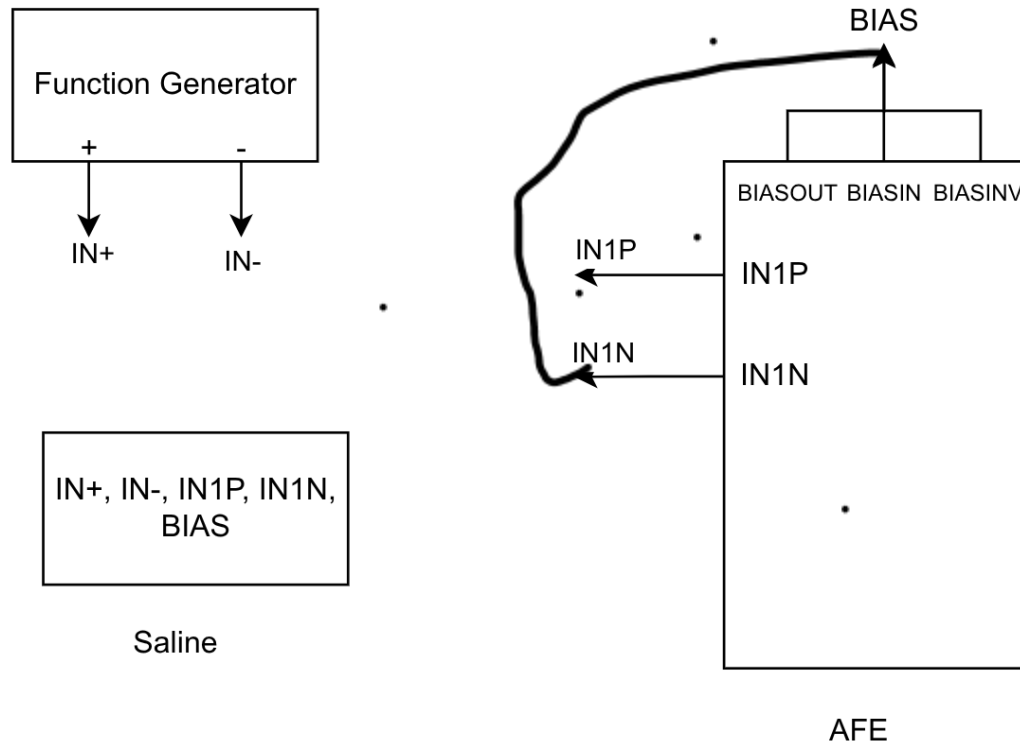
02/25/2026

Dmytro Stavskyi

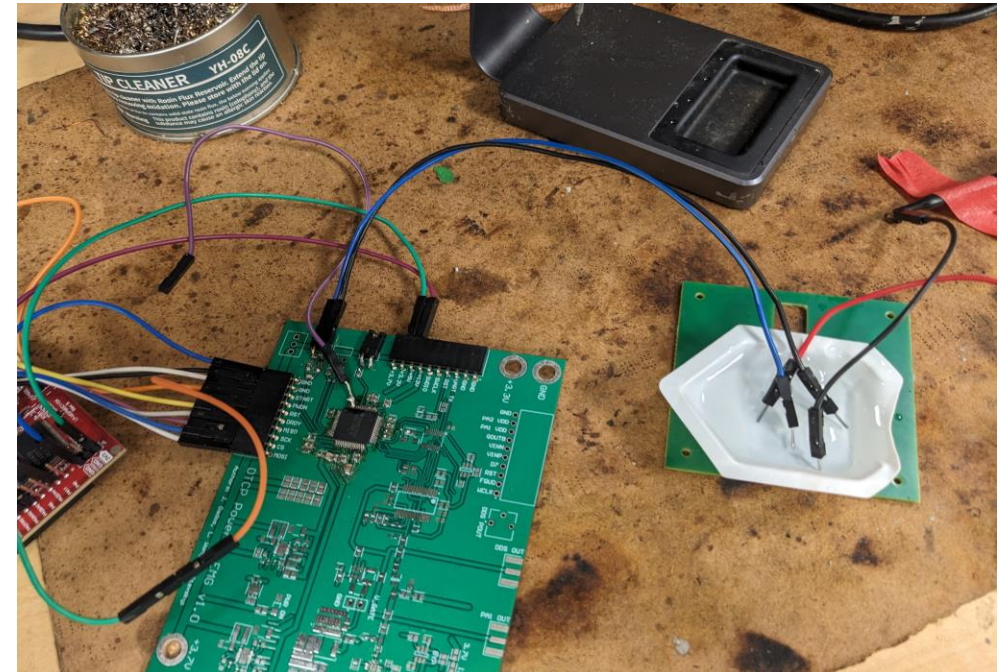
Objective

- Test the setup and the AFE measurements with saline

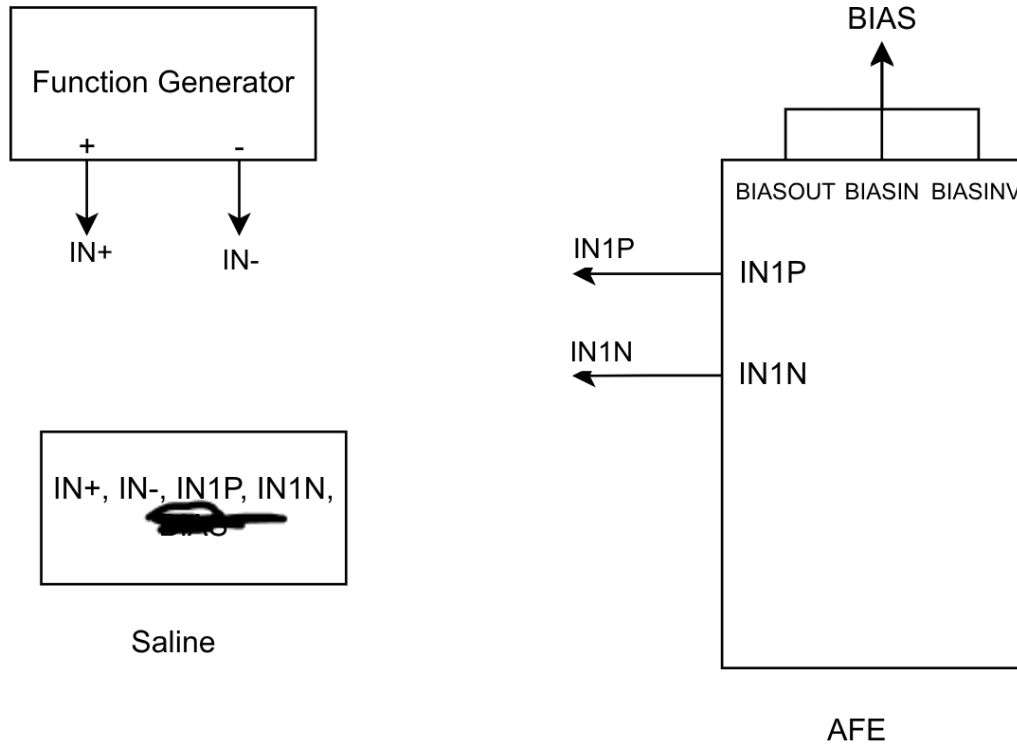
Methods



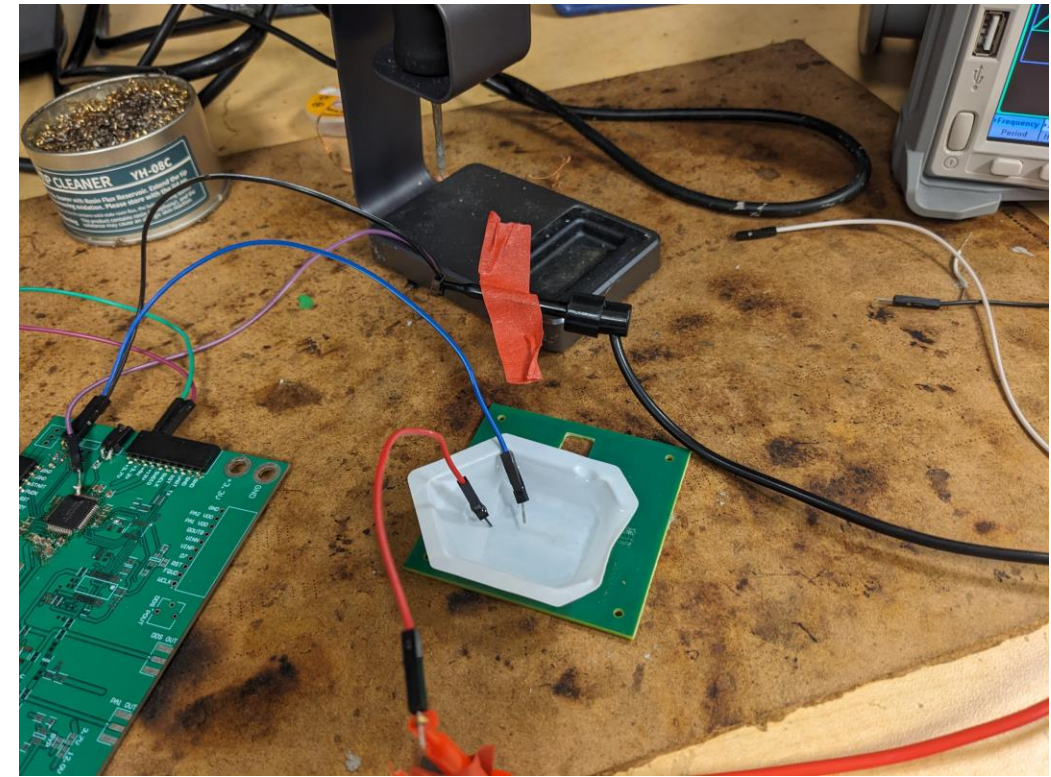
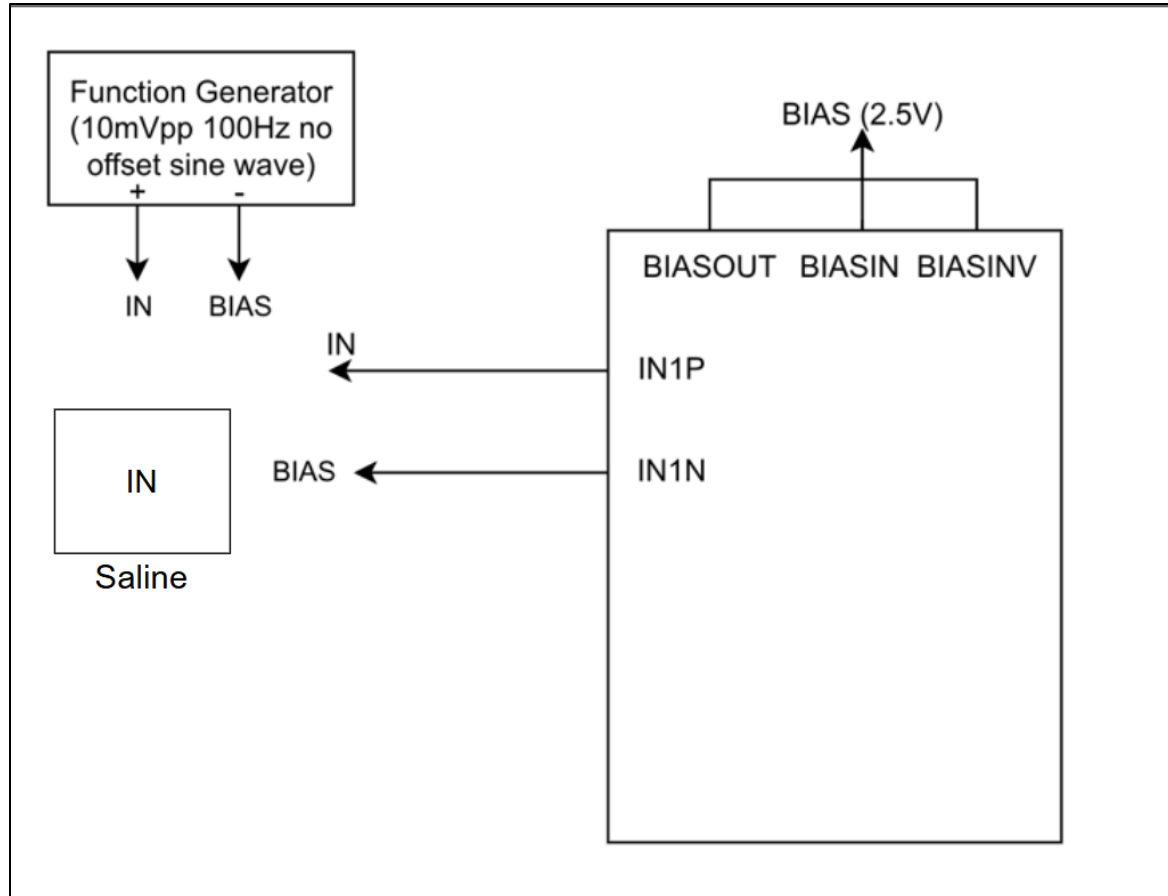
- Power is provided from the LaunchPad board, not the separate power supply



Methods (First Setup, No Bias)

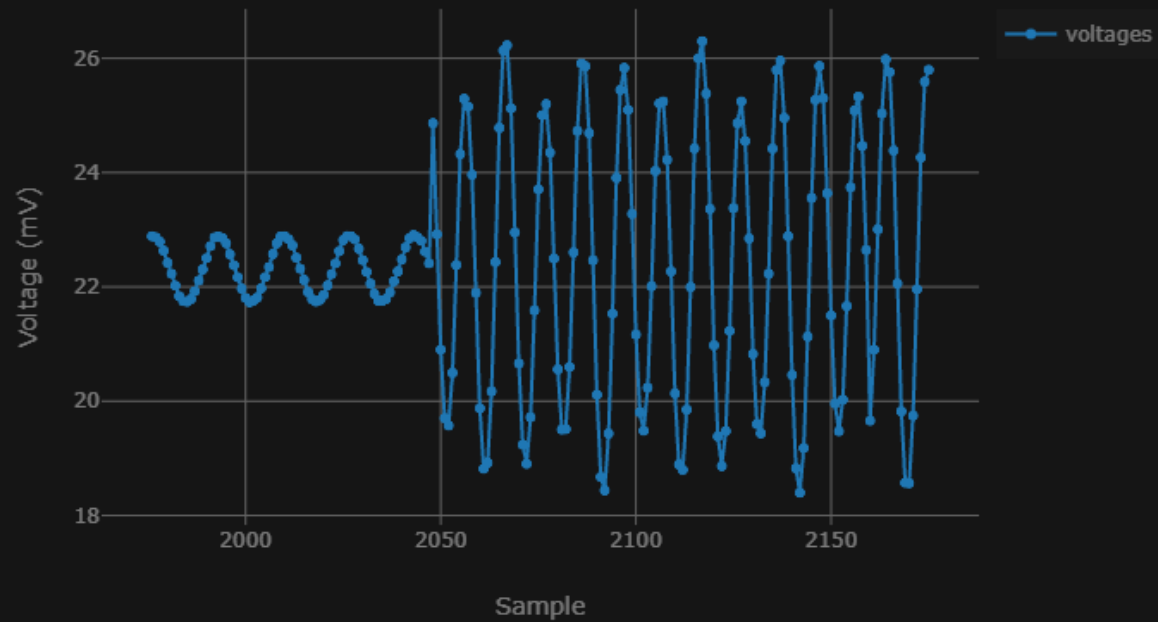


Methods

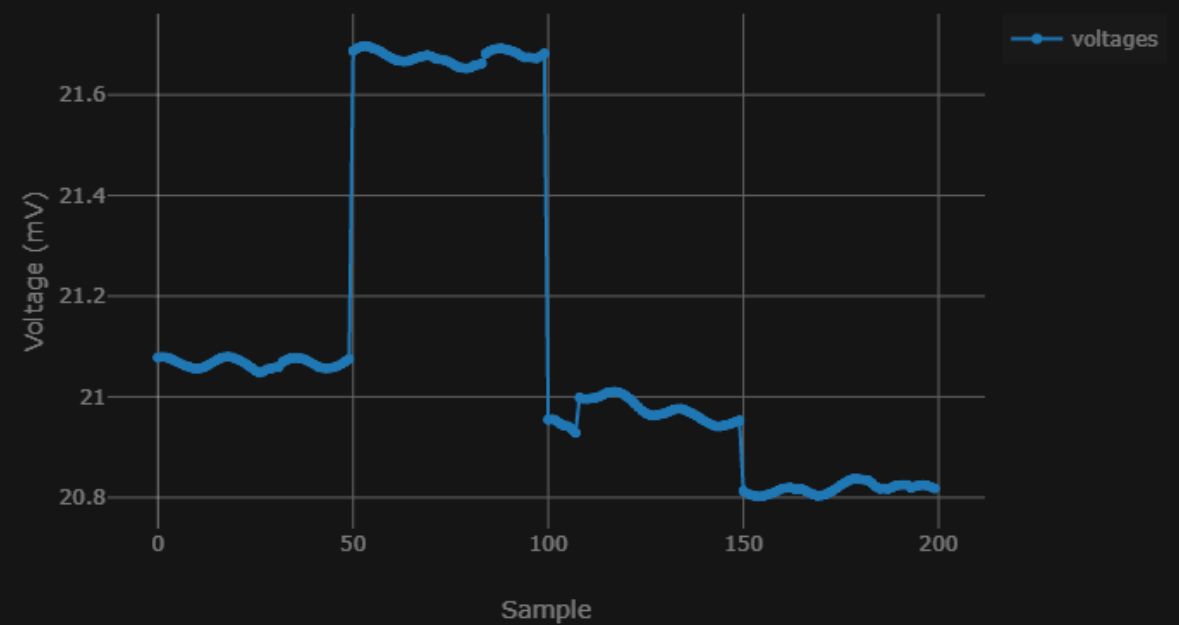


Results (First Setup)

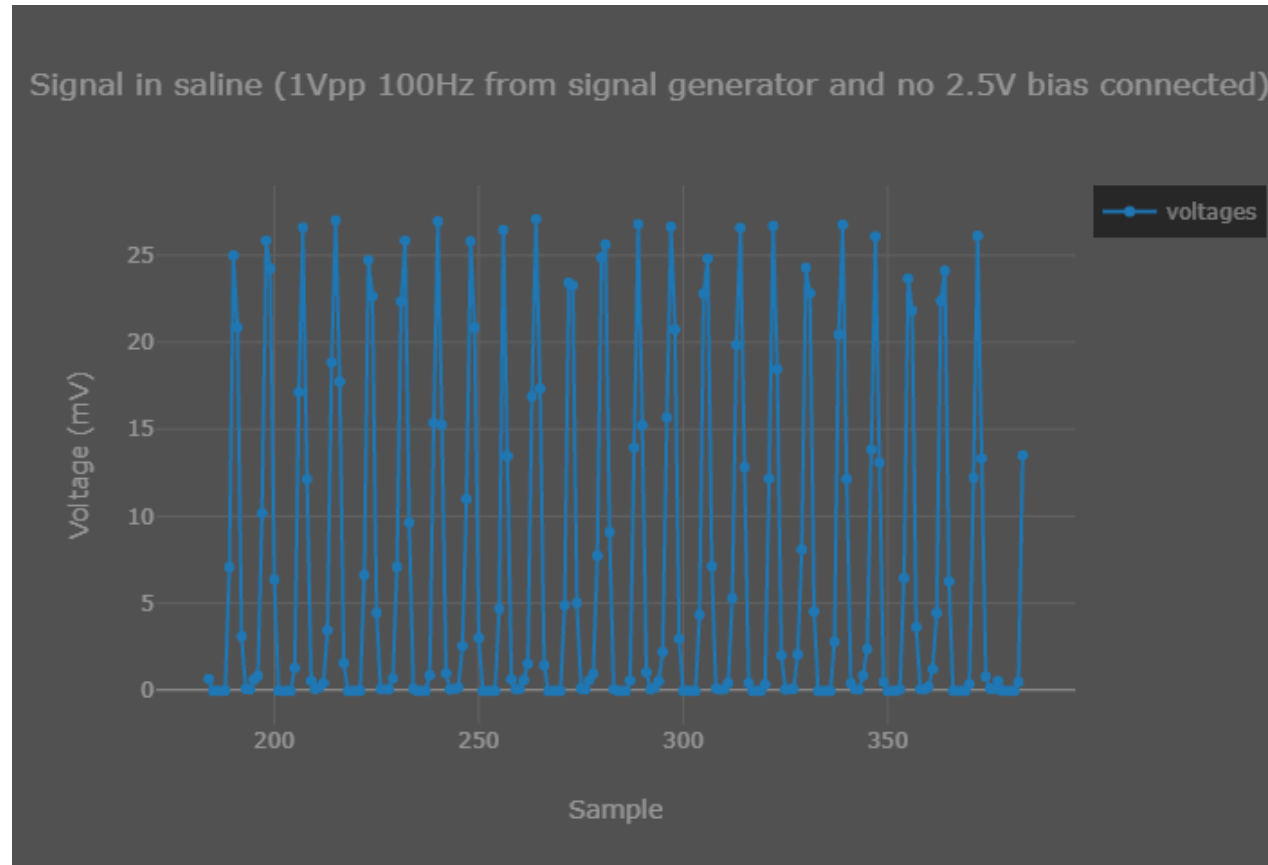
Signal in saline (1Vpp 100Hz from signal generator)



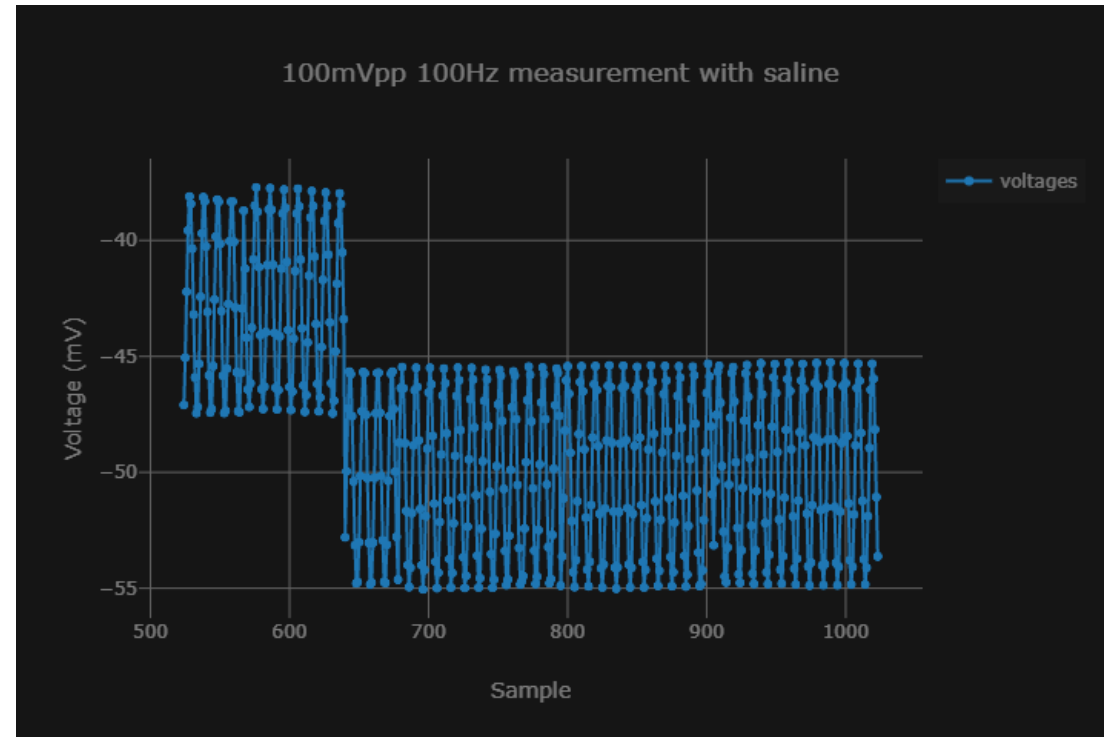
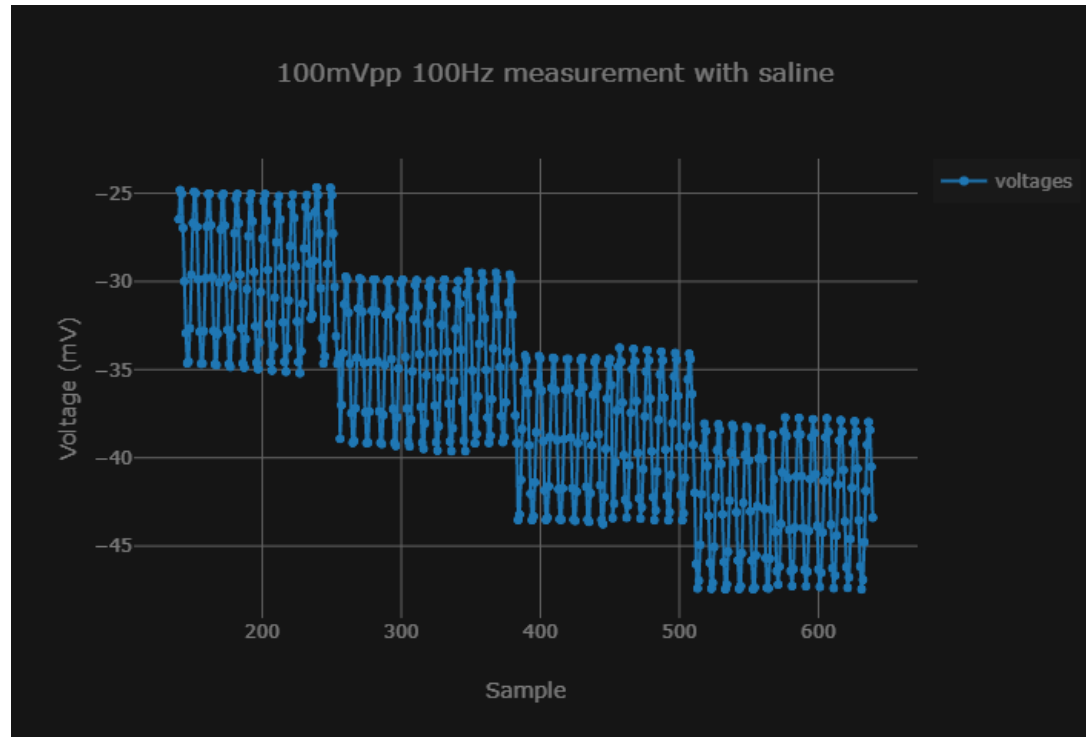
Noise floor in saline



Results (First Setup, No BIAS)

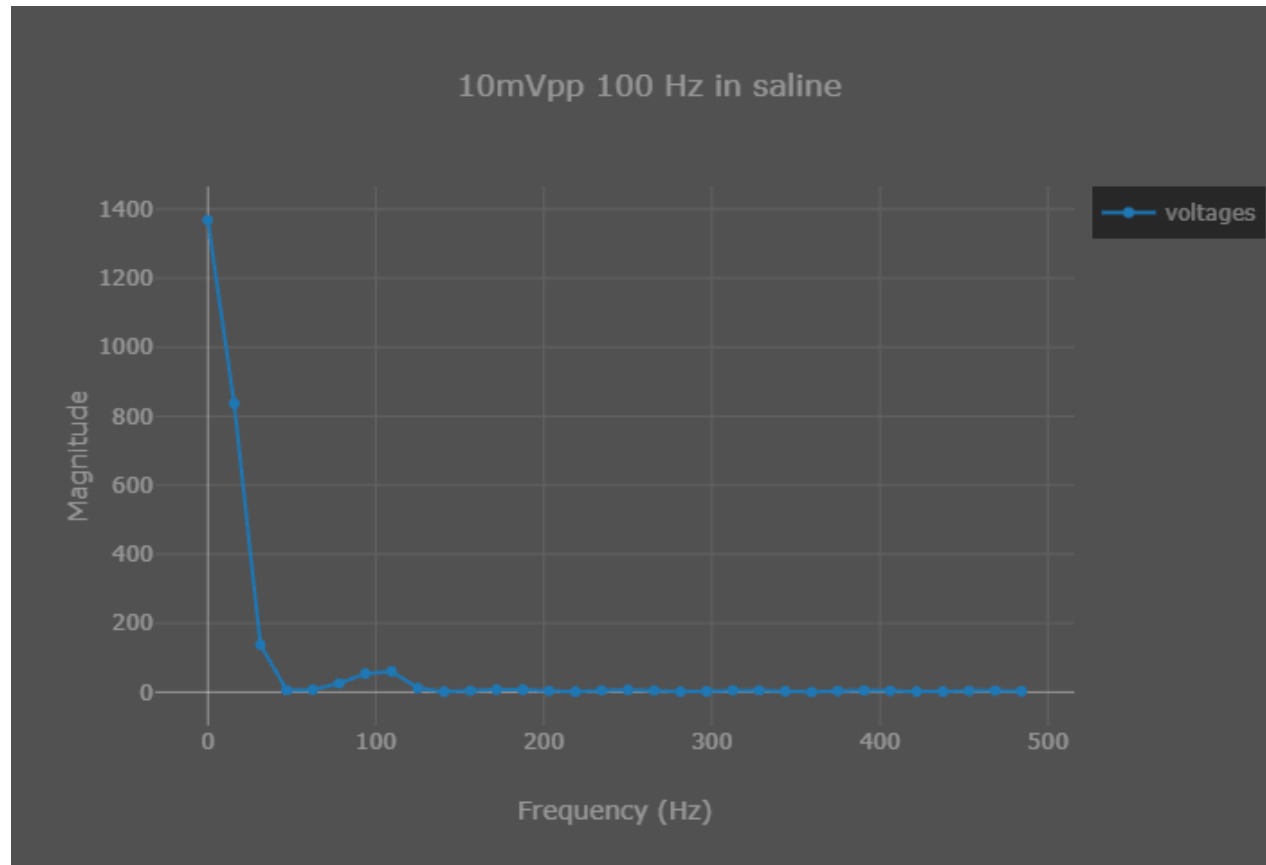


Results (Second Setup; 10mVpp 100Hz)

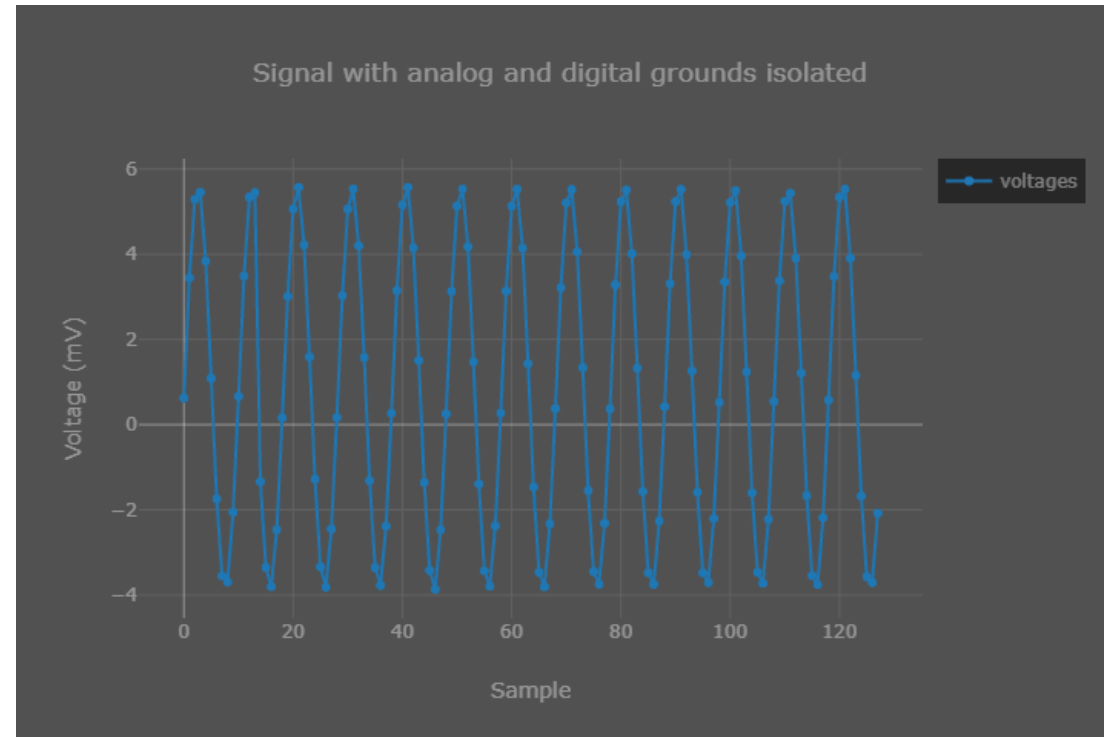
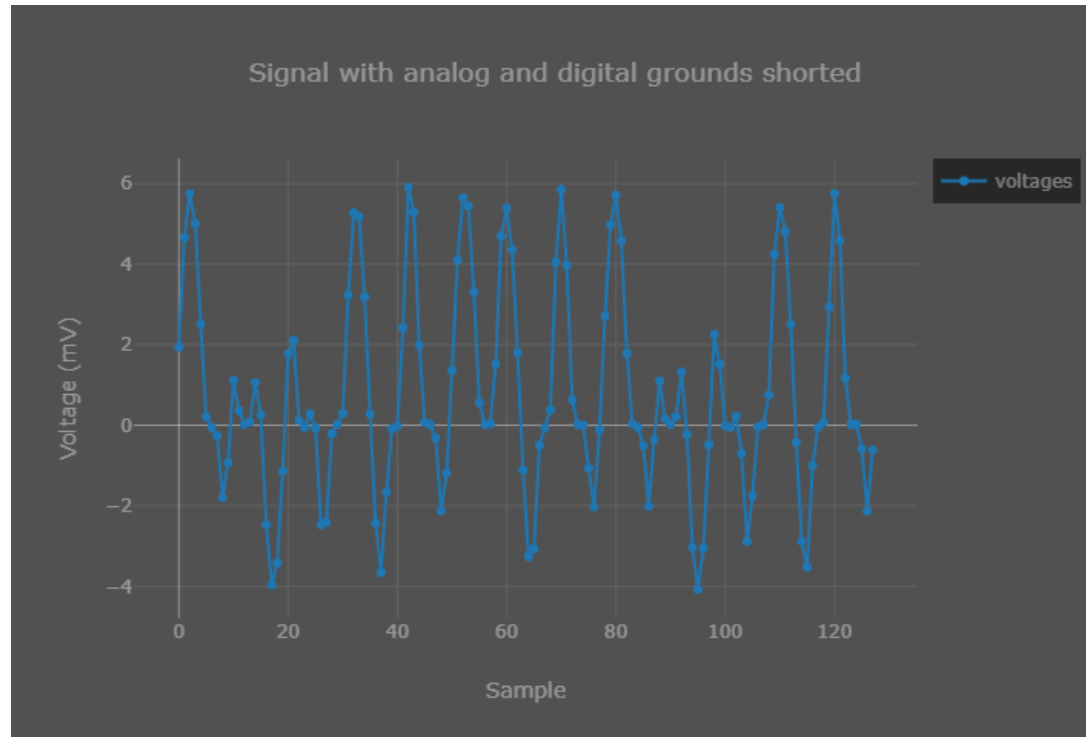


- Measured amplitude 4.794 mV

Observation / Conclusion (FFT; DC component)



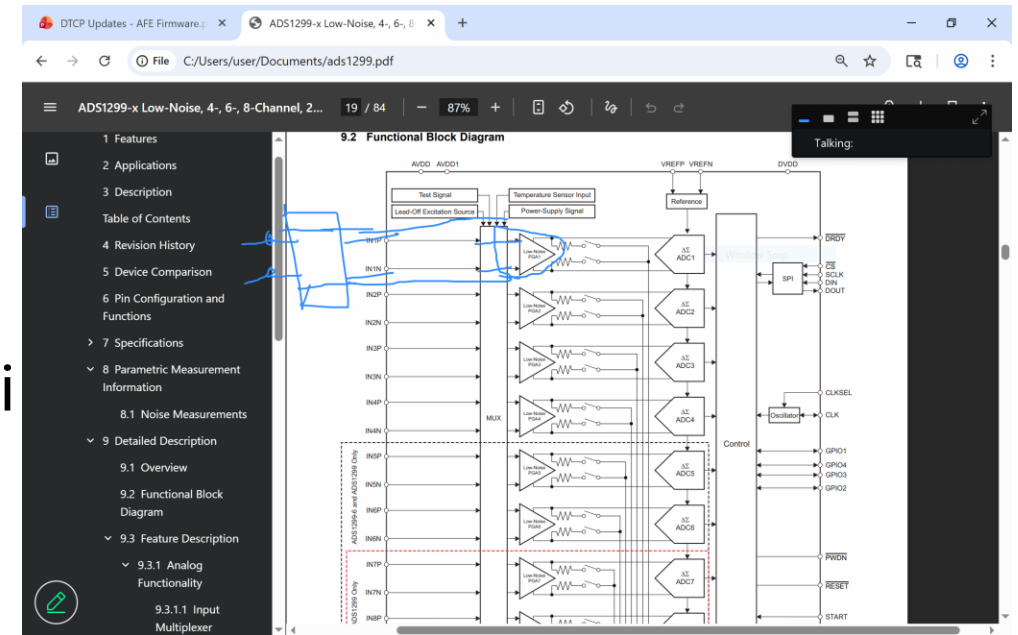
Observation / Conclusion (DGND and AGND)



Next steps

- Review the PCB schematics and note the input pin
- Improve the code
- Test the setup more
 - Repeat the tests (with detailed setups: drawings, saline connections, wire connections)
 - Internal power amplifier drawing
 - Filter for the input
 - System diagram
 - Four setups (methodology and results for each setup, and video): everything in saline, no bias in saline but shorted to the negative terminal, no bias, bias in saline but not shorted to the negative terminal

9.3.2.4.5 section of datasheet



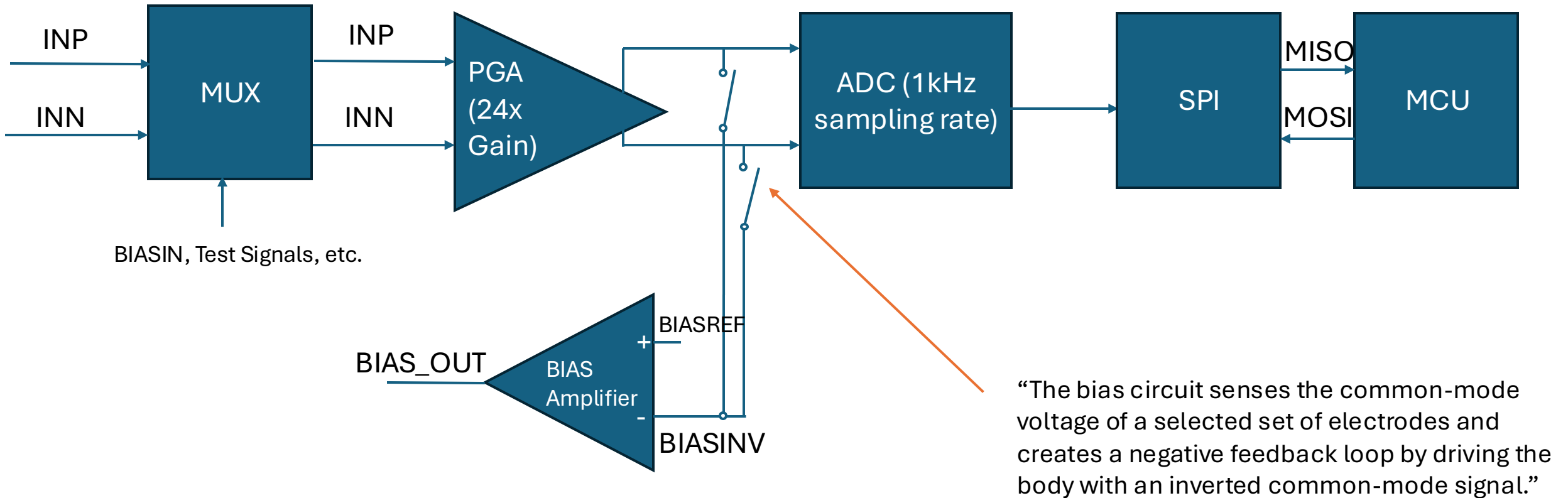
3/4/2026

Dmytro Stavskyi

Objective

- Test the setup and the AFE measurements with saline more

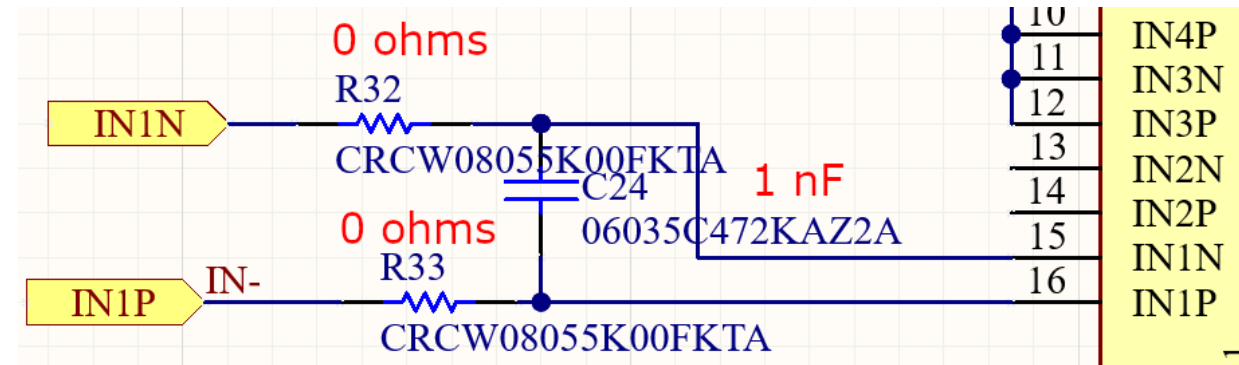
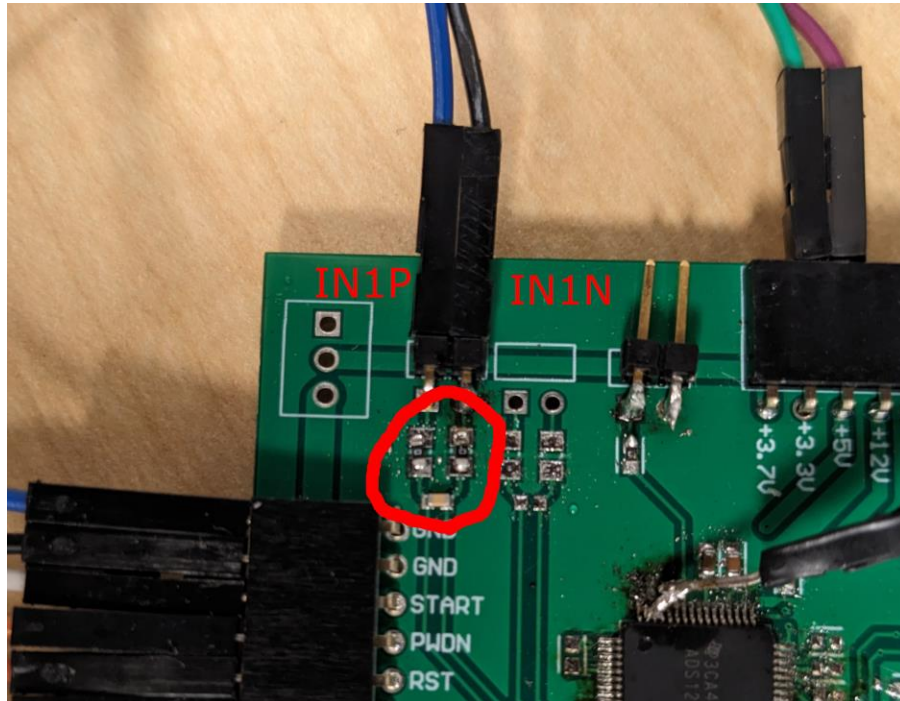
Methods (AFE System Diagram)



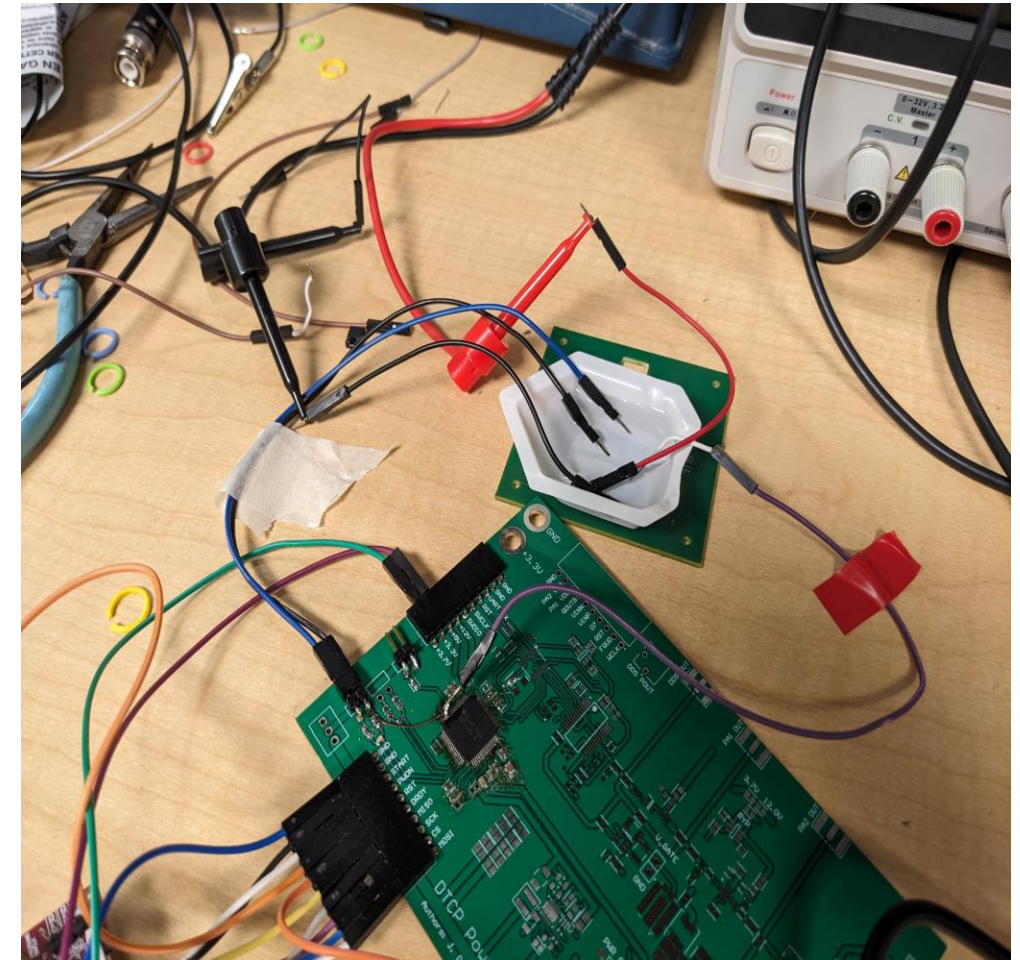
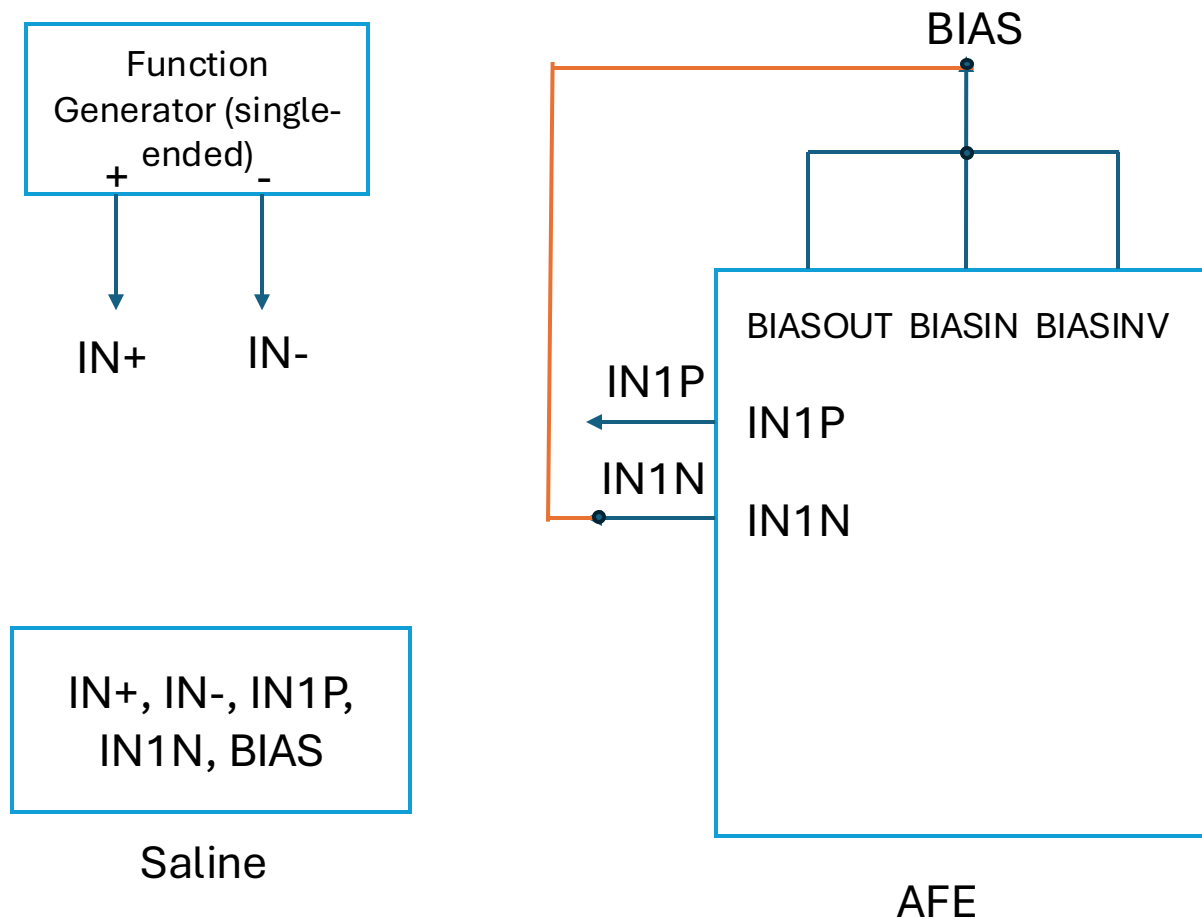
$$\text{BIASREF} = (\text{AVDD} + \text{AVSS}) / 2$$

If AVDD = 5V and AVSS = 0V, BIASREF = 2.5V

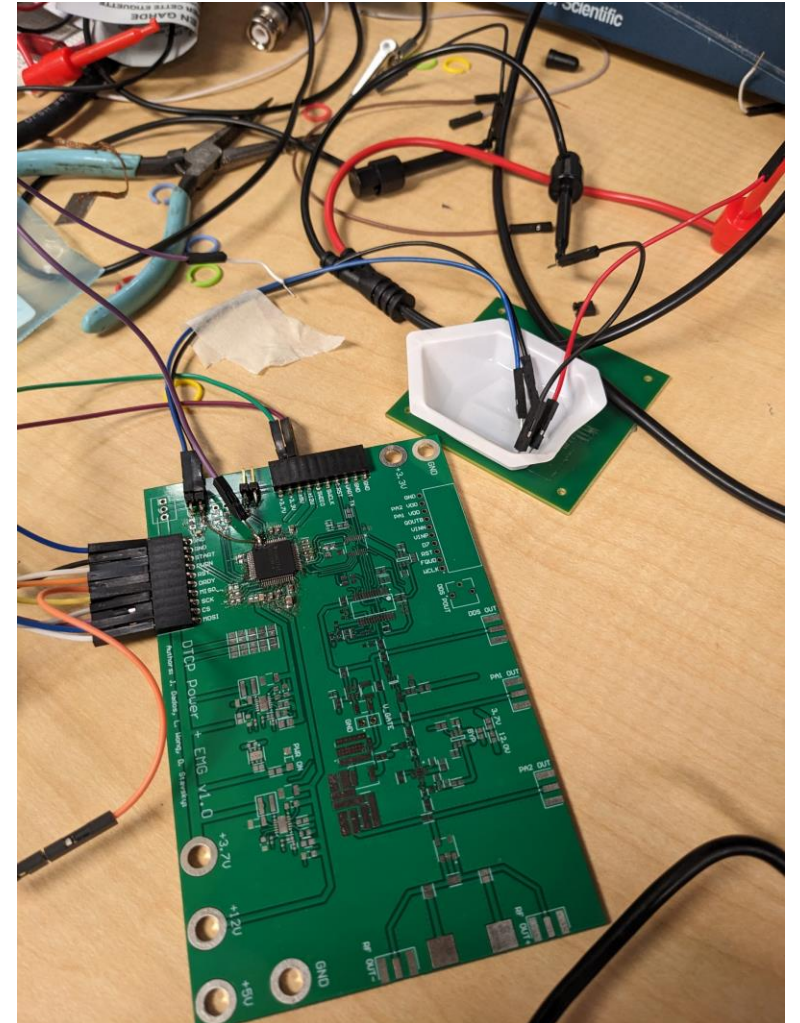
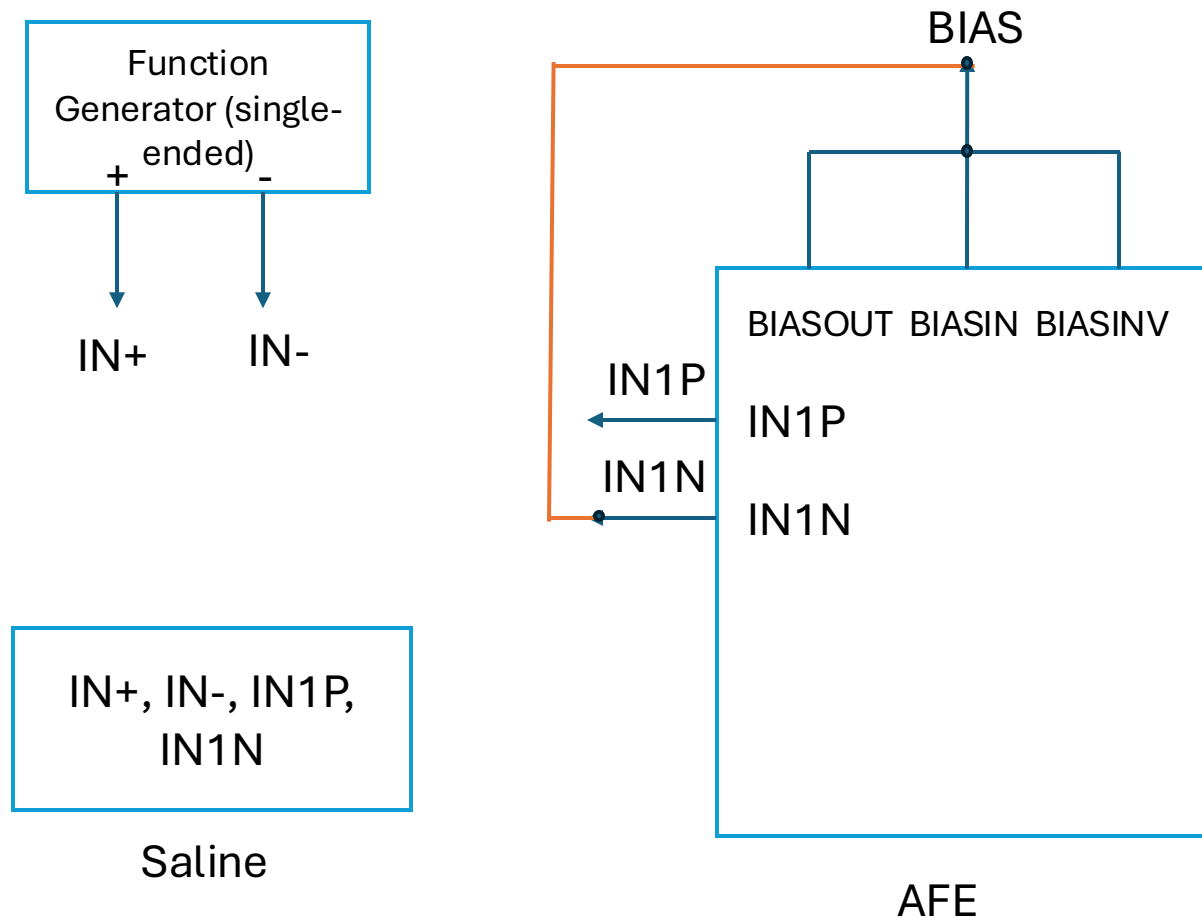
Methods (Filter on the board right now)



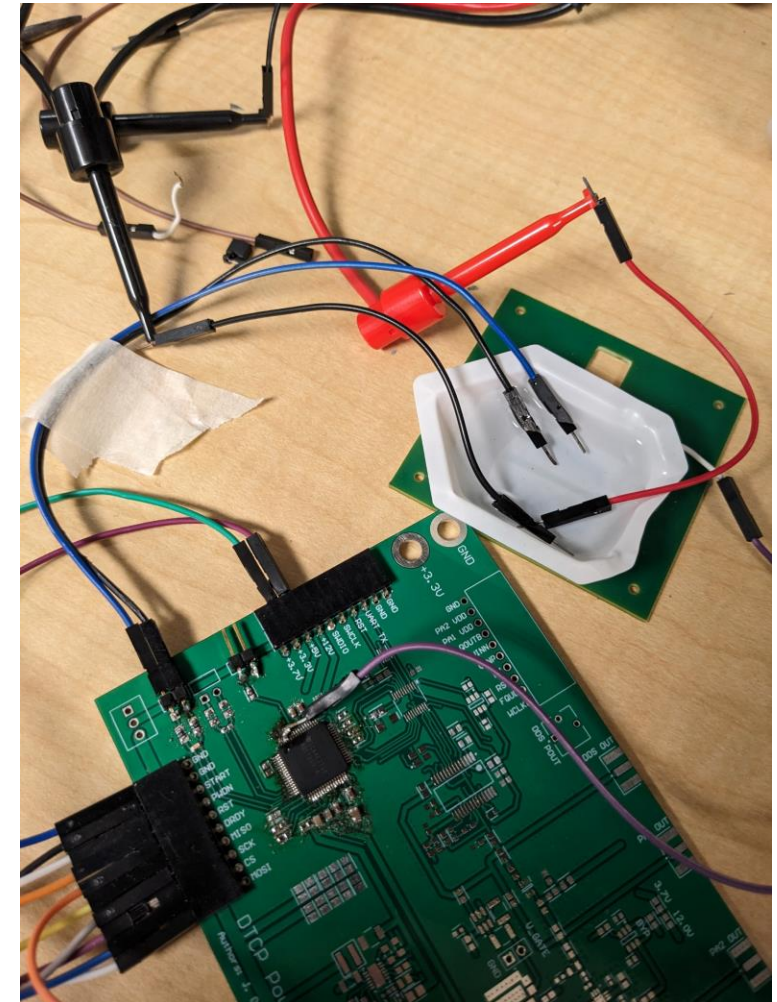
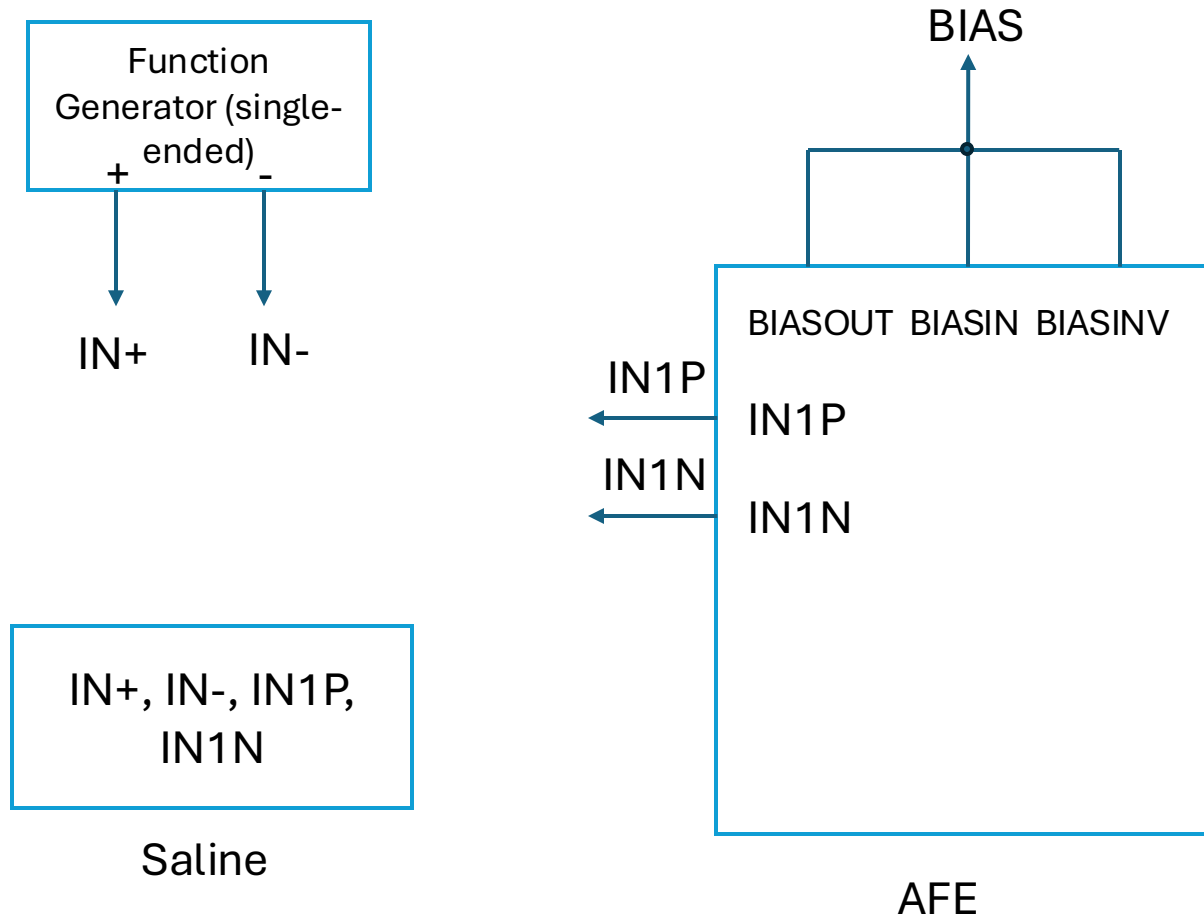
Methods (First Setup; everything in saline)



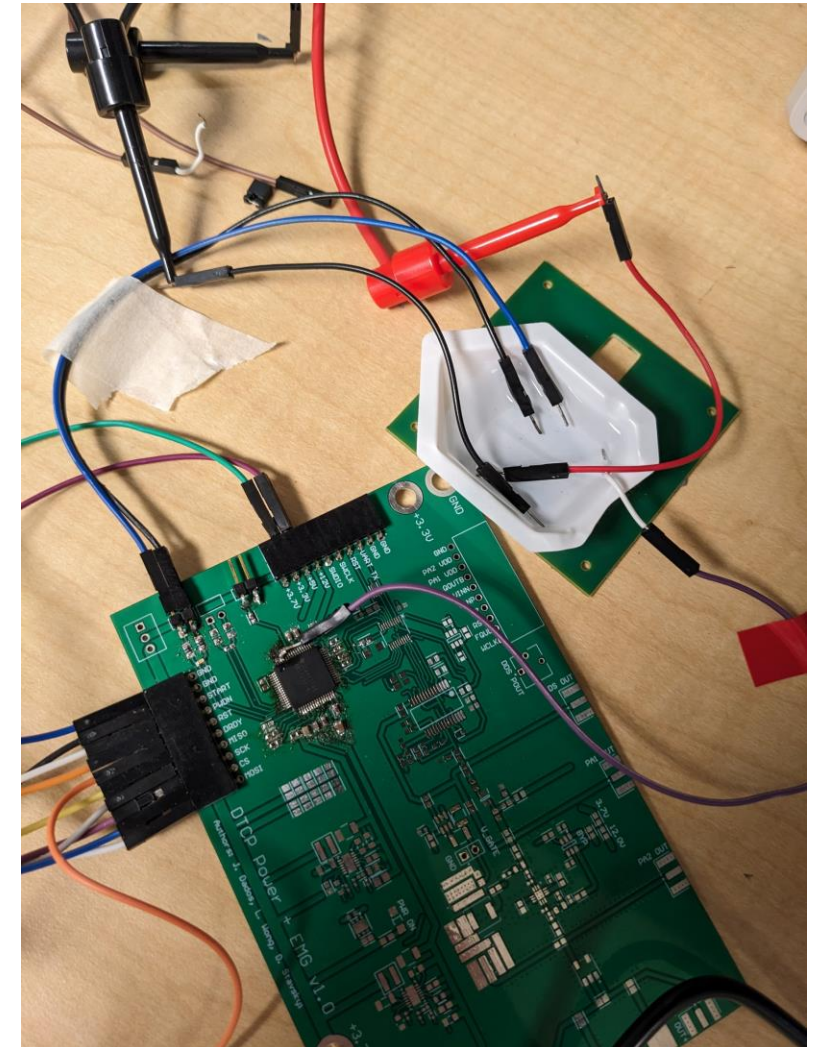
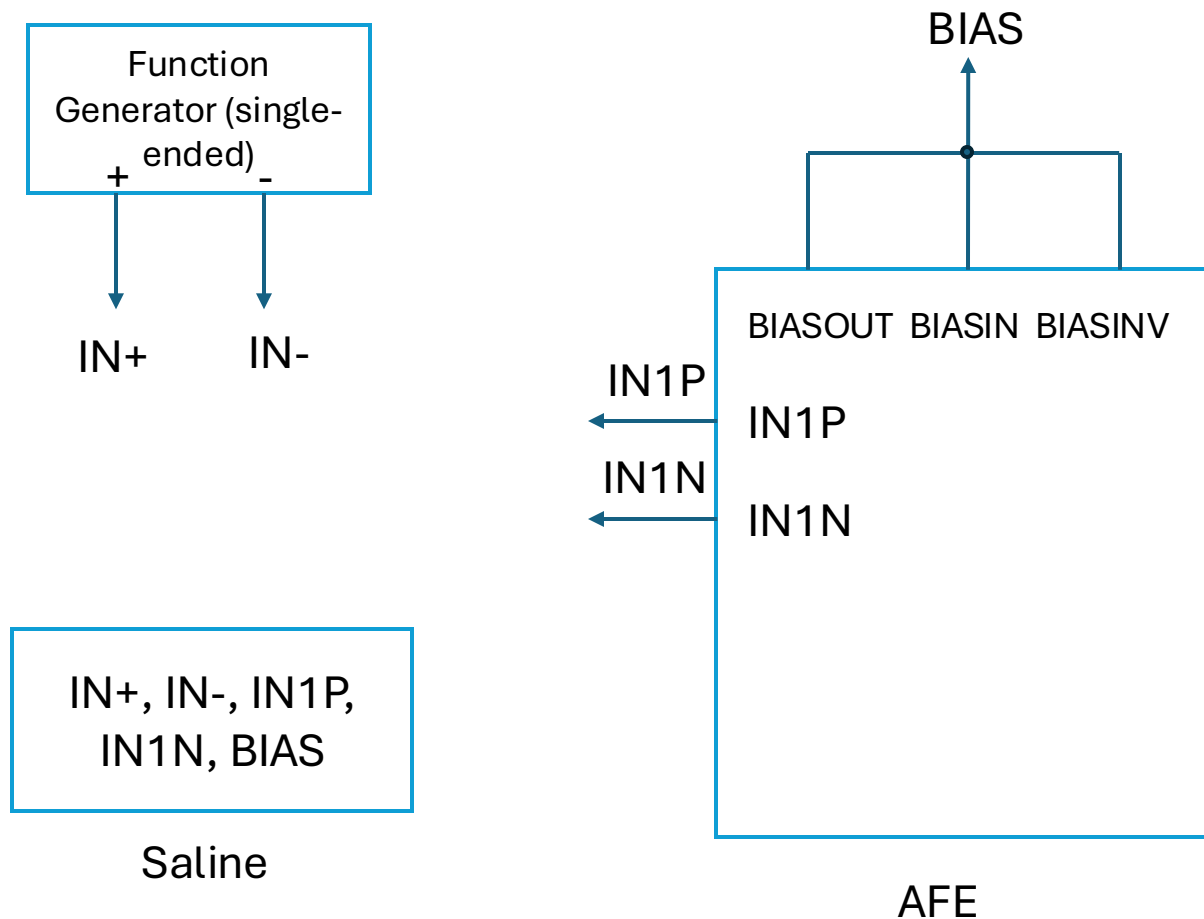
Methods (Second Setup; no bias in saline but shorted to the negative terminal)



Methods (Third Setup; no bias)



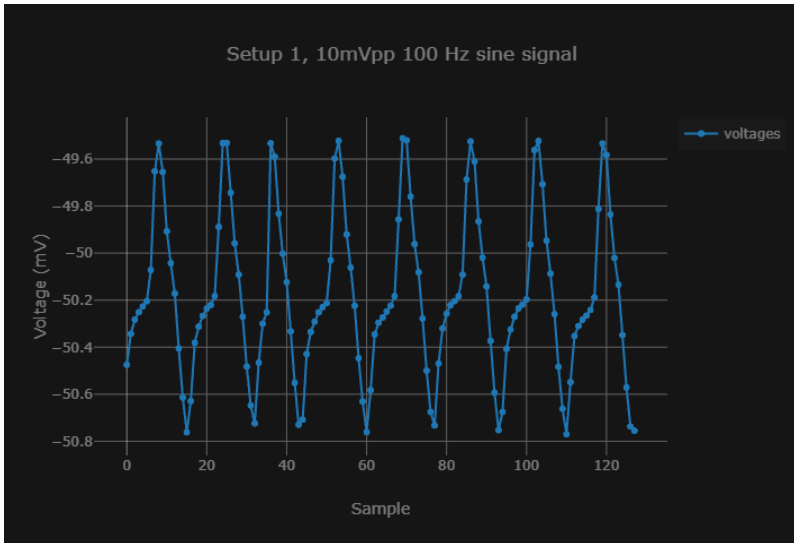
Methods (Fourth Setup; bias in saline but not shorted to the negative terminal)



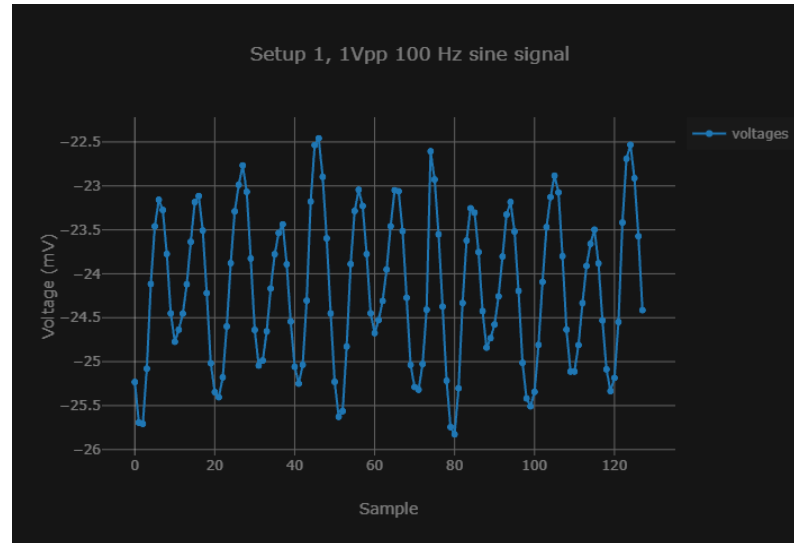
Methodology

- Apply a sinusoidal signal of 100 Hz and of amplitude of 10mVpp, 1Vpp and 2.5Vpp

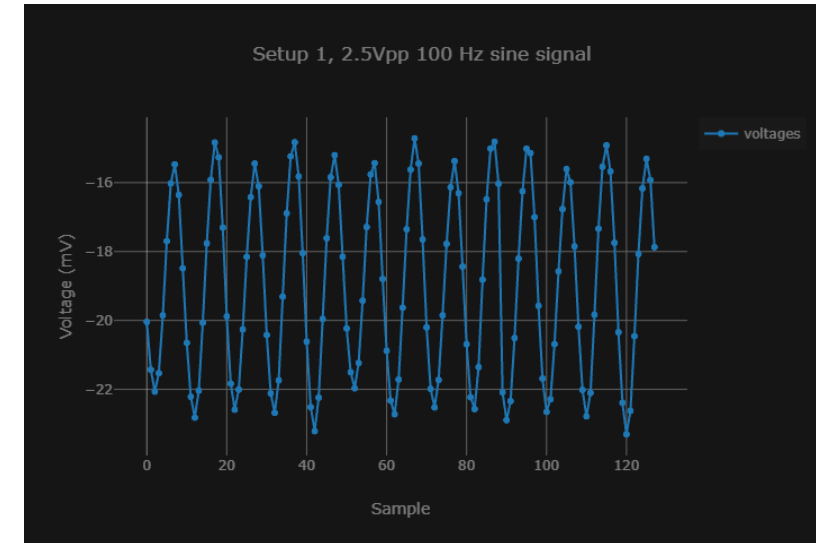
Results (First Setup; everything in saline)



Amplitude: 0.629 mV (noise ?;
double check)

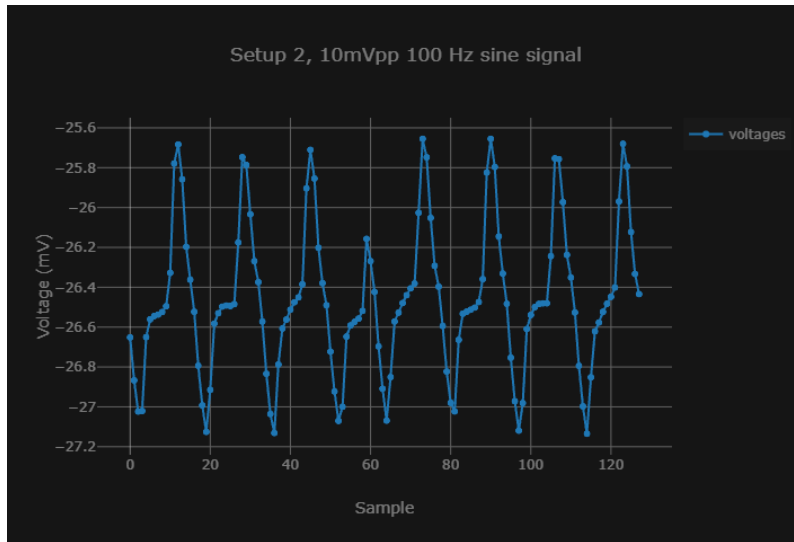


Amplitude: 1.684 mV

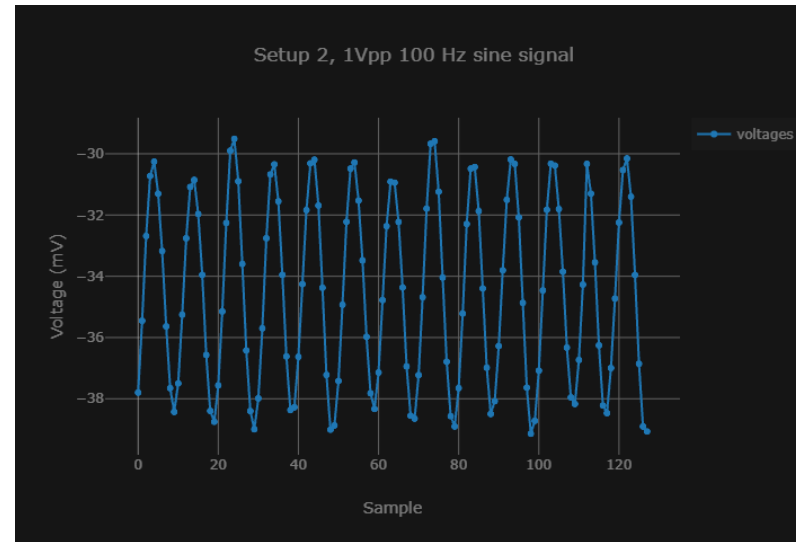


Amplitude: 4.296 mV

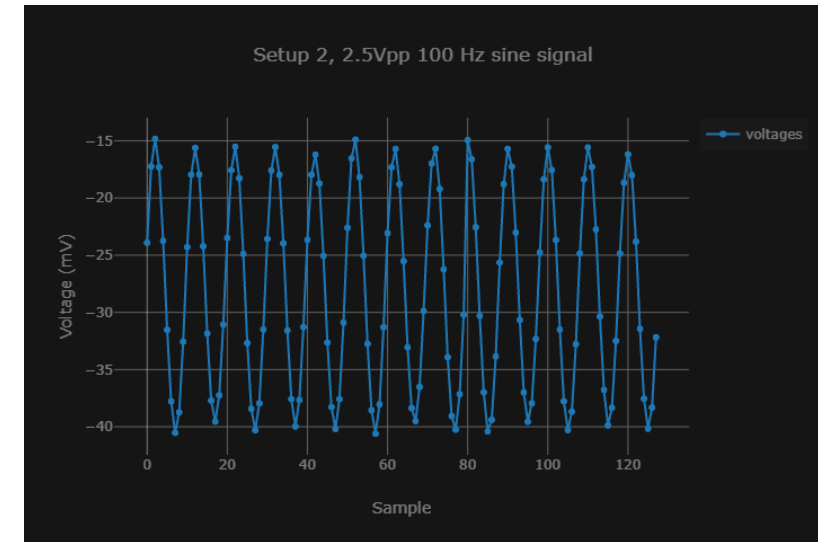
Results (Second Setup; no bias in saline but shorted to the negative terminal)



Amplitude: 0.739 mV
(noise ?)

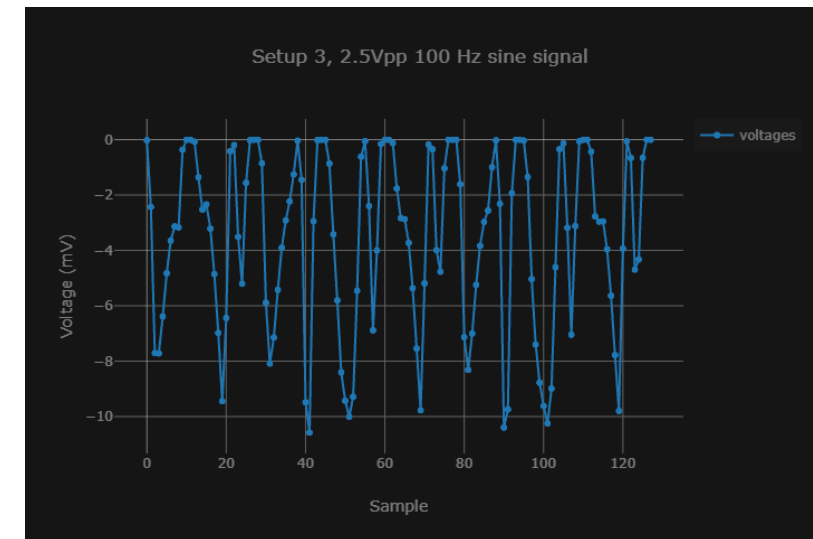
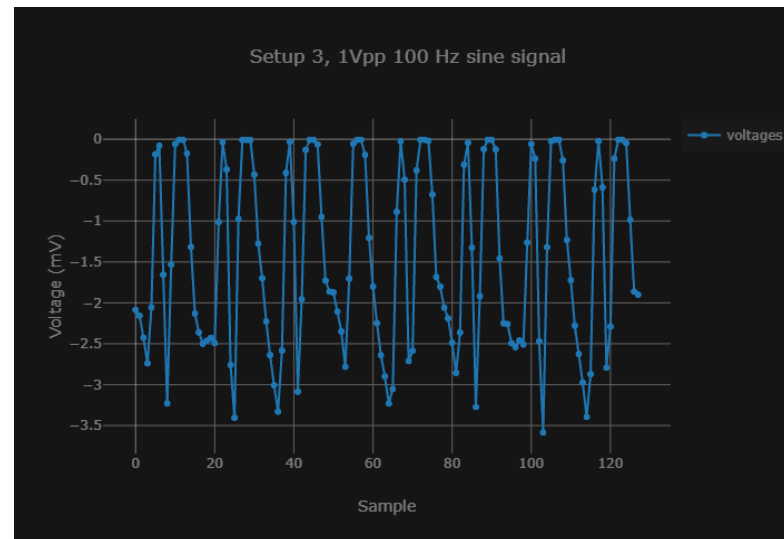
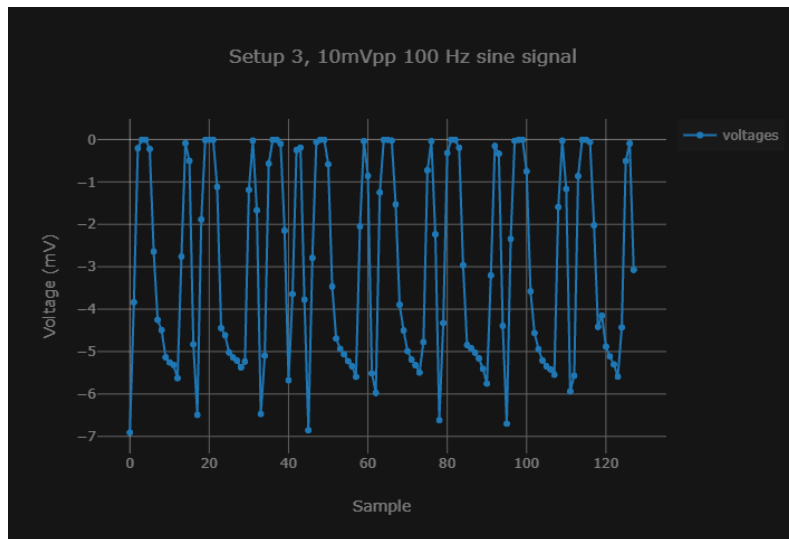


Amplitude: 4.815 mV



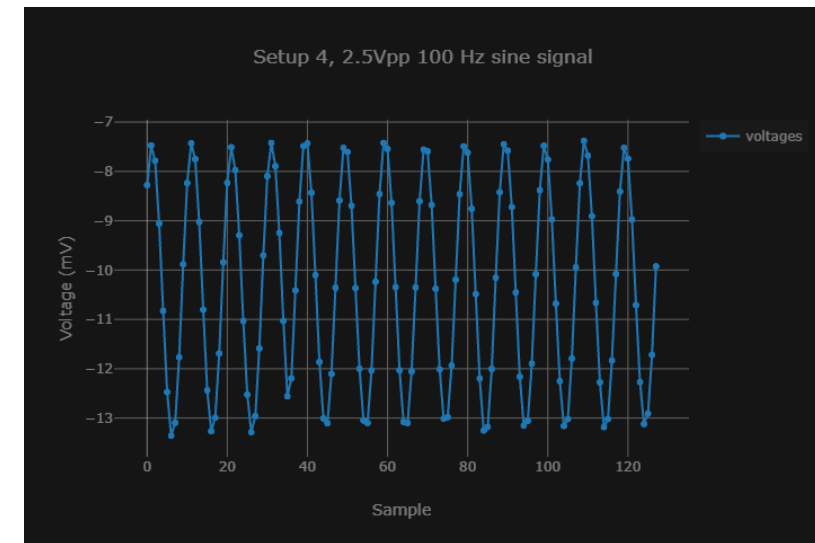
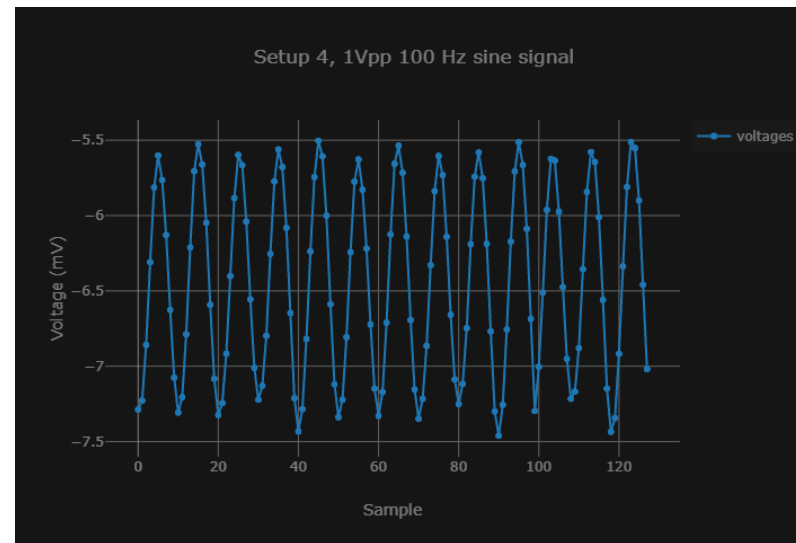
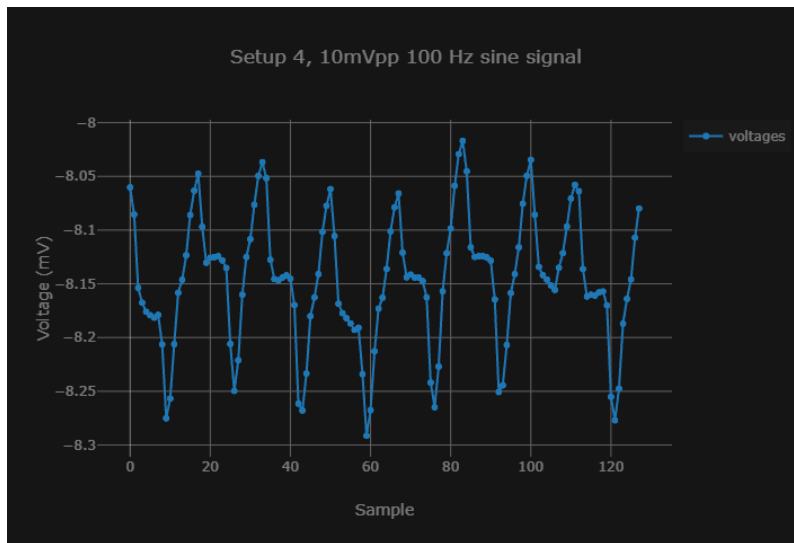
Amplitude: 12.891 mV

Results (Third Setup; no bias)



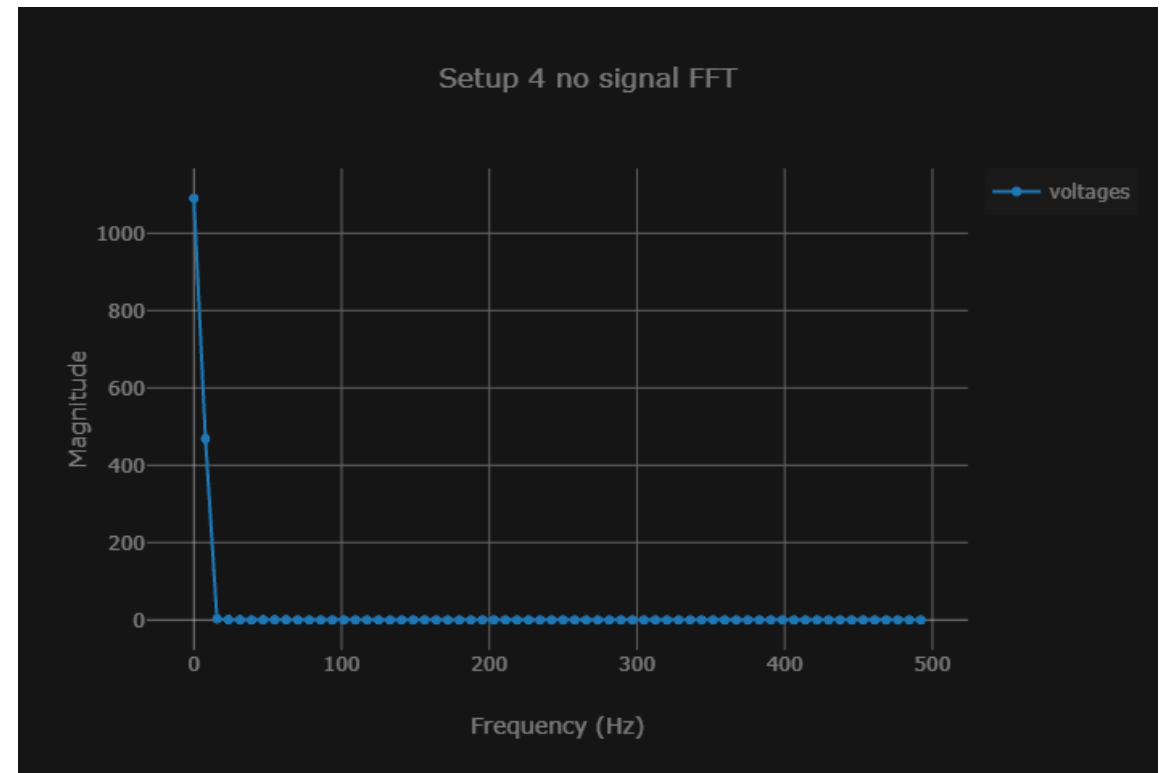
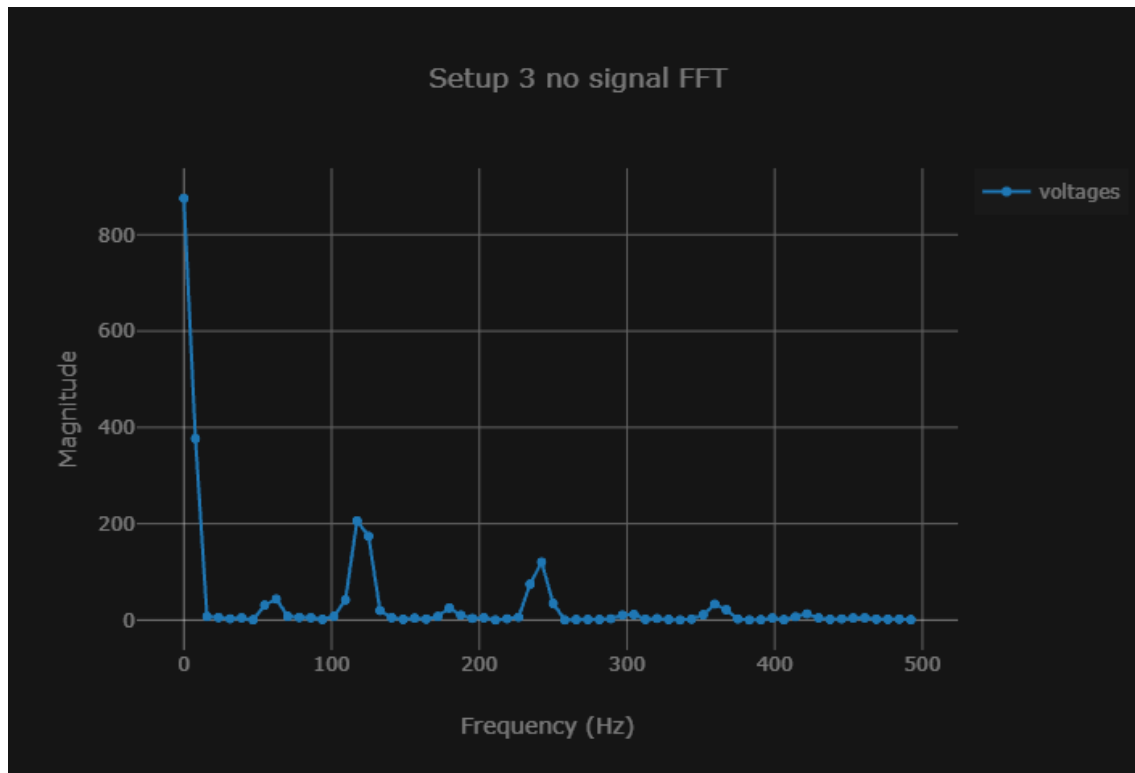
Essentially noise as the shape does not change as the amplitude increases

Results (Fourth Setup; bias in saline but not shorted to the negative terminal)



Observation / Conclusion

- In terms of amplitude, setup 2 seems to be the best, though all setups with biasing recreate a sine wave at sufficiently large signal
- Based on setup 3, where increasing the signal amplitude does not really do anything, some form of biasing is necessary
 - Biasing does help cancel noise as seen in the graphs below



Next steps

- Work with Luis to get real-time measurements with UART
 - Redo tests 1,2 and 4 with SerialPlot
- PCB Review
- Add one more analog channel and test it
- Connect SRB1 internally for bias
 - Drawing for the setup

3/11/2026

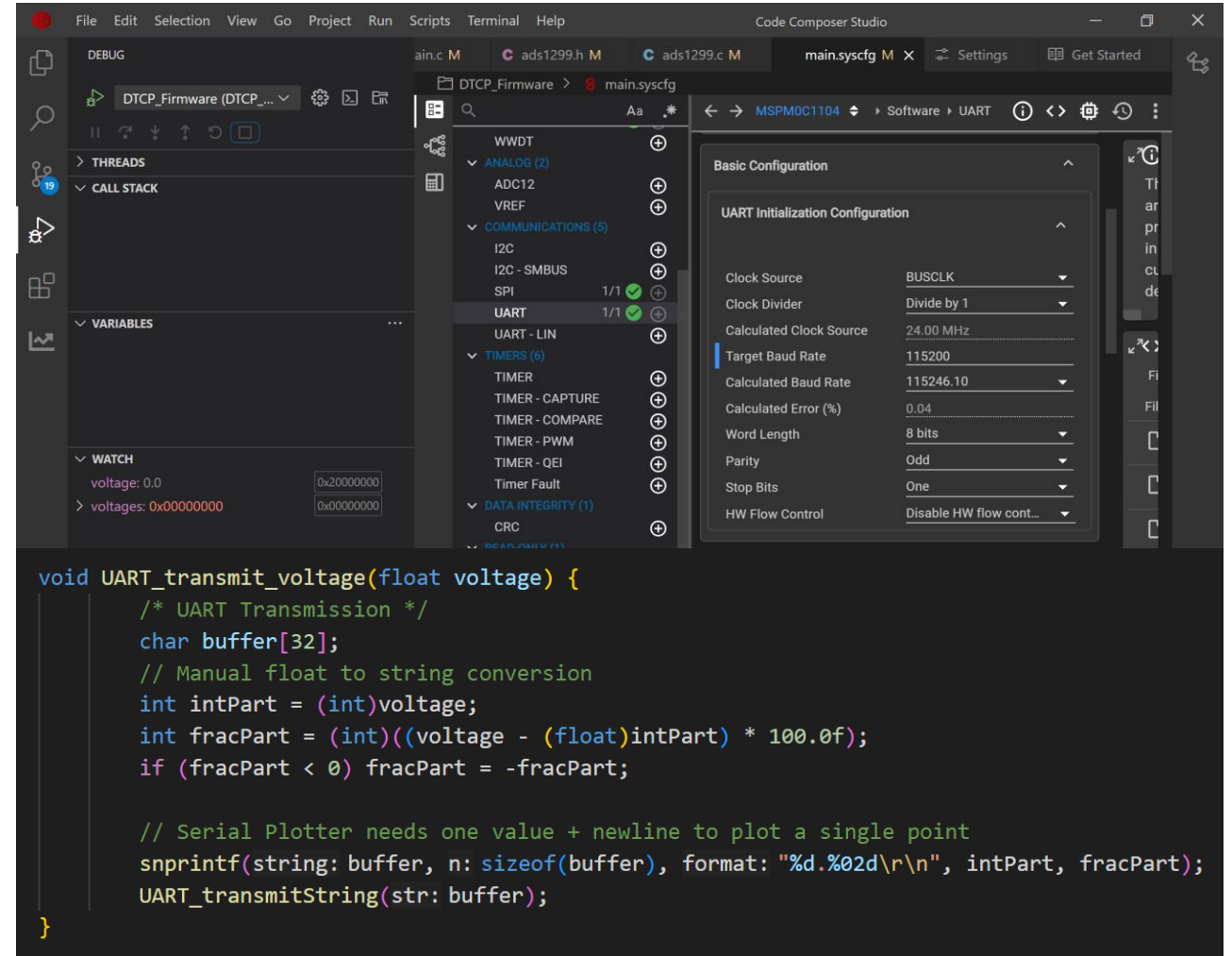
Dmytro Stavskyi, Luis Wong

Objective

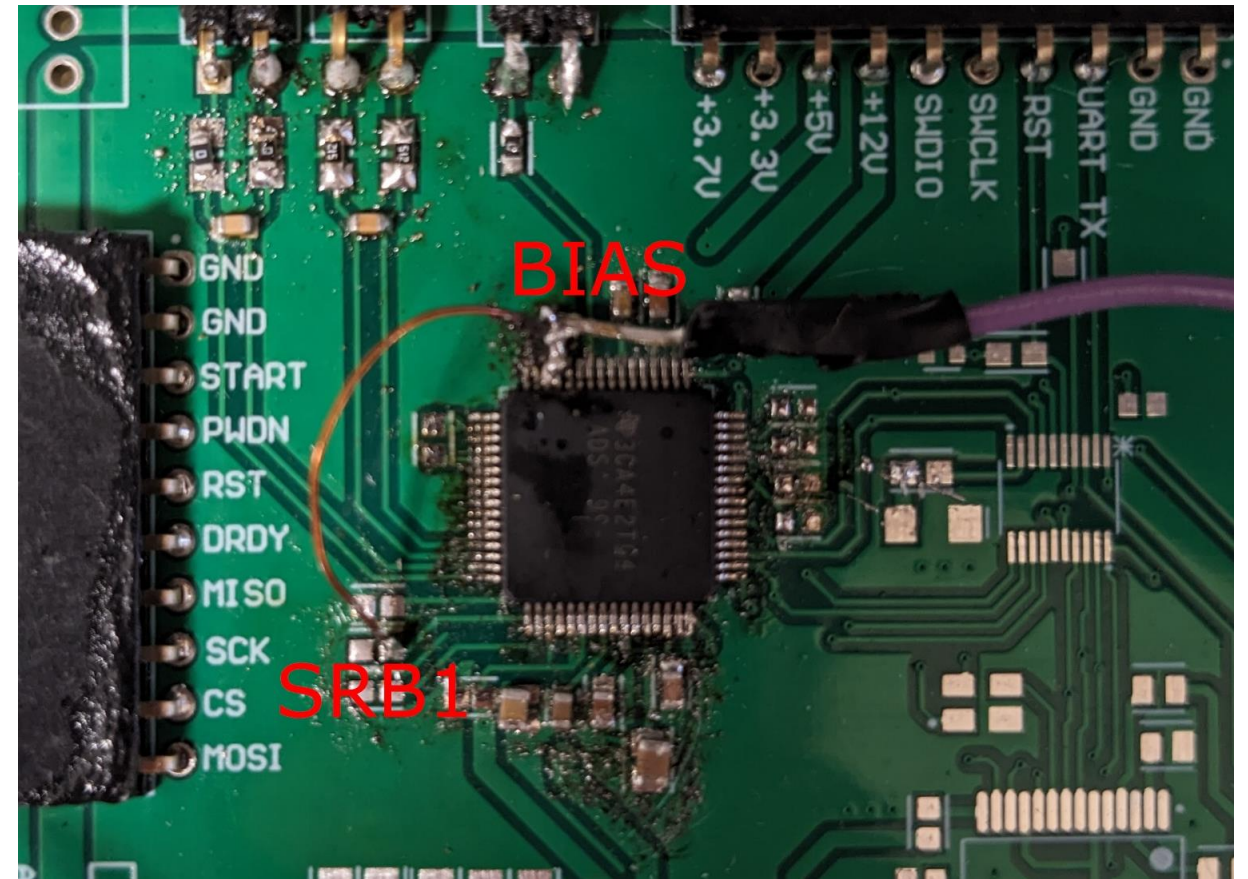
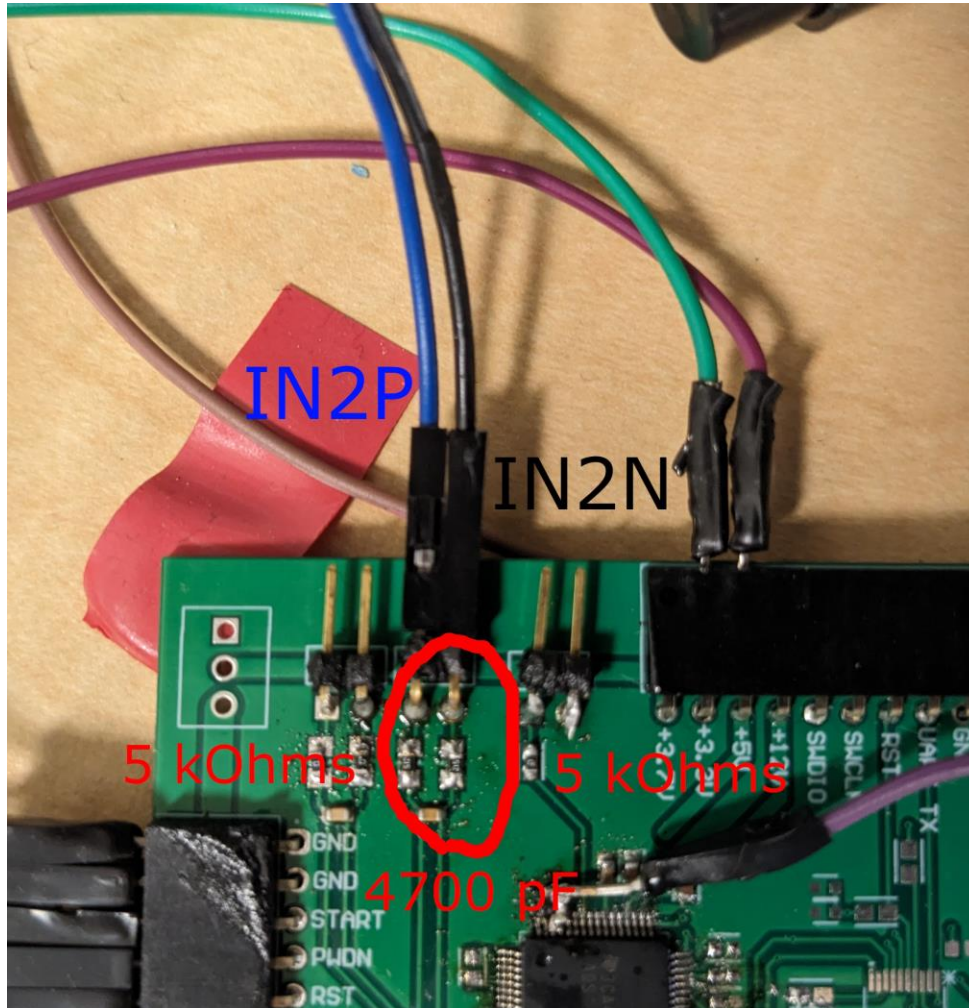
- Test the setup and the AFE measurements with saline in real-time with UART.
- Test the setup with channel 2.
- Test SRB1 (“Patient stimulus, reference, and bias signal 1”; description from the datasheet).
- Try to fix the spikes present in UART plotting.

Methods (UART)

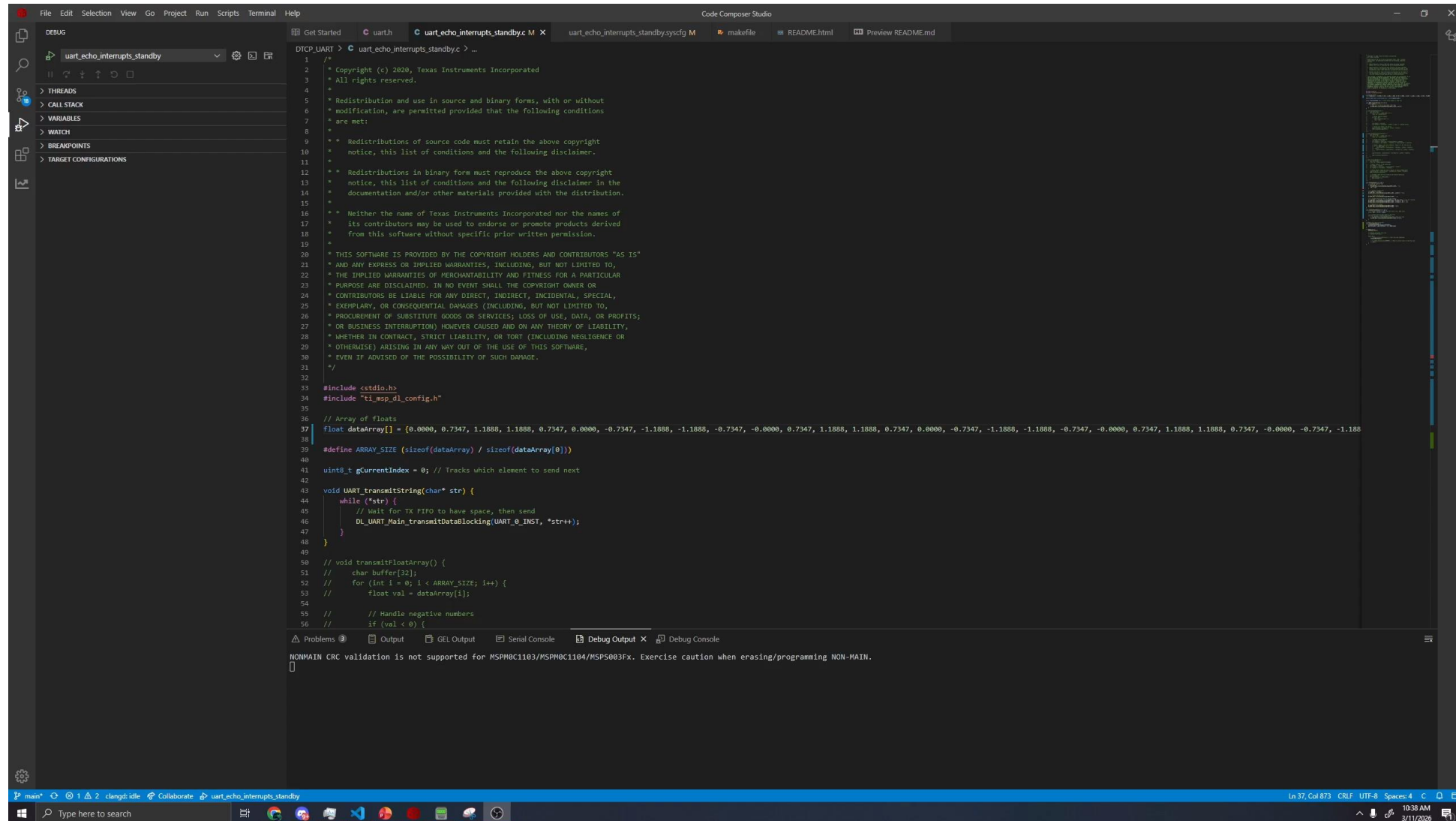
- CCS Sysconfig to change baud rate.
- SerialPlot with ASCII preset for ADS+UART.
- SerialPlot with Binary preset for UART only.
- ADS1299 connected directly to function generator.
 - Similar to setup on slide 37.
- MATLAB to generate test array (100 Hz w/ 2.5 Vpp sampled at 1kHz).



Methods (Second Channel and SRB1)



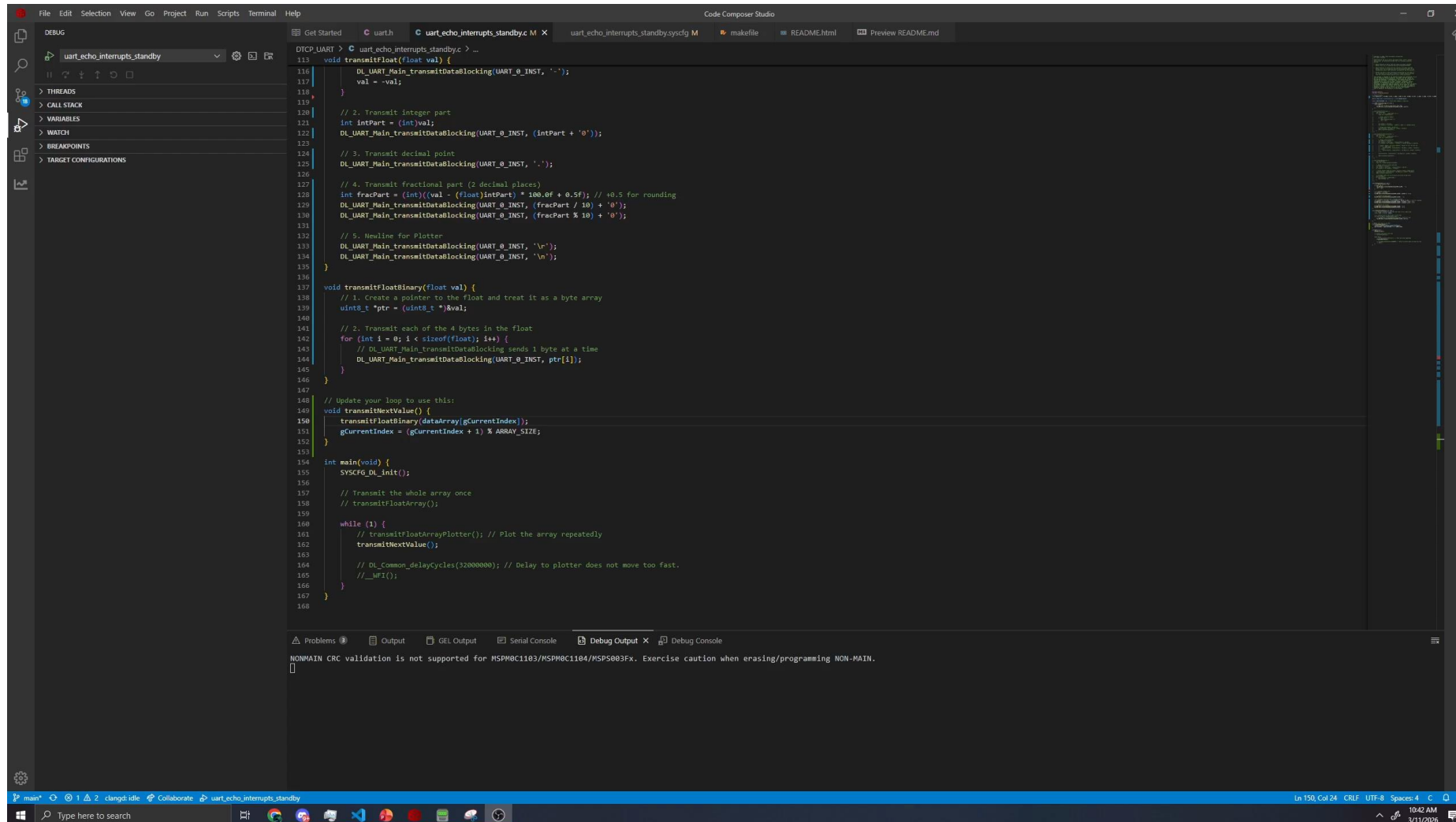
Results (UART w/ Test Array using ASCII)



```
1 /*
2  * Copyright (c) 2020, Texas Instruments Incorporated
3  * All rights reserved.
4  *
5  * Redistribution and use in source and binary forms, with or without
6  * modification, are permitted provided that the following conditions
7  * are met:
8  *
9  * - Redistributions of source code must retain the above copyright
10  *   notice, this list of conditions and the following disclaimer.
11  *
12  * - Redistributions in binary form must reproduce the above copyright
13  *   notice, this list of conditions and the following disclaimer in the
14  *   documentation and/or other materials provided with the distribution.
15  *
16  * Neither the name of Texas Instruments Incorporated nor the names of
17  * its contributors may be used to endorse or promote products derived
18  * from this software without specific prior written permission.
19  *
20  * THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS"
21  * AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO,
22  * THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR
23  * PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT OWNER OR
24  * CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL,
25  * EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO,
26  * PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS;
27  * OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY,
28  * WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR
29  * OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE,
30  * EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
31  */
32
33 #include <stdio.h>
34 #include "ti_msp_dl_config.h"
35
36 // Array of floats
37 float dataArray[] = {0.0000, 0.7347, 1.1888, 1.1888, 0.7347, 0.0000, -0.7347, -1.1888, -1.1888, -0.7347, -0.0000, 0.7347, 1.1888, 1.1888, 0.7347, 0.0000, -0.7347, -1.1888, -1.1888, -0.7347, -0.0000, 0.7347, 1.1888, 1.1888, 0.7347, -0.0000, -0.7347, -1.1888}
38
39 #define ARRAY_SIZE (sizeof(dataArray) / sizeof(dataArray[0]))
40
41 uint8_t gCurrentIndex = 0; // Tracks which element to send next
42
43 void UART_transmitString(char* str) {
44     while (*str) {
45         // Wait for TX FIFO to have space, then send
46         DL_UART_Main_TransmitDataBlocking(UART_0_INST, *str++);
47     }
48 }
49
50 // void transmitFloatArray() {
51 //     char buffer[32];
52 //     for (int i = 0; i < ARRAY_SIZE; i++) {
53 //         float val = dataArray[i];
54 //
55 //         // Handle negative numbers
56 //         if (val < 0) {
```

- No UART Spikes when running the UART only code.
- Could be because MCU only needs to worry about one thing unlike the code in the real setup where it needs to manage SPI to communicate with the ADS1299.
- Setup: Just the LaunchPad by itself.

Results (UART w/ Test Array using Binary)



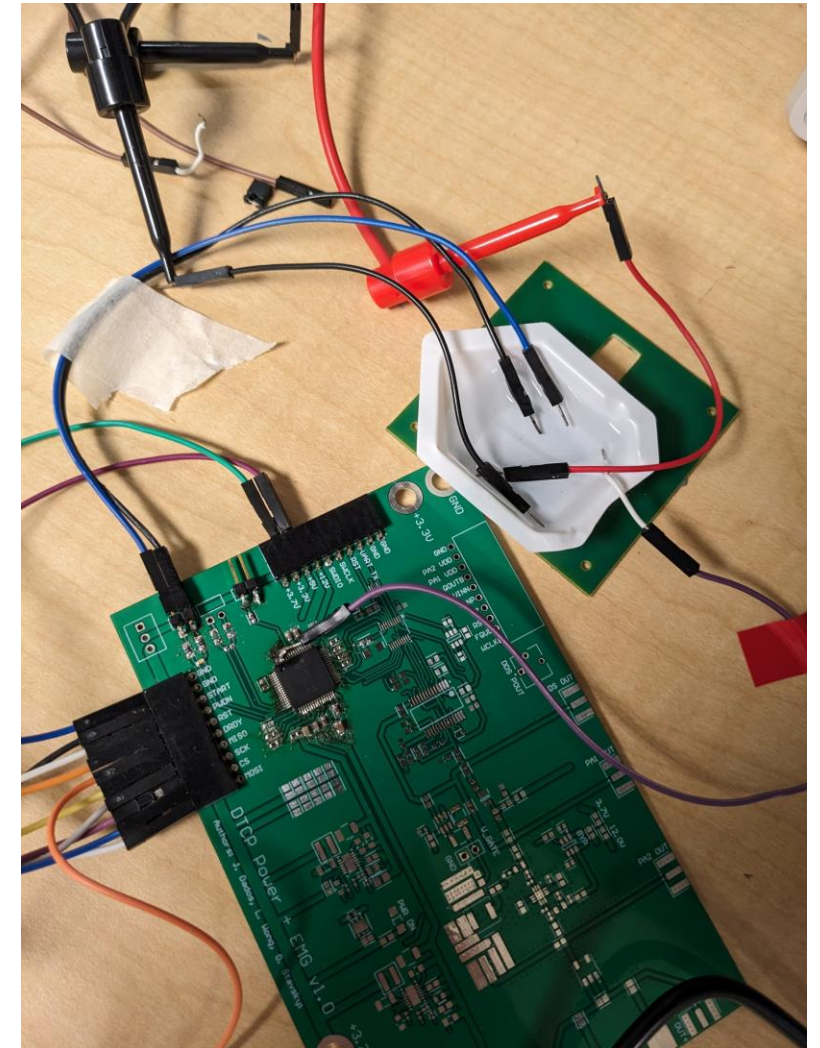
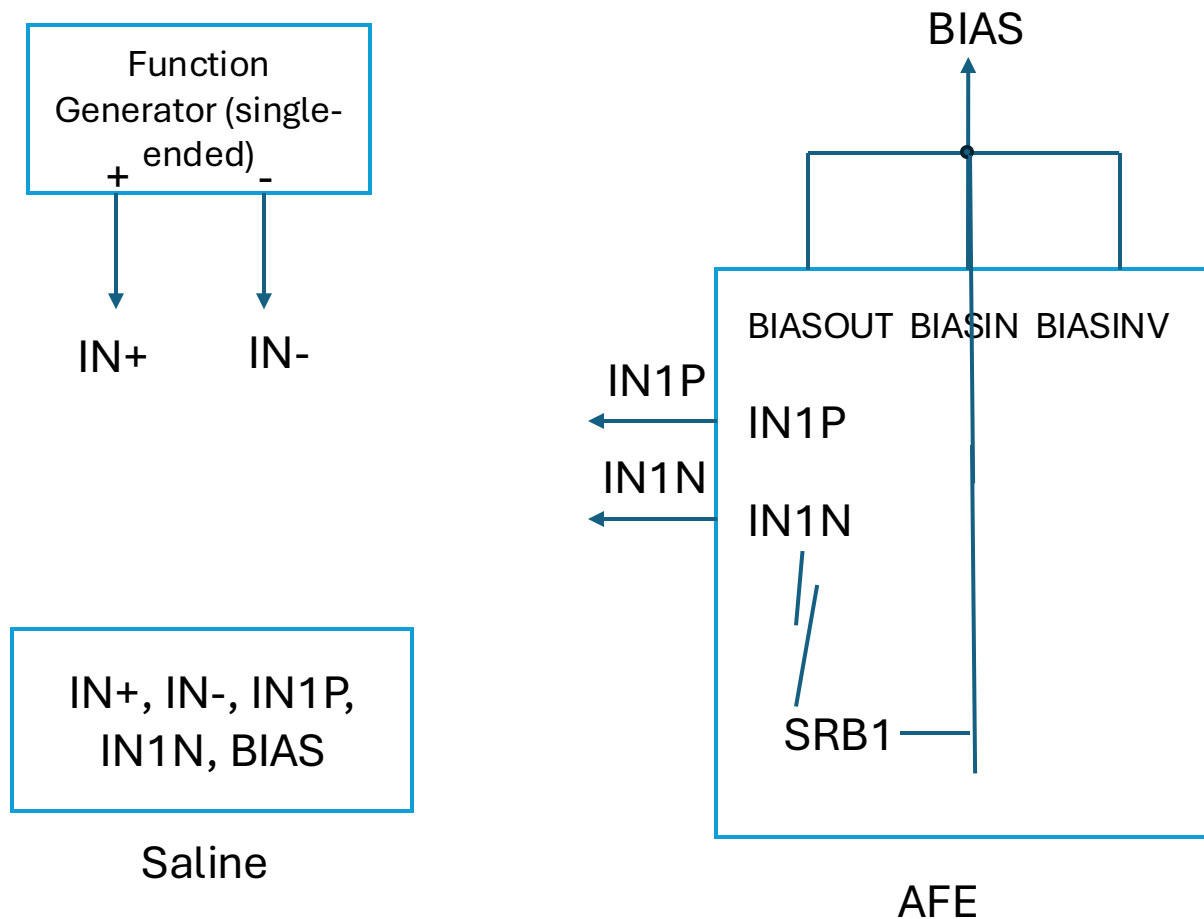
```
113 void transmitFloat(float val) {
114     DL_UART_Main_transmitDataBlocking(UART_0_TINST, '-');
115     val = -val;
116 }
117
118 // 2. Transmit integer part
119 int intPart = (int)val;
120 DL_UART_Main_transmitDataBlocking(UART_0_TINST, (intPart + '0'));
121
122 // 3. Transmit decimal point
123 DL_UART_Main_transmitDataBlocking(UART_0_TINST, '.');
124
125 // 4. Transmit fractional part (2 decimal places)
126 int fracPart = (int)((val - (float)intPart) * 100.0f + 0.5f); // +0.5f for rounding
127 DL_UART_Main_transmitDataBlocking(UART_0_TINST, (fracPart / 10) + '0');
128 DL_UART_Main_transmitDataBlocking(UART_0_TINST, (fracPart % 10) + '0');
129
130 // 5. Header for Plotter
131 DL_UART_Main_transmitDataBlocking(UART_0_TINST, 'r');
132 DL_UART_Main_transmitDataBlocking(UART_0_TINST, '\n');
133 }
134
135 void transmitFloatBinary(float val) {
136     // 1. Create a pointer to the float and treat it as a byte array
137     uint8_t *ptr = (uint8_t *)&val;
138
139     // 2. Transmit each of the 4 bytes in the float
140     for (int i = 0; i < sizeof(float); i++) {
141         // DL_UART_Main_transmitDataBlocking sends 1 byte at a time
142         DL_UART_Main_transmitDataBlocking(UART_0_TINST, ptr[i]);
143     }
144 }
145
146 // Update your loop to use this:
147 void transmitNextValue() {
148     transmitFloatBinary(dataArray[gCurrentIndex]);
149     gCurrentIndex = (gCurrentIndex + 1) % ARRAY_SIZE;
150 }
151
152 int main(void) {
153     SYS_CFG_Init();
154
155     // Transmit the whole array once
156     transmitFloatArray();
157
158     while (1) {
159         // transmitFloatArrayPlotter(); // Plot the array repeatedly
160         transmitNextValue();
161
162         // DL_Common_delayCycles(32000000); // Delay to plotter does not move too fast.
163         // __NOP();
164     }
165 }
166
167 }
168
```

Problems | Output | GEL Output | Serial Console | Debug Output | Debug Console

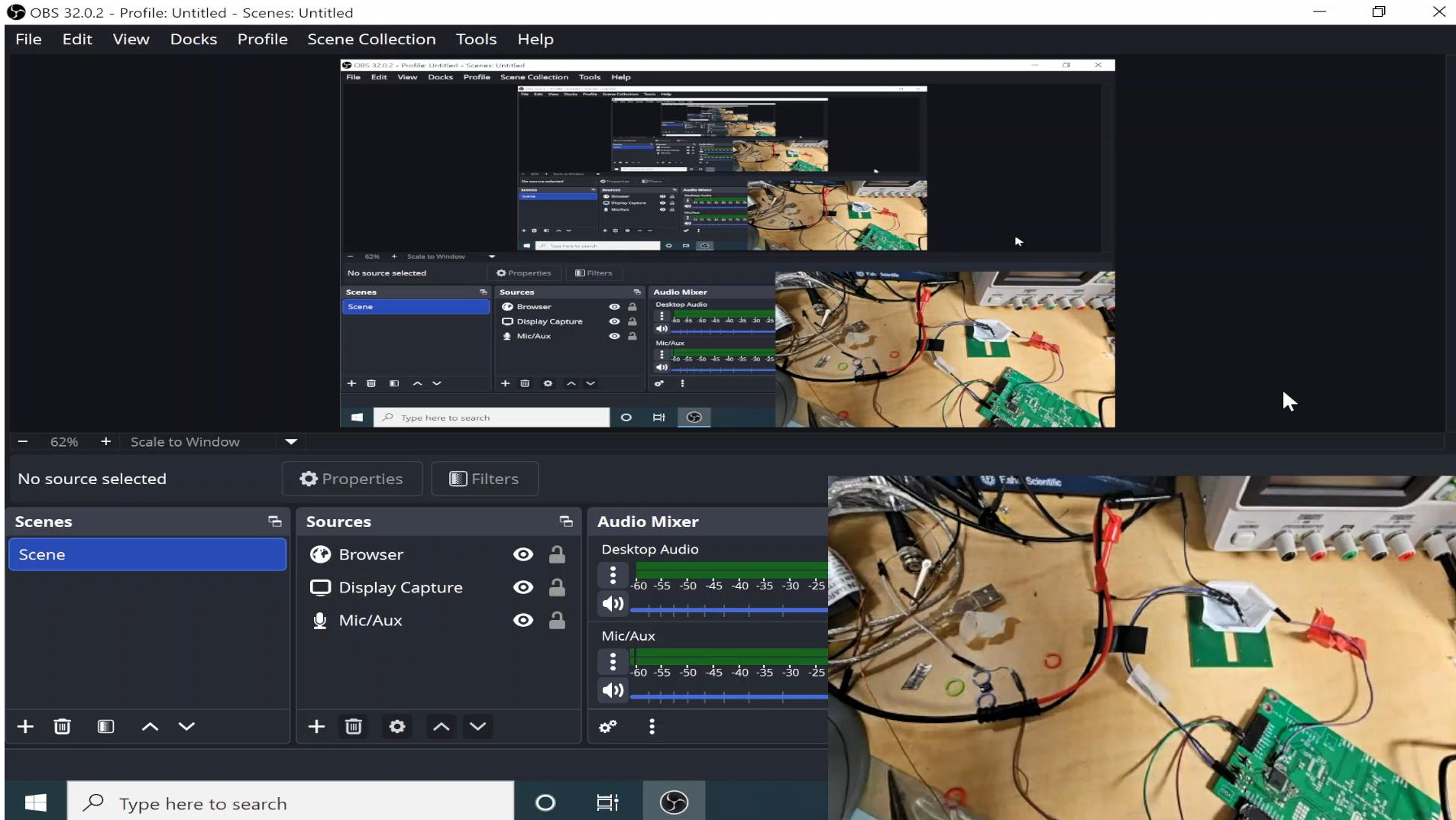
NONMAIN CRC validation is not supported for MSPM0C1103/MSPM0C1104/MSP5003FX. Exercise caution when erasing/programming NON-MAIN.

- Next time we test UART+ADS we can try sending raw binary values to see if reducing the strain on the MCU could reduce the spikes we see.
- 4 bytes (w/o header or checksum) vs 4-10+ bytes (delimited w/ \n + conversion).
- Setup: Just the LaunchPad by itself.

Methods (Fifth Setup; bias in saline but not shorted to the negative terminal)

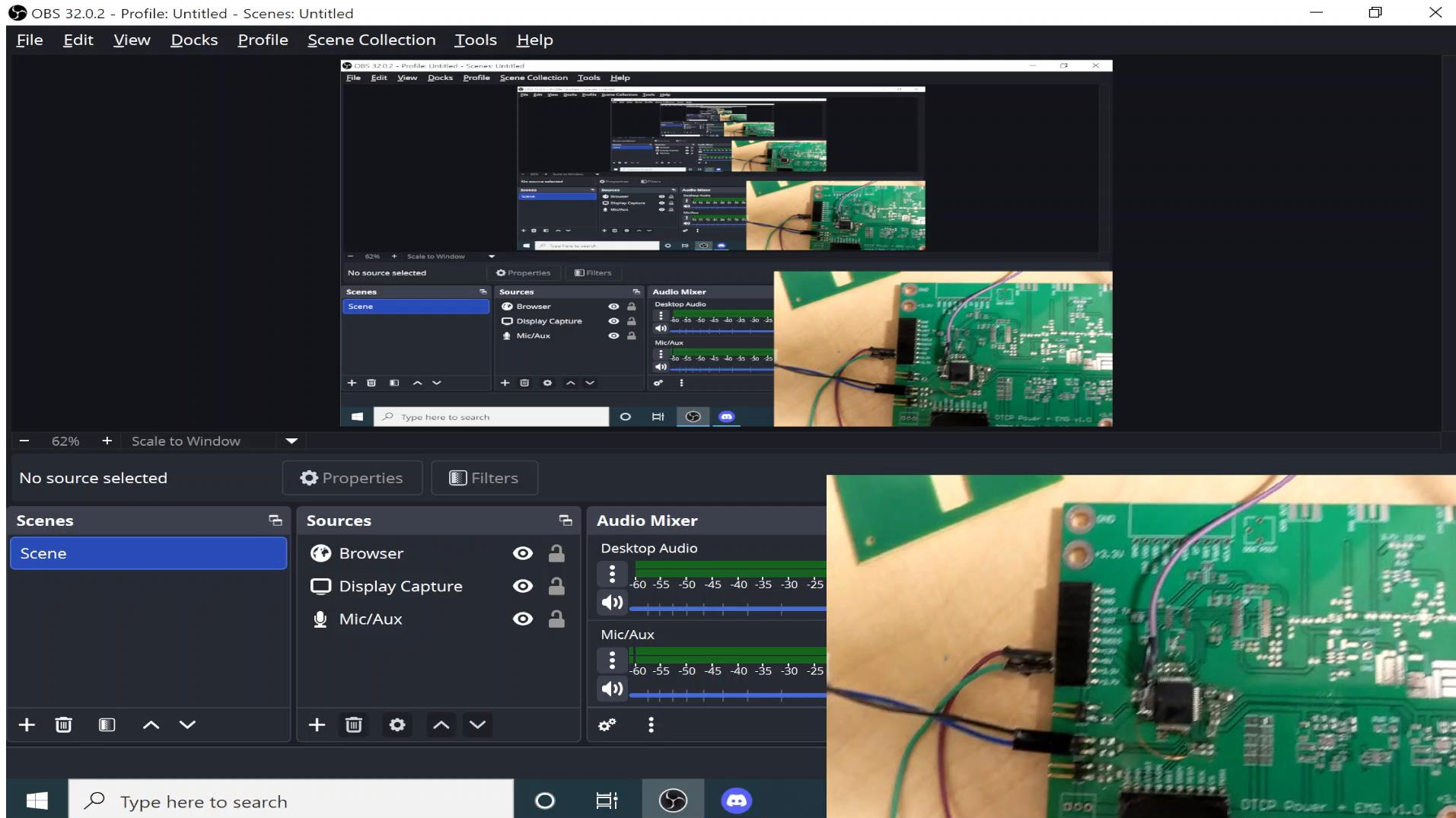


Results (Fifth Setup with channel 1)



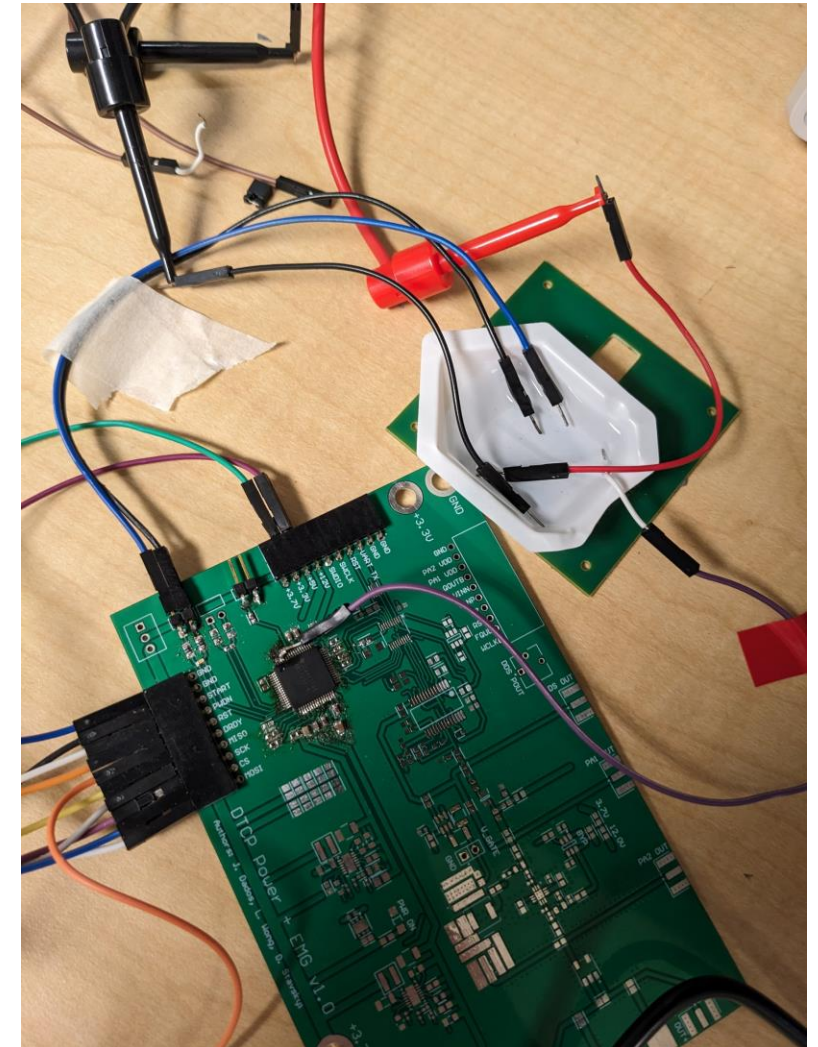
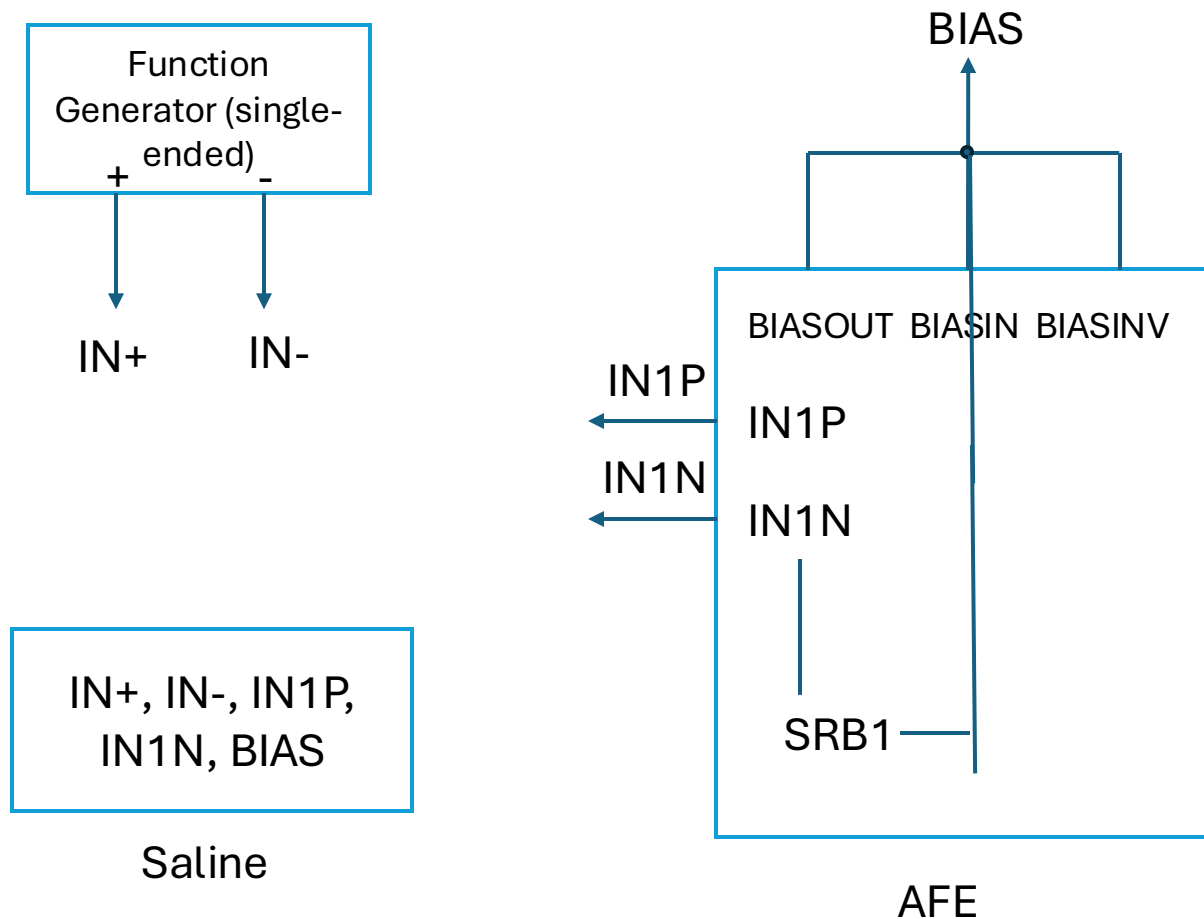
- Methodology:
Apply a sinusoidal signal of 100 Hz and of amplitude of 10mVpp, 1Vpp and 2.5Vpp
- The spikes are much bigger than the actual measurement values which increases the range

Results (Fifth Setup with channel 2)

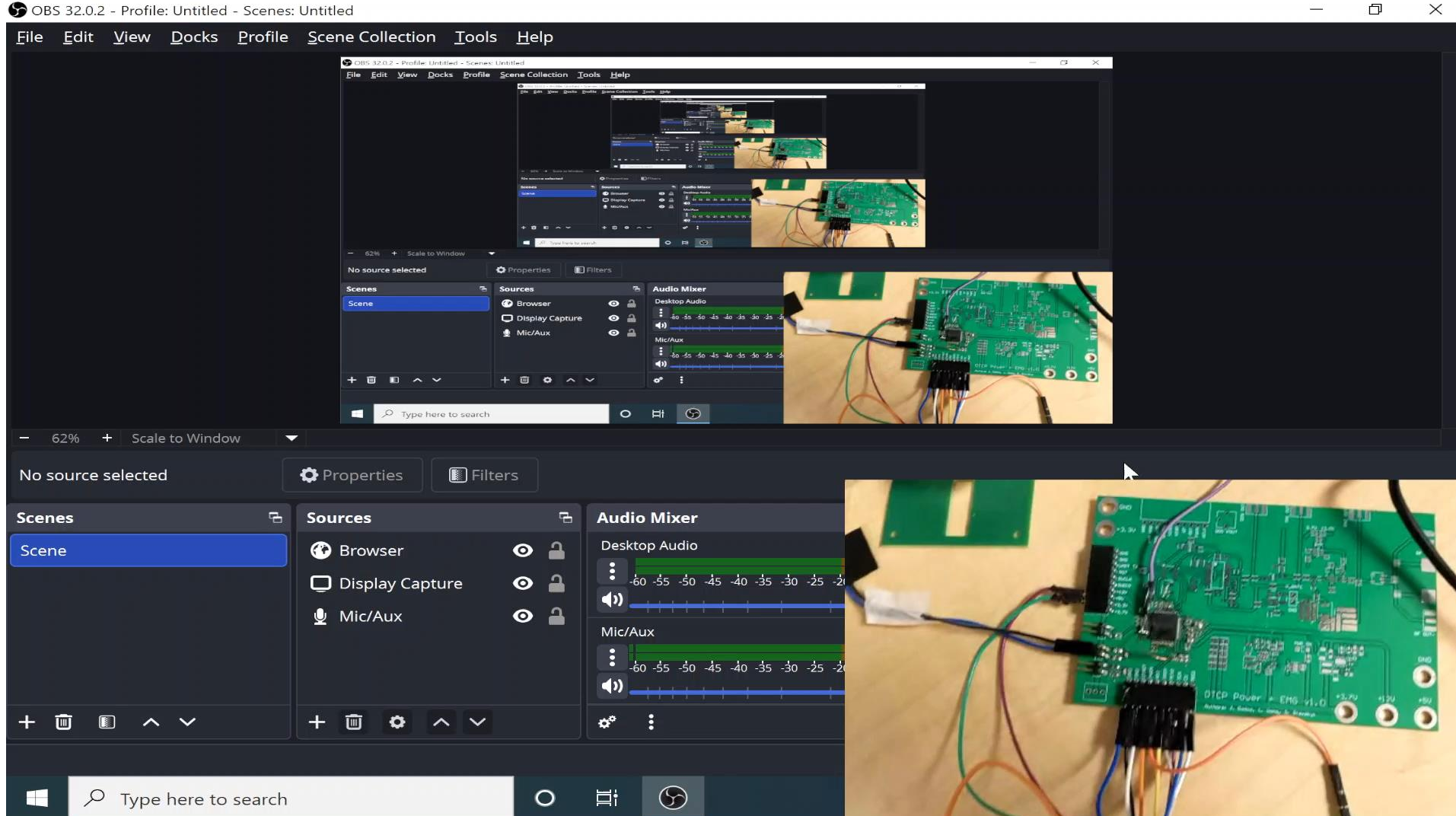


- Methodology:
Apply a sinusoidal signal of 100 Hz and of amplitude of 10mVpp, 1Vpp and 2.5Vpp
- Can make measurements with channel 2

Methods (Sixth Setup; bias in saline but not shorted to the negative terminal)



Results (Sixth Setup with channel 2 and SRB1)



- Same setup as in the previous slide but with SRB1 pin enabled in the firmware, meaning that all negative inputs of the channels are connected to SRB1
- Nothing changes as the amplitude of the input signal increases
- Because of the spikes, it is not easy to see, but it saturates to -187.5 mV (tested with CCS graph)

Observation / Conclusion (SRB1)

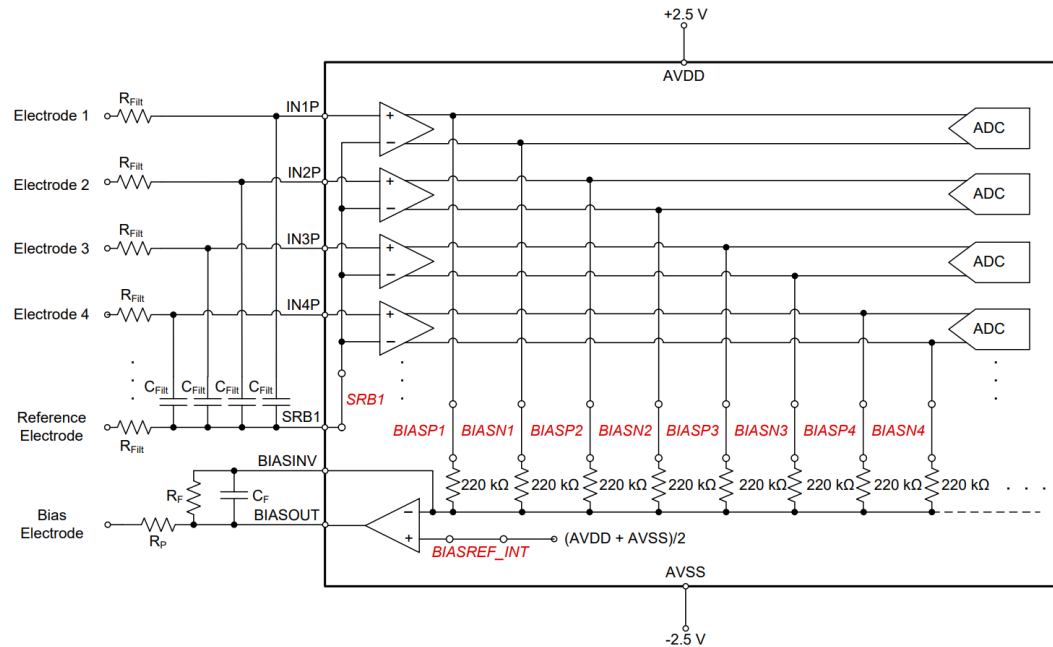
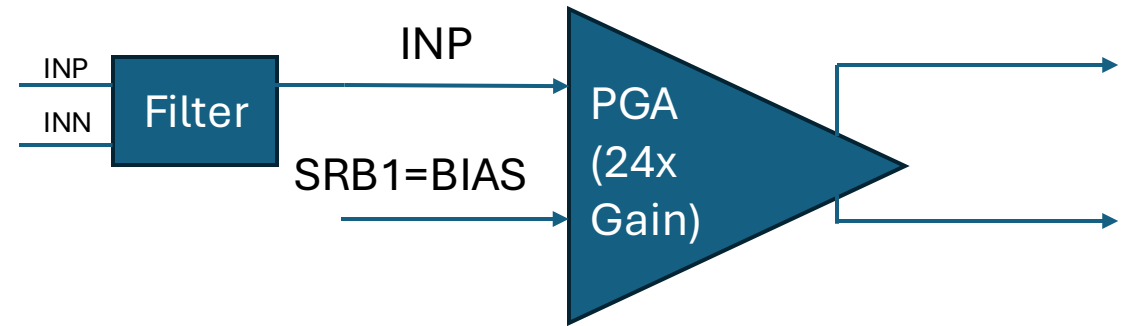


Figure 73. Example Schematic Using the ADS1299 in an EEG Data Acquisition Application, Referential Montage

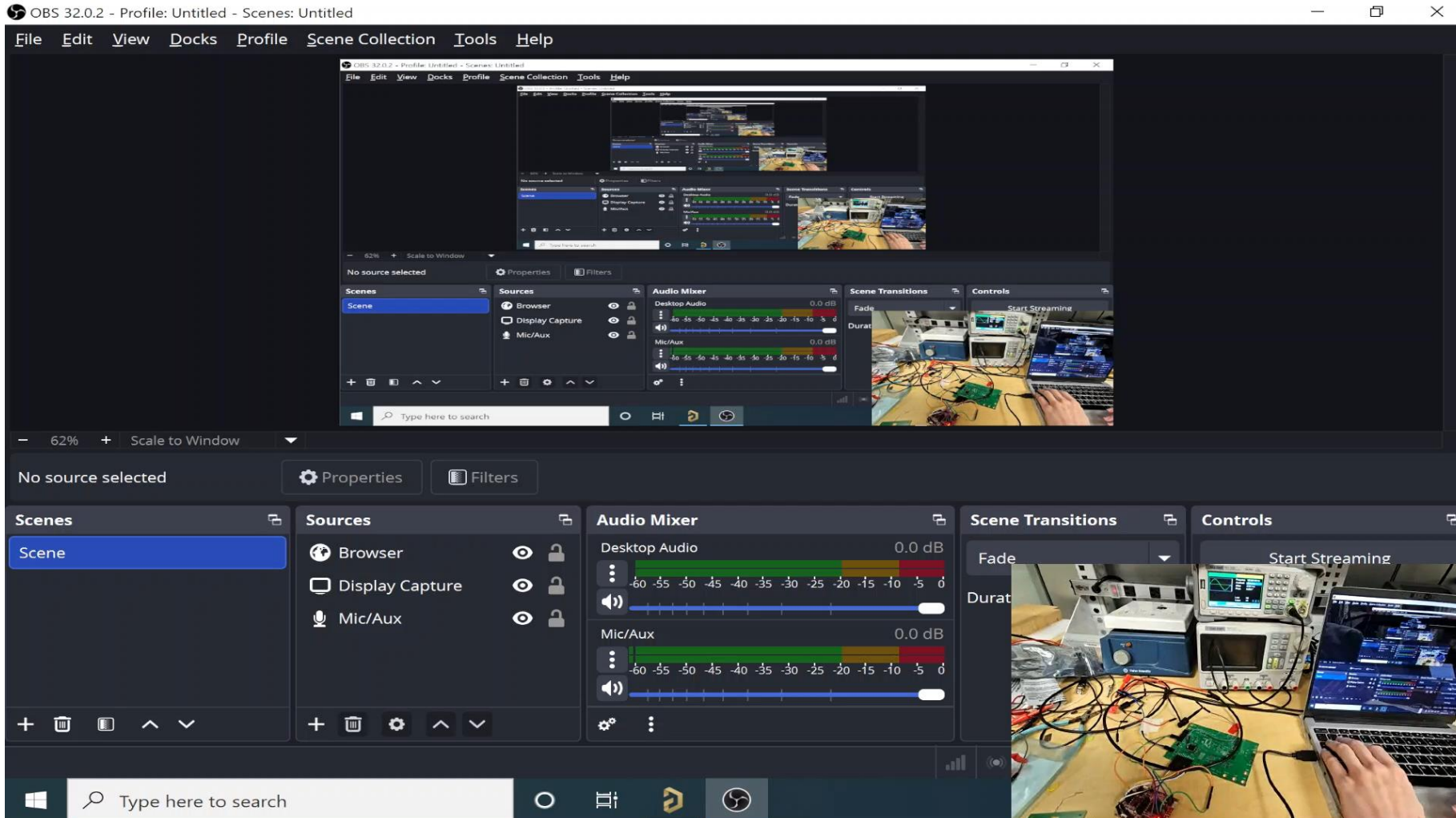
Application example that uses SRB1 from ADS1299 datasheet



What we have if we enable SRB1 on the current board

- The positive input needs to be biased as well

Observation / Conclusion (UART Spikes)



- Setup: no saline, function generator directly connected to analog input channel with bias
- In this particular case, increasing the baud rate to 115200 removed the spikes (but spikes are still present when doing measurements with saline)
- Adding parity or additional stop bit did not get rid of the spikes

Next steps

- Try UART again but send raw binary values of floats instead of sending ASCII values over UART to see if it resolves the spiking as it could be an issue of CPU stalling.
 - Also bypasses the need to manually translate the float to a string which would eliminate potential issues that could occur there.
- Test SRB1 more
 - Remove short between bias and SRB
- UART through Python (PySerial) for data processing
- Load an example EMG signal (from some dataset; with a spike) into the function generator
- Spike detection code (print inside the MCU to determine if its UART or the real data)